The Linux System Administrators' Guide
리눅스 시스템 관리자 가이드 Version 0.6.2

http://kldp.org 리눅스 한글문서프로젝트

Chapter 1. 소개

"In the beginning, the file was without form, and void; and emptiness was upon the face of the bits. And the Fingers of the Author moved upon the face of the keyboard. And the Author said, Let there be words, and there were words."

"태초에, 이 파일은 형태가 없었으며 비어 있었다. 즉, 그 bit들 위에는 그저 공허함만이 자리하고 있었다. 이때 저자의 손가락이 키보드 위를 운행하였다. 저자가 가라사대 글이 있으라 하시니, 여기에 글이 있게 되었다. "

이 리눅스 시스템 관리자 안내서 (Linux System Administrator's Guide)는 리눅스 시스템을 운용하는데 필요한 시스템 관리 방법을 설명하고 있다. 이 책은 최소한 리눅스 시스템의 기본적인 사용법은 알고 있으나, 시스템 관리에 대해서는 거의 아무것도 모르는 ("그게 뭐지?"라고 생각하는 것처럼) 사람들을 위한 책이다. 이 책은 리눅스를 설치하는 방법에 대해서는 설명하지 않는다. 설치하는 방법에 대해서는 "Installation and Getting Started"를 참고하기 바란다. 기타 리눅스 문서에 대해 더 많은 정보를 원하는 사람들을 위해서, 이 페이지의 맨 아래에 Linux Documentation Project에 대해 간략히 설명하였다.

컴퓨터를 사용 가능하게 하기 위해 필요한 모든 작업들이 곧 시스템 관리 작업이다. 여기에는 파일 백업하기(그리고 필요하면 복원하기), 새로운 프로그램 설치하기, 사용자에게 계정 만들어주기(그리고 더이상 필요없으면 지우기), 파일시스템이 망가지지 않게 하기 등의 작업들이 포함된다. 만일 컴퓨터를 집이라 한다면, 시스템 관리(administration)는 집을 유지보수(maintenance)하는 일과 같다고 할 수 있을 것이며, 여기에는 청소하기, 깨진 창문 고치기와 기타 여러가지 작업들을 포함하게 될 것이다. 그러나 시스템 관리하기를 유지보수하기라고는 하지 않는데, 시스템 관리를 설명하기에 이 개념은 너무 단순하기 때문이다. [1]

이 책의 구조는 많은 장들이 독립적으로 읽기 가능하도록 되어 있어서 만약 백업에 대한 정보를 원한다면 바로 백업에 대한 장을 읽을 수 있다. 이렇게 구성이 독립적으로 되어 있는 것은, 모든 것을 다 읽지 않고서도 필요한 부분만 조금씩 읽을 수 있도록 하여 참고서로 활용하기 쉽게 하기 위해서이다. 그러나 이 책은 기본적으로 안내서이므로, 특정한 경우에만 참고서로 쓰일 수 있을 것이다.

또한, 이 책은 완벽히 모든 것이 설명된 백과사전이 아니다. 시스템 관리자는 언제나 다른 많은 리눅스 문서들을 참고하여야 한다. 결국, 시스템 관리자라는 것은 특별한 권리와 의무를 지닌 한 사용자일 뿐인 것이다. 가장 중요한 참고자료는 매뉴얼 페이지로, 매뉴얼페이지는 명령에 대해 잘 모를때 도와준다.

이 책의 주된 목표는 리눅스 시스템 관리를 설명하는 것이지만, 다른 유닉스에 기반을 둔 운영체제에도 쓸모가 있도록 한다는 것이 일반 원칙이었다. 불행히도 일반적으로 유닉스의 다른 버전사이에는 많은 차이가 있기 때문에, 특히 시스템 관리에 대해 모든 차이점을 다 포함해 설명하기는 힘들다. 더구나 리눅스의 개발 특성에 비추어 보면, 심지어 리눅스조차도 모든 경우를 포함시키기 힘든 것이 사실이다.

또한, 하나의 공식적인 리눅스 배포본이 존재하지 않으므로 많은 사람들이 각기 그들 나름대로의 설정을 갖고 있기 마련이다. 비록 필자는 거의 유일하게 데비안 리눅스(Debian GNU/Linux)를 사용하고 있지만, 이 책은 어떤 하나의 리눅스 배포본을 기준으로 하진 않는다. 될 수 있는대로 배포본 간의 차이점을 지적하려 노력했으며, 여러가지 대안들을 설명하였다.

한편, 각각의 작업에 대해 단지 "쉬운 5단계"를 나열하기 보다는 그 일들이 어떻게 작동되는가를 묘사하려고 노력하였다. 사실 그런 많은 정보들이 모든 사람들에게 필요한 것은 아니므로, 그 부분들은 미리 표시가 되어 있으며 미리 설정된 시스템을 사용한다면 건너 뛸 수도 있다. 그러나 모든 부분을 다 읽는 것은 자연히 시스템에 대한 이해를 높여 줄 것이며, 리눅스를 사용하고 관리하는 일을 더욱 즐겁게 해줄 것이다.

모든 다른 리눅스 개발 작업과 마찬가지로, 이 책을 쓰는 작업도 자발적으로 이루어졌다. 이 책을 쓰는 것이 재미있을 것이라고 생각했고 또한 반드시 행해져야 하는 일이라고 생각해서 이 책을 썼다. 그러나 모든 자발적인 작업이 그렇듯이, 여기에 쏟아부을 수 있는 노력도 역시 한계가 있으며 지식과 경험에도 한계가 있을 수 밖에 없다. 사실, 진정한 고수가 보수를 받으며 몇년씩 집필해서 완성한 그런 문서들처럼 이 매뉴얼이 훌륭하다고는 말할 수 없을지 모른다. 하지만, 이것은 다만 자격지심에서 말해두는 것 뿐이다. 이 매뉴얼이 어느 정도 충분히 훌륭하다고 나는 믿는다.

이 매뉴얼에서는, 이미 자유롭게 사용 가능하도록 문서화 되어 있는 내용들은 거의 포함하지 않았다. 이것은 특히, 예를 들어 mkfs 명령의 자세한 사용법과 같은, 프로그램 특정적인 문서들에 적용되는 원칙이다. 여기서는 단지 그 프로그램들의 용도를 설명하였고 이 책의 목적에 필요한 만큼의 사용법만을 서술했다. 즉, 이 매뉴얼에서 언급한 부분은 모두 해당 문서 중의 일부분일 뿐이다. 만일 이 보다 더 많은 정보가 필요하다면, 그 해당 문서를 직접 찾아 보시기를 너그러운 독자 여러분께 부탁드린다.

나는 이 문서를 될 수 있는대로 개선시키기 위해 노력하고 있다. 그러므로 이 문서를 위한 좋은 아이디어가 있다면 아낌없이 충고해 주기 바란다. 잘못된 문법, 실제로 잘못된 내용, 다시 써야할 필요가 있는 분야에 대한 의견, 다양한 유닉스 버전들이 어떻게 작동되는지에 대한 정보 등, 이 모든 것들에 관심이 있다. http://www.iki.fi/viu/ 에서 저자에게 연락을 취할 수 있는 방법을 알려 줄 것이다.

많은 사람들이 직접 또는 간접적으로 이 책을 쓰는데 도움을 주었다. 특히 LDP를 통솔하며 이 문서 작업을 독려해 주신 Matt Welsh, 이 문서를 다시 작성하는데 있어 매우 가치있는 의견을 보내주신 Andy Oram, 이 문서를 완성할 수 있다는 본보기를 보여주신 Olaf Kirch, 또한 많은 사람들이 이런 문서를 필요로 하고 있다는 사실을 깨닫게 해주신 Yggdrasil의 Adam Richter, 그 밖의 많은 분들께 감사드린다.

또한, ext2에 대한 설명과, xia와 ext2 파일시스템 비교 내용, 그 밖에 디바이스 리스트 등 유용한 정보를 제공해 주신 Stephen Tweedie, H. Peter Anvin, Remy Card, Theodore Ts'o께 감사드린다(덕분에 이 문서를 더욱 두껍고 알차게 꾸밀수 있었다). 비록 이 내용들은 여기에 더 이상 수록되지 않게 되었지만, 이 분들께 가장 고맙게 여기며 이전 버전에서 이러한 기여에 대한 언급이 때로 부족했던 것을 죄송스럽게 생각한다.

그 외에 1993년에 많은 자료들을 제공해 주시고, 리눅스 저널에 실린 많은 시스템 관리 기사들도 보내주신 Mark Komarinski에 감사드린다. 그것들은 아주 유익하였으며 많은 영감을 주었다.

많은 분들이 매우 유익한 비평을 해주셨다. 기억력이 안 되어 모든 이름을 기억 못하지만, 일부분은 알파벳순으로 다음과 같다. : Paul Caprioli, Ales Cepek, Marie-France Declerfayt, Dave Dobson, Olaf Flebbe, Helmut Geyer, Larry Greenfield와 그의 아버지, Stephen Harris, Jyrki Havia, Jim Haynes, York Lam, Timothy Andrew Lister, Jim Lynch, Michael J. Micek, Jacob Navia, Dan Poirier, Daniel Quinlan, Jouni K Sepp?en, Philippe Steindl, G.B. Stotte. 그 밖에 기억을 못하는 다른 분들께는 죄송스럽게 생각한다.

The Linux Documentation Project

리눅스 문서 프로젝트(LDP)는 리눅스 운영체제를 위한 완벽한 문서를 제공하기 위해 같이 일하는 작성자, 교정자, 편집자들의 자유로운 모임이다. 이 프로젝트의 전반적인 진행 상황은 Greg Hankins가 조율해 주고 있다.

이 매뉴얼은 Linux Users' Guide, System Administrators' Guide, Network Administrators' Guide, Kernel Hackers' Guide로 이루어진 LDP의 핵심 문서들 중 하나이다. 이 매뉴얼은 sunsite.unc.edu의 /pub/Linux/docs/LDP에서 anonymous FTP를 통해 LaTeX형식, .dvi형식, 포스트스크립트 형식으로 얻을수 있다.

리눅스 문서의 질을 향상시키기 위한 글 쓰기와 편집에 참여해 보기를 권한다. 의욕이 있으신 분들은 E-mail을 <gregh@sunsite.unc.edu> 로 보내 Greg Hankins와 상의하기 바란다.

Notes

[1] 시스템 관리하기를 유지보수하기라고 하는사람들이 있는데, 그 사람들이 이 책을 읽지 않았기 때문이다. 가엾어라.

이 문서의 소스와 pre-formatted version을 얻으실 수 있습니다 리눅스 시스템의 개괄

The Linux System Administrators' Guide: 리눅스 시스템 관리자 가이드 Version 0.6.2

Chapter 2. 리눅스 시스템의 개괄

Table of Contents

"God looked over everything he had made, and saw that it was very good. " (Genesis 1:31)

"이렇게 만드신 모든 것을 하느님께서 보시니, 참 좋았다. " (공동번역 성서 - 창세기 1장 31절)

여기서는 리눅스 시스템의 전반적인 구성을 간략히 알아볼 것이다. 먼저, 운영체제의 역할들 중 핵심적인 것 몇 가지를 살펴보고, 이런 역할들을 실제로 구현해주는 프로그램들에 관해 간단히 알아보도록 하겠다. 일단은 리눅스 시스템을 포괄적으로 이해하는 것이 목적이므로, 각각의 세부적인 내용은 뒤로 미루었다.

운영체제의 구성

UNIX 계열의 운영체제는 커널(kernel)과 여러가지 시스템 프로그램(system programs) 들로 이루어져 있는데, 여기에는 업무수행을 위한 몇가지 응용 프로그램(application programs) 들도 덧붙여져 있다. 이중에서도, 특히 커널은 운영체제의 심장부라고 할 수 있는 부분이다. [1] 커널은 파일들을 디스크에 적절히 배치시키거나, 프로그램을 시동시켜 작업을 수행하게 하고, 메모리와 같은 시스템의 자원(resource)을 각각의 프로세스에 할당하며, 네트워크를 통해 패킷(packet)을 주고받을 수 있게 해준다. 그러나 커널이 모든 일들을 혼자서 처리하는 것은 아니며, 실제로 커널이 혼자서 처리하는 부분은 매우 적다. 대신에 커널은 기반 설비(tools)들을 제공함으로써 모든 작업을 가능하도록 하는데, 이 설비들을 통하지 않고서는 어떤 것도 직접 하드웨어를 다루지 못하게 한다. 이런 방식으로, 하드웨어를 동시에 사용하려는 각각의 사용자들이 서로 충돌하는 일을 막을 수 있다. 커널이 갖추고 있는 설비들을 사용하는 일은 시스템 콜(system call)을 통해서 이루어진다 : 이에 관한 상세한 내용은 해당 매뉴얼 페이지의 두번째 섹션을 참조하기 바란다.

시스템 프로그램들은 운영체제를 위해 다양한 역할을 수행해 주어야 하는데, 이를 구현하기 위해 역시 커널의 기반 설비들을 사용한다. 즉, 시스템 프로그램이나 기타 여러가지 응용 프로그램들은 모두 `커널 위에서' 실행된다고 할 수 있겠다. 이 상태를 사용자 모드(user mode)라고 부른다. 사용자 모드에서 시스템 프로그램과 기타 프로그램은 결국 그 궁극적인 역할에 차이가 있을 뿐이다 : 사용자의 업무에 필요한 역할을 하도록 되어 있는 것이 응용 프로그램이라면(놀이가 필요하다면 게임이 되겠다), 시스템 프로그램은 시스템이 동작하는데 필요한 역할을 하게 되어 있는 것이다. 그러므로 워드프로세서는 응용 프로그램, telnet 은 시스템 프로그램이라고 할 수 있다. 사실, 이런식의 구분은 애매모호한 경우가 많은데, 강박적으로 뭐든 분류하기를 좋아하는 사람들에게나 중요한 문제일 것이다.

운영체제는 또한 컴파일러와 그에 부합되는 라이브러리를 포함하고 있는데(리눅스의 경우에는 GCC와 C 라이브러리를 포함하고 있다.), 모든 종류의 프로그래밍 언어가 운영체제에 포함되어 있을 필요는 없다. 또한 각종 문서가 운영체제와 함께 첨부되어 있기도 하며, 어떤 경우에는 게임이 포함되어 있는 경우도 있다. 전통적으로는 설치 디스크나 테이프에 들어있는 내용이 그 운영체제의 모든 것으로 간주되었다 ; 그러나 이런 관점은 리눅스에는 어울리지 않는데, 그것은 리눅스가 FTP 사이트를 통해 전세계로 퍼져나가기 때문이다.

Notes

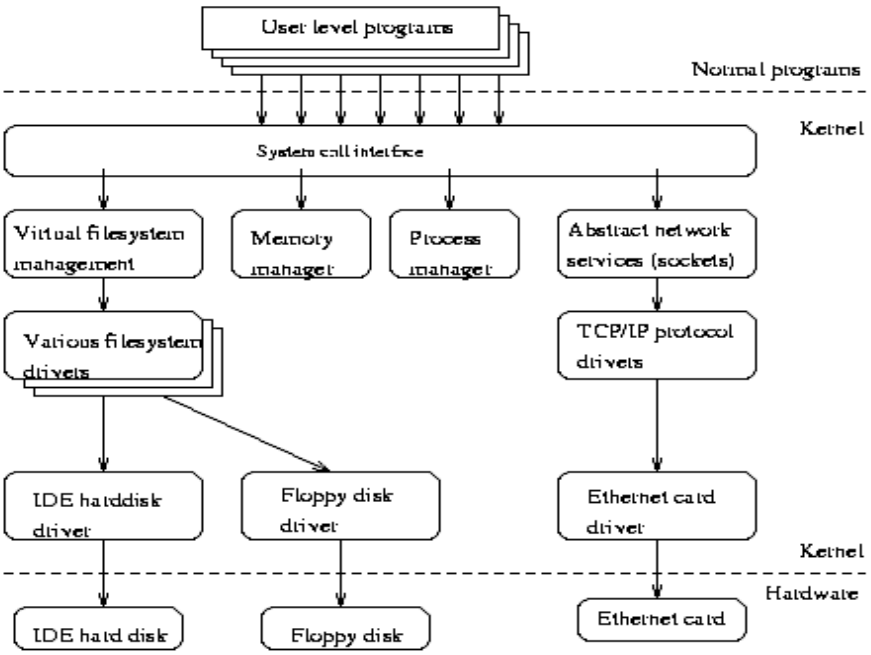
[1] 흔히, 커널이 운영체제 그 자체인 것처럼 잘못 생각하는 경우가 있는데, 사실은 그렇지 않다. 커널이 제공하는 것 이상의 많은 서비스들을 제공할 수 있어야 그것을 하나의 운영체제라고 부를 수 있을 것이다.

커널 핵심부의 구성

리눅스 커널에는 몇가지 핵심적인 부분들이 있는데 그것은 : 프로세스 관리자, 메모리 관리자, 하드웨어 장치 드라이버, 파일시스템 드라이버, 네트워크 관리자(process management, memory management, hardware device drivers, filesystem drivers, network management)이며 그 외에도 다양한 구성요소가 있다. 이 중 일부를 Figure 2-1 에 나타내었다.

Figure 2-1. 리눅스 커널 핵심부의 개괄.

아마도 커널에서 가장 중요한 구성요소는 메모리관리자와 프로세스 관리자일 것이다(이것들 없이는 꼼짝도 할 수 없다!). 메모리 관리자는 프로세스, 커널 일부분, 버퍼 캐쉬를 메모리 영역과 스왑 공간에 적절히 할당하는 역할을 한다. 프로세스 관리자는 새로운 프로세스를 생성하고 멀티태스킹을 구현하는데, 멀티태스킹은 프로세서 상의 프로세스를 계속 바꿔치기(switching)하는 기법으로 이루어진다.



커널의 가장 밑바탕은 갖가지 종류의 하드웨어 장치 드라이버들로 이루어진다. 하드웨어는 그 종류가 워낙 다양해서, 하드웨어 장치 드라이버도 그 수가 무척 많다. 흔히, 비슷한 기능이면서도 소프트웨어에 의해 구동되는 방식이 다른 하드웨어가 많은데, 이런 유사성은 비슷한 기능을 통틀어 구동시키는 일반적인 드라이버 클래스를 갖출 수 있게 해준다; 즉, 같은 클래스에 속하는 멤버 드라이버들은 자신을 제외한 커널의 나머지 부분에 대해선 같은 인터페이스를 갖는다. 그러나 각각의 드라이버들이 기능을 실제로 구현하는 방법은 서로 다르다. 예로, 모든 디스크 드라이버들은 커널의 나머지 부분에 대해 비슷한 인터페이스를 갖는데, 실제로 디스크 드라이버들은 "드라이브 초기화" "N번째 섹터 읽기" "N번째 섹터 쓰기"와 같은 조작방법을 모두 갖추고 있다.

커널이 독립적으로 제공하는 소프트웨어 서비스들 중에도 유사성을 가진 것들이 있어서, 역시 클래스란 것으로 추상화 될 수 있다. 예를 들자면, 수많은 네트웍 프로토콜들은 BSD 소켓 라이브러리라는 하나의 프로그래밍 인터페이스로 추상화되어 왔다. 또 다른 예로, 가상 파일시스템 계층(virtual filesystem (VFS) layer) 이란 것이 있는데, 이것은 파일시스템 조작방법을 실제 구현방법에서 떼어내 추상화한 것이다. 파일시스템을 사용하려는 요청은 VFS에 전해지고, VFS는 요청에 알맞은 파일시스템 드라이버를 골라 준다. 각각의 파일 시스템 드라이버는 그에 해당하는 파일시스템 조작방법을 실제로 구현해 낸다.

유닉스 시스템의 주요 기능

여기서는 UNIX가 제공하는 핵심적인 기능 몇가지를 대략적으로 알아볼 것이다. 각각의 상세한 내용은 뒤에서 살펴보도록 하겠다.

init

UNIX의 독립적인 핵심기능들은 대부분 init에 의해 제공된다. init는 모든 UNIX 시스템에서 가장 먼저 실행되는 프로세스이면서, 또한 부팅시에 커널이 수행하는 맨 마지막 과정이다. init는 시스템 시동에 필요한 갖가지 작업을 수행함으로써 부팅과정을 계속 이어나간다(파일시스템을 검사하고 마운트하기, 데몬을 시동시키기 등의 갖가지 작업을 한다).

init가 수행해야할 작업의 구체적인 목록은 시스템이 어떤 상태로 부팅되기를 원하는냐에 따라 달라진다; 여기에는 몇가지 선택이 있을 수 있다. 단일 사용자 모드(single user mode) 는 아무도 로그인하지 못하게하고, root가 콘솔에서만 쉘을 사용할 수 있는 특수한 상태이다; 일반적으로는 다중 사용자 모드(multiuser mode) 가 적용된다. 이런 상태들 중에 몇가지는 실행 레벨(run revel) 이란 개념으로 일반화된다; 단일 사용자 모드와 다중 사용자 모드는 각각 하나씩의 실행 레벨로 간주된다. 또한 여기에 부가적인 실행레벨이 있을 수 있는데, 예를 들자면 콘솔 상에서 X를 구동하기 위한 또하나의 실행 레벨이 있을 수 있다.

시스템이 일반적으로 가동되는 상태에서, init는 getty(사용자가 로그인 할 수 있도록 해준다)가 제대로 동작하고 있는지 확인하며, 또한 고아 프로세스를 인수하는 역할을 맡는다(orphan processes, 부모 프로세스가 죽어버린 프로세스; UNIX 시스템에서 모든 프로세스는 하나의 트리(tree) 구조를 이루어야만 한다. 따라서, 고아가 된 프로세스는 누군가가 양자로 삼아주어야 한다).

시스템이 셧다운 될 때, 남아있는 프로세스를 모두 종료시키고 모든 파일시스템의 마운트를 해제해 주는 것도 init이다. 그 밖에, 시스템 종료시에 수행하도록 정해준 기타 작업들도 init가 처리하게 된다.

터미널을 통한 로그인
터미널(시리얼 라인으로 연결된)이나, 콘솔(X가 구동되지 않은)을 통한 로그인은 getty 프로그램이 처리하게 된다. 우선 init는 각각 개별적인 getty 인스턴스를 각각의 로그인 터미널에 띄워 놓는다. getty는 입력받은 username을 확인하고 login 프로그램을 돌려 password를 해독하게 한다. username과 password가 정확하면, login은 쉘을 구동시킨다. username과 password가 정확하지 않거나, 사용자가 로그아웃하여 쉘이 종료되면, init는 이 사실을 간단히 알려주고 새로운 getty 인스턴스를 시작시킨다. 커널은 로그인 과정에 전혀 관여하지 않으며, 이런 모든 과정은 시스템 프로그램들이 도맡아 처리한다.

Syslog
커널과 여러가지 시스템 프로그램들은 각종 에러와 경고 메시지, 기타 일반적인 메시지들을 내놓는다. 이런 메시지들은 무척 유용한 정보이므로, 시간이 많이 흐른 뒤에도 다시 볼 수 있도록 꼭 파일로 기록해 두어야 한다. 이 일을 해주는 것이 바로 syslog이다. syslog는 여러 메시지들을 그 출처와 중요도에 따라 각기 다른 파일에 정리할 수 있다. 예를 들어, 커널이 출력하는 메시지는 따로 파일을 만들어 기록해 두는 일이 많은데, 이렇게 하는 이유는 커널 메시지가 워낙 중요하고, 문제점을 집어내기 위해선 자주 읽어 보아야 하기 때문이다.

명령의 주기적인 실행 : cron과 at
일반 사용자이거나 시스템 관리자이거나 간에, 어떤 명령을 주기적으로 반복 실행시켜야 할 필요성은 누구나 느끼게 된다. 한가지 예로, 어떤 프로그램들은 임시로 생성한 파일들을 제대로 지우지 않는데, 그래서 임시 파일용 디렉토리(/tmp,/var/tmp) 안에는 더 이상 필요하지 않은 파일들이 있을 수 있다. 이런 것들이 점점 많아지면 디스크의 공간이 낭비되므로, 시스템 관리자들은 이런 파일들을 주기적으로 지우고 싶어하기 마련이다.

이런 반복작업을 바로 cron 서비스가 해준다. 사용자는 각각의 crontab 파일을 가질 수 있는데, 여기에 각 사용자가 실행시키기 원하는 명령들과 그 시간을 적어두면 cron 데몬이 특정 시간마다 그 명령들을 실행시켜 준다.

at 서비스는 cron과 비슷하지만 한가지 면에서 다르다; at도 주어진 시간에 명령을 수행해주지만 그것을 되풀이 하지는 않는다.

그래픽 유저 인터페이스
UNIX나 Linux의 커널은 사용자 인터페이스를 내장하고 있지 않다; 그 대신, 유저 레벨의 프로그램이 사용자 인터페이스를 구현해 주는데, 이런 점은 텍스트 모드 인터페이스와 그래픽 환경의 인터페이스 둘다 마찬가지이다.

이런 방식은 시스템을 보다 융통성있게 해준다. 그러나 각각의 프로그램들이 서로 다른 유저 인터페이스를 가지게 되고, 이에따라 시스템을 익히기가 어려워지는 점은 약간의 단점이라 할 수 있다.

Linux에서 기본적으로 사용하는 그래픽 환경은 X이다(X Window System이라고도 불리운다). 그런데 X도 역시 유저 인터페이스를 내장하고 있지 않기는 마찬가지이다; 다만 그래픽 유저 인터페이스를 구현하기 위한 기반 설비들, 즉 윈도우 시스템 그 자체만을 제공한다. X 위에서 구현되는 유저 인터페이스에는 여러가지 스타일이 있는데, 그 중에 가장 많이 쓰이는 것으로는 아데나, 모티프, 오픈룩 스타일을 꼽을 수 있다.(Athena, Motif, and Open Look)

네트워킹
네트워킹이란 둘 이상의 컴퓨터를 연결하여 서로 커뮤니케이션 할 수 있도록 해주는 것을 말한다. 실제로 연결과 커뮤니케이션이 이루어지는 방법은 상당히 복잡한 감이 있지만, 이를 통해 얻을 수 있는 이득은 실로 막대한 것이다.

UNIX 운영체제는 다양한 네트워킹 지원을 갖추고 있다. 가장 기본적인 서비스들 -- 파일시스템, 프린팅, 백업 등등 -- 은 모두가 네트워크 상에서도 이루어질 수 있다. 이를 통해, 저비용과 결함허용 같은, 마이크로컴퓨팅과 분산컴퓨팅의 잇점들을 모두 살리면서도 시스템을 중앙집중식으로 보다 쉽게 관리하는 것이 가능해진다.

아쉽게도 여기서는 네트워킹에 관해 깊이있게 다루지 않는다; 네트워크가 기본적으로 어떻게 움직이는지를 포함해서 더

깊이있는 내용을 알고 싶은 사람은 Linux Network Administrators' Guide를 읽어보기 바란다.

네트워크를 통한 로그인

네트워크를 통한 로그인은 일반적인 로그인과는 좀 다르다. 일반적인 로그인의 경우에는 로그인 가능한 각각의 터미널을 잇는 개별적인 시리얼 라인이 일정하게 존재하고 있다. 반면에, 네트워크를 통해 로그인하는 경우에는 네트워크를 통한 가상의 연결이 이루어질 뿐이며 그 연결도 일정하지 않다. [1] 따라서 각각의 가상연결마다 미리 getty를 띄워놓고 기다린다는 것은 불가능하다. 네트워크로 로그인하기 위해서는 몇가지 다른 방법이 필요한데, TCP/IP 네트워크에서는 telnet과 rlogin을 주로 사용한다.

네트워크 로그인에서는 getty들을 대신해서 각각의 로그인 방법마다 데몬 하나씩을 독립적으로 띄워 놓고(즉, telnet과 rlogin은 서로 다른 데몬이 필요하다) 로그인 요청이 있는지를 잘 듣고 있도록 한다. 로그인 요청 하나가 들어오면, 데몬은 자기자신의 새로운 인스턴스 하나를 실행시켜 그에 응하게 한다; 그리고 원래의 인스턴스는 다른 요청이 있는지 다시 귀 기울이고 있게 된다. 새로운 인스턴스가 하는 일은 getty와 비슷하다.

네트워크 파일 시스템

네트워크를 통해 얻을 수 있는 커다란 이점 중 하나는 네트워크 파일 시스템(network file system)을 통해 파일을 공유할 수 있다는 점이다. 이것은 Network File System 또는 NFS라고 불리며, Sun사에 의해 개발되었다.

네트워크 파일 시스템을 통하면, 한 컴퓨터에서 프로그램이 어떠한 파일 조작을 하더라도 그것을 네트워크 건너편의 다른 컴퓨터로 보낼 수 있다. 이것은 다른 컴퓨터에 있는 파일들이 마치 자신의 컴퓨터에 있기라도 한 것처럼 프로그램을 착각하게 만든다. 이렇게 하면 프로그램들을 특별히 수정하지 않아도 되므로, 정보의 공유를 아주 손쉽게 할 수 있다.

전자 우편

전자 우편은 컴퓨터를 통한 커뮤니케이션에서 가장 중요한 위치를 차지하고 있다. 편지는 특별한 형식의 파일에 저장되며, 편지를 읽고 보내기 위해서는 특정한 메일 프로그램을 사용하여야 한다.

각 사용자는 새로 온 편지가 보관되는 편지함(incoming mailbox, 특정 형식의 파일이다)을 갖고 있게 된다. 누군가 편지를 보내면, 메일 프로그램은 받는 이의 편지함이 어디 있는지 확인하고 그 파일의 뒤에 편지 내용을 덧붙여 놓는다. 만일 누군가가 다른 컴퓨터에 있는 사용자에게 편지를 보낸다면, 메일 프로그램은 편지를 배달하기에 적당한 위치에 있는 다른 컴퓨터를 찾아서 그 편지를 넘겨준다.

이러한 전자 우편 시스템은 많은 프로그램들로 이루어진다. 편지를 배달해 주는 일은 하나의 프로그램이 도맡아 처리하는데, 이런 프로그램을 mail transfer agent 또는 MTA라고 하며 sendmail이나 smail 같은 것이 있다. 반면에 사용자들은 편지를 읽고 쓰기 위해 각자 다양한 프로그램을 사용하기 마련이다. 이런 프로그램을 mail user agent 또는 MUA라고 하며, pine이나 elm이 대표적이지만 그 종류가 무척이나 다양하다. 보통 편지함은 /var/spool/mail에 위치하는 것이 일반적이다.

인쇄

프린터는 한번에 한사람만이 쓸 수 있다. 하지만 프린터를 한사람만 계속 사용한다면 그것은 아주 비경제적인 일일 것이다. 그래서, 프린터는 프린트 큐(print queue)라는 것을 구현해 주는 소프트웨어를 통해 서로 공유할 수 있도록 되어있다: 모든 프린트 작업은 큐로 보내지며 그곳에서 차곡차곡 쌓여있다가 자기 차례가 오면 자동으로 인쇄된다. 이렇게 하면 사용자들이 인쇄작업의 순서에 신경쓰지 않아도 되고, 또한 프린터를 서로 장악하기 위해 싸울 필요도 없어진다. [2]

실제로 프린트 큐 소프트웨어는 프린트 작업을 디스크에 쌓아두는(spool) 일을 한다. 즉, 프린트 작업들은 실제로 출력되기 전까지 파일의 형태로 큐에서 대기하게 된다. 이런 방식은 응용 프로그램들이 인쇄물을 빨리 프린터 큐로 내보내 버리고 다른 일을 할 수 있도록 해준다; 즉, 응용프로그램은 자신의 인쇄물이 프린터에서 실제로 출력될 때까지 기다릴 필요가 없게 된다. 따라서 하나의 문서를 작성한 뒤 그것이 인쇄될 때까지 기다리지 않고도 즉시 다른 문서를 작성할 수 있으므로 무척이나 편리하다.

파일시스템의 얼개

파일시스템은 많은 부분으로 나누어 질 수 있다; 보통은 /bin, /lib, /etc, /dev를 포함하는 루트(root) 파일시스템과

/usr, /var, /home 같은 몇가지 다른 파일시스템으로 나누게 된다. /usr 파일시스템에는 일반 프로그램들과 내용이 변화하지 않는 데이터들이 위치하게 되고, /var 파일시스템에는 내용이 계속 변화하는 데이터(log 파일 같은 것)들이 위치하게 된다. 또한 /home 파일시스템은 모든 사용자들의 개인 파일을 위한 공간이다. 이런 분할은 하드웨어의 사정과 시스템 관리자의 결정에 따라 얼마든지 바뀔 수 있으며, 심지어 모든 것을 하나의 파일시스템에 몰아 넣을 수도 있다.

파일시스템의 얼개에 대해서는 다음에 이어지는 Chapter 3 부분에서 좀 더 상세한 내용을 다룬다; 또한 Linux Filesystem Standard 문서를 읽어본다면 더욱 더 상세한 내용을 접할 수 있을 것이다.

Notes
[1] 따라서, 무척 많은 수의 연결이 이루어질 수도 있지만, 네트워크의 대역폭은 워낙 한정된 자원이므로 이를 통해 동시로그인 할 수 있는 수에는 현실적인 한계가 있을 수 밖에 없다.

[2] 즉, 싸울 필요없이 그저 프린터 큐에 인쇄물을 넣어두면 된다. 이런 방식을 쓰면, 지금 누구 때문에 자신의 인쇄 작업이 지연되고 있는지 쉽게 알 수 없으므로 사무실 내의 인간 관계가 원만히 유지될 수 있다.

Chapter 3. 디렉토리 트리의 개괄
Table of Contents
배경
루트 파일시스템
/etc 디렉토리
/dev 디렉토리
/usr 파일시스템
/var 파일시스템
/proc 파일시스템
" Two days later, there was Pooh, sitting on his branch, dangling his legs, and there, beside him, were four pots of honey..." (A.A. Milne)

" 이틀 후에, 푸우가 나타났는데, 그는 나뭇가지 위에 앉아, 다리를 흔들고 있었다, 그리고 거기, 그 옆에는, 네통의 꿀단지가 있었다... " (A.A. Milne의 아기곰 푸우 중에서)

디렉토리는 마치 나뭇가지와도 같은 계층구조를 이루고 있는데, 이를 가리켜 트리(tree) 구조라고 한다. 이번 장에서는 표준 리눅스 디렉토리 트리 구조의 주요 부분을 FSSTND 파일시스템 표준에 근거하여 살펴 볼 것이다. 또한 여기서는 여러가지 목적에 알맞게 디렉토리 구조를 분할하는 일반적인 방법에 대해 개괄적으로 알아 볼 것이며 이렇게 디렉토리를 특별히 분할하는 취지에 관해서 설명할 것이다. 그리고 디렉토리 분할 방법의 몇가지 다른 대안에 대해서도 알아보기로 하겠다.

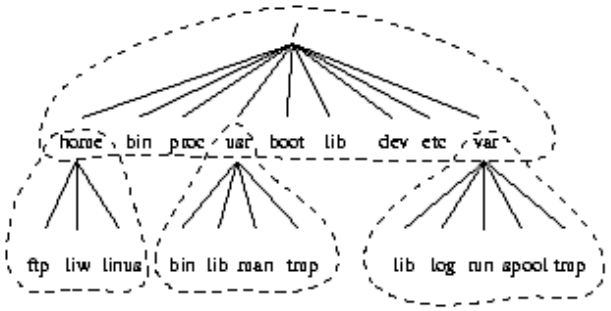
배경
여기서 다룰 내용은 대체로 리눅스 파일시스템 표준안(Linux filesystem standard, FSSTND, version 1.2 - 참고문헌을 볼 것)에 기반하고 있다. 이 표준안은 리눅스에서 파일시스템을 어떻게 조직할 것인가에 대한 표준을 제정하기 위해 만들어진 문서로서, 이런 표준에 따라 각 파일들의 위치가 일관되게 유지된다면 리눅스용 프로그램의 작성,포팅이 쉬워지고 또한 리눅스 머신을 관리하기도 쉬워지는 등 많은 잇점을 지니게 된다. 사실 이런 표준안이 어떤 강제력을 지니고 있는 것은 아니지만 거의 대부분의 리눅스 배포판에서 이를 따르고 있으며, 특별한 이유없이 이 표준을 어기는 것은 별로 바람직한 일이 못된다. FSSTND는 전통적인 유닉스 방식과 최신 경향을 함께 반영하려 노력하고 있으며, 이를 통해 다른 유닉스 경험자들이 리눅스에 보다 친숙함을 느낄 수 있도록 배려하고 있다. 물론 리눅스 경험자가 다른 유닉스를 접하는 경우에도 마찬가지이겠다.

이번 장에서는 FSSTND에 대해 상세하게 다루지는 않는다. 리눅스 시스템 관리자라면 좀 더 깊이있는 이해를 위해 FSSTND를 꼭 읽어보기 바란다.

또한 여기서는 모든 파일들에 대해 일일이 다루지 않으며 파일시스템의 관점에서 시스템의 구조를 이해하는데 치중할 것이다. 각각의 파일에 대한 정보는 이 문서의 다른 곳을 찾아보거나 매뉴얼 페이지(man page)를 참고하기 바란다.

전체 디렉토리 트리는 분할이 가능하도록 되어 있다. 분할된 각 부분들은 서로 다른 디스크나 파티션에 들어가게 되는데, 이렇게 하면 디스크 공간의 제약에서 벗어날 수 있고 백업하기도 수월해지며 그 밖에 다른 시스템 관리 작업도 한결 손쉬워진다. 파일시스템의 주요 부분을 꼽아보자면 루트(root) 파일시스템, /usr 파일시스템, /var 파일시스템 그리고 /home 파일시스템을 들 수 있으며(Figure 3-1을 보세요), 각 부분들은 서로 다른 의도를 가지고 만들어진 것이다. 이런 디렉토리 트리는, CD-ROM 같은 읽기 전용 장치나 NFS를 통해 파일시스템의 일부를 공유하고 있는 리눅스 머신들의 네트워크에 알맞도록 설계되어져 왔다.

Figure 3-1. 유닉스 디렉토리 트리의 각 부분들. 점선은 각 파티션 영역의 경계를 나타낸다.



아래에 디렉토리 트리 각 부분의 역할에 대하여 설명하였다.

루트 파일시스템은 각 머신마다 고유한 것으로서(비록 루트 파일시스템이 램 디스크나 네트워크 드라이브에 있을 수도 있지만, 보통 로컬 디스크에 있는 것이 일반적이다), 시스템을 부팅시키고 다른 파일시스템을 마운트할 수 있는 상태로 만드는데 필요한 파일들을 갖고 있다. 또한 루트 파일시스템은 단일 사용자 상태를 위한 파일들을 포함하고 있으며, 파일시스템을 수리하고 손상된 부분을 백업으로부터 복구하는데 필요한 도구들을 갖추고 있다.

/usr 파일시스템에는 시스템이 정상적으로 가동되는 데에 필요한 모든 명령들과 라이브러리들 그리고 매뉴얼 페이지들이 위치한다. /usr 파일시스템에 있는 파일들은 어떤 머신에 한정된 것이어서는 안되며, 보통 실행 도중에 그 내용이 고쳐질 필요가 없는 것들이어야 한다. 이렇게 하면 네트워크 상에서 파일들을 공유하는 것이 가능해지는데, 이를 통해 디스크 공간을 절약하여 비용절감의 효과를 누릴 수 있고(/usr 파일시스템은 보통 수백메가바이트 이상의 공간을 차지한다) 시스템 관리도 쉽게 할 수 있다(응용프로그램을 업데이트하기 위해선 공유되고 있는 /usr 파일시스템만 업데이트하면 된다. 즉 머신 하나하나에 손댈 필요가 없다). /usr 파일시스템은 로컬 디스크에 있는 경우라 하더라도 언제나 읽기 전용으로 마운트하는 것이 좋은데, 이것은 시스템이 크래쉬된 경우에 파일시스템이 손상되는 것을 방지하기 위한 것이다.

/var 파일시스템은 스풀(spool) 디렉토리들, 로그 파일들, 포맷된 매뉴얼 페이지들, 그리고 각종 임시파일들 같은, 계속 변화하는 파일들을 위한 공간이다. 전통적으로는 이런 파일들을 /usr 아래에 넣어 두는 것이 관례였는데, 이렇게 되면 /usr를 읽기 전용으로 마운트할 수 없기 때문에 별로 바람직한 일이 못된다.

/home 파일시스템은 각 사용자들의 홈 디렉토리를 갖고 있으므로, 이곳은 아주 중요한 데이터들이 있는 곳이라고 할 수 있겠다. 이렇게 /home을 독립적인 파일시스템으로 만들어 두는 이유는 백업을 쉽게 하기 위해서이다; 즉, /home 이외의 부분들은 거의 변동이 없기 때문에 백업을 그렇게 자주 받아둘 필요는 없다. 그러나 홈 디렉토리들은 무척 중요하므로 따로 백업을 자주 받아두어야 한다. /home 파일시스템이 비대해지면 이것을 다시 /home/students 또는 /home/staff 같이 하위 파일시스템으로 세분화하는 것이 좋겠다.

위에서 각 부분들이 서로 다른 파일시스템인 것으로 가정했지만, 사실 꼭 그렇게 해야만 하는 것은 아니다. 만일 단일 사용자용 시스템이거나 관리를 단순하게 하고 싶은 경우라면 모든 것을 하나의 파일시스템에 몰아 넣는 것도 가능하다. 또한 위에서 제시한 방법 이외에도, 여러가지 상황에 따라 얼마든지 다른 형태로 파일시스템을 구성할 수 있다. 중요한 것은, /usr나 /var 같은 표준적인 이름들을 사용할 수 있도록 해야 한다는 것이다; 즉, 만일에 /var가 /usr 파일시스템의 아래에 존재한다고 하더라도 /usr/lib/libc.a 또는 /var/log/messages 같은 경로들을 사용할 수 있어야만 한다. 이런 경우에는 /var를 /usr/var의 심볼릭 링크로 만들어 둠으로써 표준을 지키도록 할 수 있다.

유닉스 파일시스템 구조에서, 같은 목적의 파일들은 같은 장소에 보관된다. 즉 모든 명령들과, 모든 데이터들, 모든 문서들은 각기 독립적인 장소에 따로 보관된다. 그런데 같은 목적의 파일들을 구분하는데는 좀 다른 원칙이 적용될 수도 있다. 즉 모든 Emacs용 파일들과 모든 TeX용 파일들을 구분해서 각각 다른 곳에 모아두는 식이다. 하지만 두번째 방식을 채

택한다면 파일을 공유하기가 어려워질 뿐더러(각각의 프로그램 디렉토리는 공유가능한 파일들과 그렇지 않은 파일들을 함께 가지고 있을 것이기 때문이다) 원하는 파일을 찾아내는 일도 쉽지 않게 되는 단점이 있다(만일 매뉴얼 페이지들이 엄청나게 다양한 장소에 흩어져 있다면, 매뉴얼 페이지 검색 프로그램을 만드는 일은 아마 끝없는 악몽으로 여겨질 것이다).

루트 파일시스템
루트 파일시스템은 보통 크기를 작게 만든다. 왜냐면 루트 파일시스템은 아주 중요한 파일들을 담고 있는데, 크기가 작고 자주 갱신되지 않는 파일시스템일 수록 손상될 위험은 줄어들기 때문이다. 만일 루트 파일시스템이 손상된다면 특별한 방법(한 예로, 플로피로 부팅하는 방법)을 쓰지 않는 이상 부팅은 불가능해진다. 이런 일은 꼭 피해야만 될 일이다.

루트 디렉토리(/ 디렉토리)에는 /vmlinuz라고 불리는 부트 이미지 파일만 넣어두는 것이 일반적이지만, 부트 이미지마저도 /boot라는 디렉토리 안에 넣어두고 루트 디렉토리에는 파일을 두지 않는 경우도 많다. 그 밖의 다른 파일들은 모두 루트 파일시스템의 하위 디렉토리 안에 존재한다.

/bin
이 곳에는 부팅할 때 필요한 명령어들이 들어 있다. 또한 부팅 후에는 일반 사용자들도 이 곳의 명령들을 사용할 수 있다. bin은 명령어들의 '저장고'라는 뜻이다.

/sbin
이 곳은 /bin 디렉토리와 비슷하지만, 주로 시스템 관리를 위한 명령들이 보관된다. 일반 사용자들은 제한적으로만 이 곳의 명령들을 사용할 수 있다.

/etc
여기는 각 머신의 고유한 설정 파일들이 위치하는 곳이다.

/root
루트 사용자의 홈 디렉토리이다.

/lib
공유 라이브러리가 있는 곳이다. 이 곳의 라이브러리들은 루트 파일시스템에 있는 프로그램들이 사용한다.

/lib/modules
로딩 가능한 커널 모듈들이 위치하는 곳이다. 특별한 경우, 장애를 복구하기 위해 시스템을 부팅할 때도 커널 모듈들이 필요하다(예로서, 네트워크 드라이버와 파일시스템 드라이버가 있다).

/dev
장치 파일들이 있는 곳이다. 장치 파일은 일반적인 파일과는 다른 특수 파일로서, 마치 파일을 읽고 쓰듯이 하드웨어를 다룰 수 있게 해준다.

/tmp
임시 파일들을 위한 공간이다. 부팅이 이루어지고 난 뒤에 실행되는 프로그램들은 /tmp가 아닌 /var/tmp를 사용해야 하는데, 보통 /var/tmp는 좀 더 여유공간이 많은 디스크 상에 위치하는 경우가 많기 때문이다.

/boot
LILO 같은 부트스트랩 로더가 사용하는 공간으로, 커널 이미지들이 이곳에 위치하게 된다. 부트스트랩 로더는 부트 이미지의 위치를 파악하여 부팅을 시작시켜 주는 프로그램으로서, 부트 이미지라는 것은 결국 부팅에 사용되는 커널 이미지이다. 부트 이미지는 보통 루트 디렉토리에 넣어 두거나 또는 /boot에 다른 커널 이미지들과 같이 넣어 둔다. 만약 많은 수의 커널 이미지를 갖고 있다면 /boot는 공간을 많이 차지할 것이므로 이런 경우에는 따로 독립적인 파일시스템을 만들어 주는 것이 좋다. 또한, 대용량 IDE 디스크에서 부트 이미지가 첫번째 1024 실린더 안에 있도록 하기 위해서 /boot를 독립된 파일시스템(1024 실린더 안에 있는 파일시스템)으로 만들기도 한다. 이렇게 하는 이유는 대부분의 부트스트랩 로더들이 1024 실린더 밖에 있는 부트 이미지를 인식하지 못하기 때문이다.

/mnt
시스템 관리자에 의해 임시로 마운트된 파일시스템들이 위치할 곳(mount point)이다. 이 곳은 어디까지나 임시로 사용하는 곳이므로 프로그램들은 /mnt에 무엇이 마운트되었는지 자동적으로 인식하지는 않는다. /mnt는 보통 하위 디렉토리로 분할하여 사용하게 된다(예를 들어 /mnt/dosa라는 곳은 MS-DOS 파일시스템을 사용하는 플로피 디스크를 마운트하는 곳일 것이다. 혹시 /mnt/exta라면 이것은 아마 ext2 파일시스템을 사용하는 플로피 디스크를 마운트하는 곳일 것이다).

/proc, /usr, /var, /home
/home에는 각 사용자들의 홈 디렉토리가 위치한다. /proc, /usr, /var에도 각각 다른 파일시스템이 마운트된다. 자세한 내용은 뒤에서 설명하겠다.

/etc 디렉토리
/etc 디렉토리는 많은 파일들을 포함하고 있는데, 그 중 몇가지를 아래에 설명하였다. 여기에 설명되지 않은 파일들에 대해서 알아보고자 한다면, 우선 그 파일이 어느 프로그램에 속한 것인지를 파악한 후 그 프로그램의 매뉴얼 페이지를 살펴보기 바란다. 또한 이곳에는 많은 네트워킹 설정 파일들이 있는데 이런 파일들에 대한 자세한 내용은 Networking Administrators' Guide를 참고하기 바란다.

/etc/rc or /etc/rc.d or /etc/rc?.d
시스템 시작시나 실행 레벨이 바뀔 때 실행되는 스크립트들이다. 혹은 그런 스크립트를 모아둔 디렉토리일 수도 있다. 더 자세한 내용은 init를 다룬 부분을 참고하기 바란다.

/etc/passwd
이것은 사용자들의 데이터베이스 파일로서 이곳에는 사용자들의 username, 실제 이름, 홈 디렉토리의 위치, 암호화된 패스워드, 기타 정보들이 수록된다. 이 파일의 형식에 대해 자세한 내용은 passwd 매뉴얼 페이지를 참고하기 바란다.

/etc/fdprm
플로피 디스크 파라미터 테이블이다. 이 파일은 비슷비슷한 플로피 디스크들 사이의 차이점에 대한 정보를 담고 있다. 더 자세한 내용은 setfdprm 매뉴얼 페이지를 참고하기 바란다.

/etc/fstab
이 곳에는 시스템 시작시 mount -a 명령(/etc/rc 같은 곳에 설정되어 있다)에 의해 자동으로 마운트될 파일시스템들이 나열되어 있다. 리눅스의 경우에는 swapon -a 명령에 의해 사용되는 스왑 영역에 대한 정보도 수록되어 있다. 더 자세한 정보는 the section called 마운트하기와 마운트 풀기 in Chapter 4와 mount 매뉴얼 페이지를 참고하기 바란다.

/etc/group
/etc/passwd와 비슷하지만, 사용자들의 정보가 아닌 각 그룹들의 정보가 기재된다. 더 자세한 정보는 group 매뉴얼 페이지를 참고하기 바란다.

/etc/inittab
init의 설정파일이다.

/etc/issue
getty는 로그인 프롬프트가 뜨기 전에 이 파일의 내용을 화면에 뿌려준다. 이곳에는 시스템의 간단한 정보나 환영메시지를 적는 것이 보통이지만, 무엇을 적는냐하는 것은 전적으로 시스템 관리자 맘이다.

/etc/magic
file 명령의 설정 파일이다. 이곳에는 다양한 파일 형식들에 대한 정보가 포함되어 있는데, file 명령은 이것을 기반으로 파일의 정체를 추측해 낸다. magic과 file의 매뉴얼 페이지를 보면 더 많은 정보를 얻을 수 있다.

/etc/motd
Message Of The Day, 즉 '오늘의 메시지' 파일이다. 로그인할 때마다 자동으로 이 파일의 내용이 출력되며, 이곳에 어떤

내용을 적을 것인지는 시스템 관리자의 맘대로다. 보통은 시스템 가동 중지 예고 같은 것을 할 때 주로 쓰인다.

/etc/mtab
여기에는 현재 마운트되어 있는 파일 시스템의 목록이 들어 있다. 이것은 스크립트에 의해 초기화되며, mount 명령에 의해 그 내용이 자동으로 갱신된다. 마운트되어 있는 파일시스템의 목록이 필요한 경우에 쓰이는데, 예를 들면 df 명령이 이 파일을 읽는다.

/etc/shadow
섀도우 패스워드 소프트웨어가 설치되어 있는 시스템의 경우에는 이곳에 섀도우 패스워드가 보관된다. 섀도우 패스워드라는 것은 /etc/passwd 파일에서 암호화된 패스워드 부분만을 떼어내 /etc/shadow에 보관해 두는 것을 말한다; /etc/shadow는 단지 루트 사용자만이 읽을 수 있기 때문에 패스워드가 쉽게 크랙되는 것을 막을 수 있다.

/etc/login.defs
login 명령의 설정 파일이다.

/etc/printcap
/etc/termcap과 비슷한 것이지만 프린터를 위한 것이다. 문법도 다르다.

/etc/profile, /etc/csh.login, /etc/csh.cshrc
시스템이 시작될 때나 로그인이 이루어질 때, Bourne 쉘이나 C 쉘에 의해 실행되는 파일들이다. 이 파일들을 사용하면 모든 사용자들의 기본 환경을 설정해 줄 수 있다. 자세한 내용은 각각의 쉘에 대한 매뉴얼 페이지를 보기 바란다.

/etc/securetty
이 곳에서는 루트의 로그인이 허용되는 안전한 터미널을 지정한다. 보통은 가상 콘솔들만 나열되어 있는데, 이것은 누군가가 모뎀이나 네트워크를 통해 시스템에 침입하여 수퍼유저 권한을 얻는 일을 막도록 하기 위한 것이다.

/etc/shells
여기서는 신뢰할 수 있는 쉘이 어떤 것인지를 지정한다. chsh 명령으로 로그인 쉘을 바꿀 때 이 곳에 나열된 쉘들만 지정할 수 있다. 또한 FTP서비스를 제공하는 ftpd 서버 프로세스는 사용자의 쉘이 /etc/shells에 나열된 것과 일치하는 지를 확인하고, 만약 일치하지 않는다면 로그인을 거부한다.

/etc/termcap
여러가지 터미널들의 특성을 데이터베이스로 만들어 둔 것이다. 이 곳에는 다양한 종류의 터미널들이 각각 어떤 "이스케이프 시퀀스(escape sequence)"를 통해 제어될 수 있는지 기재되어 있다. 프로그램들은 현재 터미널의 종류가 어떤 것인지를 확인하고 /etc/termcap에서 해당 터미널에 알맞는 이스케이프 시퀀스를 찾아서 사용하게 된다. 따라서 각각의 프로그램이 터미널들의 특성에 대해 일일이 알고 있을 필요가 없으면서도, 대부분의 터미널에서 잘 동작하게 된다. 더 자세한 내용은 termcap, curs_termcap, terminfo 매뉴얼 페이지를 참고하기 바란다.

/dev 디렉토리
/dev 디렉토리는 모든 하드웨어 장치에 대한 장치 파일들을 가지고 있다. 장치 파일들의 이름은 특별한 명명법을 가지고 있다; 이 명명법은 Linux device list 문서에 설명되어 있다. (장치 파일들은 설치시에 생성되며, 설치 후에는 /dev/MAKEDEV 스크립트에 의해 생성될 수 있다.) /dev/MAKEDEV.local은 시스템 관리자가 작성하는 스크립트로서 특정한 로컬 장치 파일들을 생성하거나 링크를 만드는 데 쓰인다(즉 표준 MAKEDEV 파일에 그 내용이 없는 몇몇 비 표준적인 장치 드라이버들을 위한 스크립트이다).

/usr 파일시스템
/usr 파일시스템은 쉽게 커지는데, 모든 프로그램들이 이 곳에 설치되기 때문이다. 보통 /usr 디렉토리에는 배포판에서 제공하는 파일들이 들어 있으며, 그 밖에 따로 설치되는 프로그램들과 내부적 용도의 프로그램들은 /usr/local에 들어가는 것이 일반적이다. 이렇게 하면, 배포판을 업그레이드 하거나 아예 새로운 배포판으로 바꾼다고 해도 전체 프로그램을 다시 설치할 필요가 없게 된다. /usr의 몇몇 하위 디렉토리들을 아래에 설명하였다(몇가지 중요하지 않은 디렉토리들은 설명하지 않았다; 이들에 대한 자세한 내용은 FSSTND를 참고하기 바란다).

/usr/X11R6
X Window System의 모든 파일들이 이곳에 들어 있다. X의 개발과 설치를 보다 손쉽게 하기 위해서, X는 전체 디렉토리 트리에 통합되지 않고 독자적인 디렉토리 트리를 가지고 있다. 그래서 /usr/X11R6의 디렉토리 구조는 /usr 자체의 디렉토리 구조와 아주 흡사하게 되어 있다.

/usr/X386
/usr/X11R6과 비슷한 것으로, X11 Release 5 를 위한 것이다.

/usr/bin
사용자들을 위한 대부분의 명령들이 이 곳에 들어있다. 그 밖에 몇몇은 /bin이나 /usr/local/bin에 있기도 한다.

/usr/sbin
시스템 관리를 위한 명령들 중, 루트 파일시스템에는 있을 필요가 없는 명령들이 이 곳에 있게 된다. 즉, 대부분의 서버 프로그램들이 이 곳에 위치한다.

/usr/man, /usr/info, /usr/doc
각각 매뉴얼 페이지, GNU Info 문서들, 그리고 기타 다른 문서들을 위한 공간이다.

/usr/include
C programming language를 위한 헤더 파일들이 이 곳에 있다. 원칙적으로는 /usr/lib 아래에 있어야 하겠지만, 예전부터 이 위치에 있어왔던 전통이 워낙 강력해서 아직도 이곳에 남아 있게 되었다.

/usr/lib
프로그램들과 서브시스템들의 고정적인 데이터 파일들이 이곳에 위치한다. 또한 전체 시스템에 폭넓게 적용될 수 있는 site-wide한 설정 파일들도 이곳에 있다. lib이라는 이름은 library에서 유래된 것으로, 원래는 이곳이 programming subroutine들의 라이브러리가 있던 곳이었기 때문에 이런 이름이 붙게 되었다.

/usr/local
이 곳은 내부적인 용도의 프로그램들과 기타 파일들을 위한 곳이다.

/var 파일시스템
/var 파일시스템에는 시스템 운용 중 계속 갱신되는 데이터들이 모여 있다. 이 데이터들은 각 시스템에 고유한 것으로서, 네트워크를 통해 공유될 수 있는 성질의 것이 아니다.

/var/catman
이 곳은 포맷된 매뉴얼 페이지(man page)들이 잠시 대기(cache)하는 곳이다. 매뉴얼 페이지는 여러가지 형식으로 출력될 수 있는데, 출력될 형식에 알맞도록 먼저 포맷을 한 후 보게 된다. 포맷되지 않은 매뉴얼 페이지들은 보통 압축된 형태로 /usr/man/man* 에 위치한다; 어떤 매뉴얼 페이지들은 미리 포맷되어 있기도 한데, 이런 것들은 /usr/man/cat* 에 들어있는 것이 일반적이다. 포맷은 처음 볼 때만 한번하면 되고, 그 뒤에 같은 페이지를 보는 사람은 /var/man에 있는 것을 바로 꺼내볼 수 있으므로 포맷될 때까지 기다릴 필요가 없어진다(/var/catman 디렉토리는 자주 깨끗이 해주어야 하는데, 이것은 임시 디렉토리 안을 자주 지워줘야 하는 것과 같은 이유에서다).

/var/lib
일반적인 시스템 운용시 계속 갱신되는 파일들을 위한 공간이다.

/var/local
/usr/local 아래에 설치된 프로그램(즉, 시스템 관리자가 설치한 프로그램)들의 다양한 데이터가 보관되는 곳이다. 그 밖에 내부적으로 사용할 목적으로 설치된 프로그램이라 하더라도 /var의 하위 디렉토리를 사용하여 데이터를 보관하는 것이 좋은데, 예를 들면 /var/lock 같은 것이 있겠다.

/var/lock
잠금 파일(lock file)이 있는 곳이다. 많은 프로그램들이, 특정한 장치나 파일을 독점적으로 사용하고 있을 때 /var/lock에다 잠금 파일을 만드는 관례를 따르고 있다. 다른 프로그램들은 /var/lock에 잠금 파일이 있는지 알아보고 장치나 파일의 사용 여부를 결정하게 된다.

/var/log
다양한 프로그램들의 로그 파일이 있는 곳인데, 그 중에서도 특히 login과 syslog의 로그 파일이 이곳에 위치한다. login의 로그 파일은 /var/log/wtmp에 위치하며, 시스템의 모든 로그인, 로그아웃 정보를 기록한다. syslog의 로그 파일은 /var/log/messages에 위치하며, 커널과 시스템 프로그램들의 모든 메시지들을 기록한다. /var/log 안에 있는 파일들은 크기가 무제한으로 커질 수 있으므로 정기적으로 지워주어야 한다.

/var/run
이 곳에 있는 파일들은 시스템의 현재 정보들을 담고 있는데, 부팅을 다시하면 그 내용이 바뀌게 되는 것들이다. 예를 들면, 현재 로그인한 사용자들에 대한 정보는 /var/run/utmp 파일에 기록되어 있다.

/var/spool
메일이나 뉴스, 프린터 큐 같은, 대기 상태에 있는 작업들을 위한 디렉토리가 이 곳에 있으며, 각각의 작업들은 /var/spool 밑에 고유의 하위 디렉토리를 가지고 있다. 예를 들면, 각 사용자들의 편지함은 /var/spool/mail 아래에 위치하고 있는 식이다.

/var/tmp
/tmp에 있는 임시 파일들보다는 좀 더 오래 유지될 필요가 있는 임시 파일들이 이곳에 오게 된다(이곳에 있는 파일들 중에서도 아주 오래된 파일들은 시스템 관리자가 직접 지워 버릴 것이다).

/proc 파일시스템
/proc 파일시스템은 실제로 존재하지 않는 일종의 환영이다. 이 파일시스템은 커널이 메모리 상에 만들어 놓은 것으로 디스크에는 존재하지 않는다. /proc은 시스템의 갖가지 정보를 제공해 주는데, 원래는 주로 프로세스에 대한 정보를 제공했기 때문에 proc(process)이란 이름을 갖게 되었다. 이 곳에 있는 중요한 파일과 디렉토리들을 아래에 설명하였다. /proc 파일시스템에 관한 더욱 자세한 정보는 /proc 매뉴얼 페이지를 찾아보기 바란다.

/proc/1
프로세스 번호 1번에 대한 정보가 있는 디렉토리이다. 각 프로세스는 자신만의 디렉토리를 /proc 아래에 갖고 있게 되는데, 자신의 프로세스 식별 번호(process identification number)가 그 디렉토리의 이름이 된다.

/proc/cpuinfo
프로세서의 정보가 들어있다. cpu의 타입, 모델, 제조회사, 성능 등에 관한 정보를 알려준다.

/proc/devices
현재 커널에 설정되어 있는 장치의 목록을 볼 수 있다.

/proc/dma
현재 어느 DMA 채널이 사용 중인지를 알려준다.

/proc/filesystems
어떤 파일시스템이 커널에 설정되어 있는지를 알 수 있다.

/proc/interrupts
현재 어느 인터럽트가 사용 중인지, 그리고 얼마나 많이 사용되었는지를 알 수 있다.

/proc/ioports
현재 어느 I/O 포트가 사용 중인지를 알려준다.

/proc/kcore
이것은 시스템에 장착된 실제 메모리의 이미지이다(즉, 실제 메모리의 내용을 그대로 본뜬 것이다). 따라서 이 파일의 크기는 실제 메모리의 크기와 정확히 일치하는 것처럼 보인다. 그러나 이 파일은 프로그램이 필요로 하는 부분의 이미지만 그때 그때 만들어 내도록 되어 있어서, 실제로 메모리를 그만큼 차지하고 있는 것은 아니다. (/proc 파일시스템의 내용을 다른 곳에 복사하지만 않는다면, /proc 안의 내용은 아무런 디스크 공간을 차지하지 않는다는 점을 알아두자.)

/proc/kmsg
커널이 출력하는 메시지들이다. 이것은 syslog 파일에도 기록된다.

/proc/ksyms
커널이 사용하는 심볼들의 표를 보여준다.

/proc/loadavg
시스템의 평균부하량(load average)을 보여준다. 지금 시스템이 해야하는 일들이 얼마나 많은지 알려주는 세가지 지표를 볼 수 있을 것이다.

/proc/meminfo
메모리 사용량에 관한 정보를 보여준다. 실제 메모리와 가상 메모리를 모두 다룬다.

/proc/modules
현재 어떤 커널 모듈이 사용되고 있는지를 알려준다.

/proc/net
네트워크 프로토콜들의 상태에 대한 정보가 들어 있다.

/proc/self
이 곳은 이 디렉토리를 들여다보는 프로그램 자신의 프로세스 디렉토리로 링크가 되어 있다. 즉, 서로 다른 두 프로세스가 /proc를 본다면 그들은 서로 다른 링크를 보게 되는 것이다. 이렇게 하면 프로그램들이 자신의 프로세스 디렉토리가 어디인지를 쉽게 알 수가 있게 된다.

/proc/stat
이 곳에는 시스템의 상태에 관한 다양한 정보가 있다. 즉, 부팅된 후 page fault가 몇번 일어났는가 하는 것들을 알아 볼 수가 있다.

/proc/uptime
시스템이 얼마나 오랫동안 살아 있었는지 보여준다.

/proc/version
커널의 버전을 알려준다.

위에 나열한 파일들 대부분이 알아보기 쉬운 텍스트 파일로 되어 있긴 하지만, 어떤 경우에는 쉽게 알아보기 힘든 형식을 가지고 있기도 하다. 그래서 이런 파일들을 좀 더 쉽게 알아볼 수 있도록 해주는 많은 명령들이 준비되어 있다. 예를 들어 /proc/meminfo 파일은 메모리 사용량을 byte 단위로 나타내고 있는데, free 명령은 이것을 kilobyte 단위로 좀 더 알기 쉽게 나타내 준다(그리고 그 외에 몇가지 유용한 정보를 덧붙여 보여준다).

Chapter 4. 디스크 및 다른 저장장치 사용하기

"On a clear disk you can seek forever. "
"깨끗한 디스크 위에선, 영원을 추구할 수 있다. "

리눅스시스템을 설치하거나 업그레이드할 때에는 디스크에 많은 작업을 해야할 필요가 있다. 디스크에 파일을 저장하기 위해 디스크에 파일시스템을 만들어야 하고 시스템의 여러 부분들을 위해 공간을 확보해야 한다.

이 장은 이러한 모든 초기작업에 대해 설명한다. 보통 일단 시스템을 구성하고 나면 플로피를 사용하는 것을 빼고는 다시 그러한 작업을 안해도 될 것이다. 만약 새 디스크를 추가하거나 디스크를 잘 조절하여 사용하고 싶다면 이 장을 다시 읽을 필요가 있을 것이다.

디스크를 관리하는 기본적인 일들은 다음과 같다.

디스크 포맷하기. 배드섹터를 찾는 일같은 디스크를 사용하기 위한 여러가지 일들이다.(포맷하기는 요새 대부분의 하드디스크에서는 필요하지 않다.)

여러작업들이 서로 영향을 주지않도록 디스크를 사용하고 싶다면 하드디스크를 여러 파티션으로 나눠라. 파티션을 하는 이유는 첫째, 같은 디스크에 다른 운영체제를 설치할 수 있기 때문이고 둘째, 개인 파일과 시스템파일을 떼어놓아 백업을 간단하게 하고, 시스템이 망가졌을 때 시스템 파일을 보호하는데 보탬이 되기 때문이다.

알맞은 형식의 파일시스템을 각 디스크나 파티션에 만들어라. 파일시스템을 만들기 전까지 리눅스에 디스크는 아무 쓸모가 없다. 파일시스템을 만든 후부터 디스크에 파일을 만들수 있고 접근할수 있다.

하나의 트리구조를 만들기 위해 자동으로, 혹은 필요할때 수동으로 다른 파일시스템을 마운트해라.(수동으로 마운트한 파일시스템은 보통 수동으로 마운트를 풀어줄 필요가 있다.)

Chapter 5에서는 가상메모리와 디스크캐싱에 대한 정보를 포함하고 있는데 디스크를 사용할 때 알아둘 필요가 있다. 이 장은 하드디스크, 플로피, 시디롬, 테이프를 설명한다.

두 종류의 장치
유닉스 그리고 리눅스는 두종류의 다른 장치를 인식한다. 랜덤-억세스 블럭 디바이스(random-access block devices)(디스크같은)와 캐릭터 디바이스(character devices)(테이프나 시리얼라인같은)로 장치들의 일부분은 시리얼(serial)이고 일부분은 랜덤-억세스이다. 각 지원되는 장치들은 파일시스템에서 장치파일(device file)로 표시된다. 장치파일을 읽거나 장치파일에 쓰면 데이타는 그것이 가리키는 장치로 왔다갔다한다. 이러한 방법으로 특별한 프로그램(그리고 인터럽트를 잡는다던가 시리얼포트를 폴링한다던지의 특별한 프로그래밍 방법론도)은 장치에 접근하는데 필요치 않다. 예를 들면 프린터에 파일을 보낼 때 다음과 같이 하면 된다.

```
$ cat filename > /dev/lp1
$
```

그러면 파일의 내용이 프린트된다.(물론 파일이 프린터가 이해할수 있는 형식이어야 한다.) 그러나 같은 시간에 몇명의 사람들이 파일을 프린터로 보낼 수도 있으므로 보통 프린트할 파일을 프린트에 보내기 위해 특별한 프로그램(보통 lpr)이 사용된다. 이 프로그램은 한번에 하나의 파일이 프린트되도록 하며, 하나의 파일이 끝나면 바로 자동적으로 다음 파일을 보낸다. 비슷한 것이 다른 장치파일에도 필요하다. 사실 장치파일에 대해선 좀처럼 걱정할 필요가 없다.

장치들이 파일시스템에서 파일로(/dev 디렉토리안에서) 보여지므로 ls나 다른 적당한 명령으로 장치파일이 있는지 단지 보는것은 쉽다. ls -l의 결과에서 첫째 열은 파일의 퍼미션과 유형을 포함한다. 예를 들어, 시스템에서 시리얼 장치를 조사해보면,

```
$ ls -l /dev/cua0
crw-rw-rw-   1 root     uucp       5,  64 Nov 30  1993 /dev/cua0
$
```

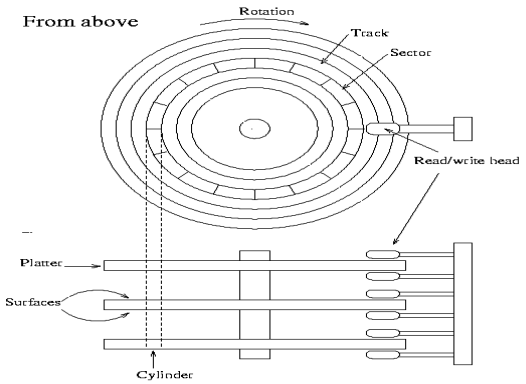
첫째 열의 첫째 글자는, 즉 crw-rw-rw-에서 `c'는 파일의 유형을 사용자에게 알려주는 것으로 이 경우는 캐릭터 디바이스(character devices)이다. 일반적인 파일의 첫째 글자는 `-'이고, 디렉토리는 `d'이고, 블록 디바이스(block devices)는 `b'이다.더 많은 정보를 원하면 ls의 man page를 보면 된다.

장치자체는 설치되어 있지 않더라도 보통 모든 장치파일은 존재한다는 것을 유의해야 한다. 그래서 단지 /dev/sda가 있다고 SCSI하드디스크가 있는건 아니다. 모든 장치 파일을 가지고 있는 것은 설치프로그램을 쉽게 만들고 새로운 하드웨어를 설치하는 것을 쉽게 한다.(정확한 파라미터를 찾을 필요도 새로운 장치를 위한 장치파일을 만들 필요도 없다.)

하드 디스크

이 절에서는 하드디스크와 관련된 용어를 소개한다. 만약 용어나 개념을 이미 알고 있다면 이 절은 넘어갈 수 있다.

Figure 4-1은 하드디스크안의 중요부분의 개략도이다.



하드디스크는 하나 이상의 둥그런 플래터(platter)로 구성되고, 플래터의 한면이나 양면은 데이타를 저장하기 위해 자기물질로 덮여 있다. [1] 각 표면(surface)마다 기록된 데이타를 조사하거나 바꾸는 읽기-쓰기 헤드(read-write head) 가 있다. 플래터는 축을 중심으로 회전하는데, 높은 수행능력을 가진 하드디스크는 더 빠른 속도로 돌지만 대표적인 속도는 분당 3600회전이다. 헤드는 플래터의 반지름을 따라 움직이고, 플래터의 회전과 혼합되어 헤드는 표면의 모든부분에 접근할수 있다.

CPU와 디스크는 디스크제어기(disk controller) 를 통해 통신한다. 다른 형식의 디스크에 달린 제어기라도 컴퓨터의 다른부분과의 인터페이스는 같기 때문에 디스크제어기를 통해 통신함으로써 컴퓨터의 나머지 부분은 드라이브를 어떻게 사용하는지 몰라도 된다. 그래서, 컴퓨터는 헤드를 적당한 위치로 옮기고 정확한 위치가 헤드 밑으로 올 때까지 기다리며, 필요한 다른 모든 즐겁지 않은 일을 길고 복잡한 전기신호들로 보내는 대신 단지 "야! 디스크, 내가 원하는 것 내봐."라고 하면 된다.(실상, 제어기에 있는 인터페이스는 여전히 복잡하나, 없는것보다는 낫다.) 또 제어기는 캐싱이나 자동으로 배드섹터를 교체하는 일도 한다.

위에 적은것이 보통 하드웨어에 대해 이해할 필요가 있는 모든 것이다. 플래터를 돌리고 헤드를 움직이는 모터, 기계적인 부분의 움직임을 제어하는 전자공학같은 많은 다른 요소들이 있지만 하드디스크의 작동원리를 이해하는데는 대부분 적절하지 않다.

표면은 보통 트랙(track) 중심이 같은 원으로 나뉘어지고 트랙은 차례로 섹터(sector)로 나뉘어진다. 이 구분은 하드디스크상의 위치를 나타내고 파일에 디스크공간을 할당하기 위해 사용된다. 하드디스크에서 정해진 위치를 찾기 위해 "표면 3, 트랙 5, 섹터 7"라고 말할 것이다. 보통 섹터수가 모든 트랙마다 같지만, 어떤 하드디스크는 바깥쪽 트랙에 좀더 많은 섹터를 만든다.(모든 섹터는 크기가 같아서 더 길이가 긴 바깥쪽 트랙에서는 더 많은 섹터수가 들어맞는다.) 일반적으로, 한 섹터는 512바이트의 정보를 지닐 것이다. 디스크 자체는 한섹터보다 더 작은 양의 데이타를 처리할 수 없다.

Figure 4-1. 하드디스크 구조의 간략한 도해

각 표면은 같은 방법으로 트랙(과 섹터)로 나뉘어진다. 이건 한 표면를 맡는 헤드가 한 트랙 위에 있으면 다른 표면을 맡는 헤드들도 상응하는 트랙 위에 있다는 것을 의미한다. 모든 상응하는 트랙들을 묶어서 실린더(cylinder) 라 한다. 한 트랙(실린더)에서 다른 트랙(실린더)로 움직이는 것은 시간이 걸리므로, 때로 같이 호출되는 데이타들(즉, 하나의 파일)을 한 실린더안에 있게 하면 그 모든 데이타들을 읽기 위해 헤드를 움직일 필요가 없어진다. 이것은 성능을 향상시킨다. 이런 식으로 파일을 놓는 것은 항상 가능한 것은 아니다. 디스크 상의 여러곳에 저장된 파일들은 프레그멘트(fragmented : 조각난, 산산히 부서진)되었다고 한다.

표면(혹은 헤드,숫자는 같다), 실린더, 섹터의 수는 매우 다양하다. 각 숫자의 명세 사항을 하드디스크의 결합구조(geometry)라 한다. 이들은 CMOS램(CMOS RAM)이라 불리우는 특별하고 건전지로 동력이 공급되는 기억장치에 저장되는데, 운영체제는 CMOS램으로부터 부팅시나 드라이버 초기화 때 결합구조를 불러올 수 있다.

불행히도, 바이오스(BIOS) [2] 는 설계의 한계를 지니고 있는데, CMOS램 안에 1024보다 큰 트랙 수를 명시하지 못한다는 것으로, 1024는 큰 하드디스크엔 넘 작다. 이걸 극복하기 위해, 디스크제어기는 결합구조에 대해 속이고, 컴퓨터가 준 어

드레스(address)를 현실에 맞는 것으로 변환시킨다. 예를 들면, 하드디스크가 8헤드, 2048트랙, 트랙당 35섹터를 가지고 있다고 하자. [3] 이 하드의 제어기는 컴퓨터에게 거짓말을 하고 하드가 트랙의 한계를 넘어서지 않는 16헤드와 1024트랙, 트랙당 35섹트를 가지고 있다고 선언하고, 컴퓨터가 준 어드레스를 헤드수는 절반으로 트랙수는 2배로 해서 변환시킬 수 있을 것이다. 현실에서는 계산이 더 복잡할 수 있는데, 숫자들이 이렇게 간단하진 않기 때문이다 (다시말하지만, 자세한 것은 원리를 이해하는데는 적합하지 않다). 이 변환은 디스크가 어떻게 구성되는지를 운영체제에게 왜곡시켜 보여줘서, 성능을 올리기 위해 모든 데이타를 한 실린더안에 넣는 것을 비현실적으로 만든다.

변환은 오로지 IDE디스크의 문제이다. SCSI디스크는 순차적인 섹터번호(즉, 제어기는 순차적인 섹터번호를 헤드,실린더와 섹터 세가지로 변환시킨다)와 시피유가 제어기와 통신하기 위해 완전히 다른 방법을 사용하므로 SCSI디스크는 위 문제와 상관이 없다. 그러나, 컴퓨터는 역시 SCSI디스크의 실제 결합구조를 알지 못한다는 점을 유의해라.

리눅스는 때로 실제 디스크 결합구조를 모를 것이기 때문에, 리눅스의 파일시스템은 파일들을 한 실린더 안에 저장하려고 하지 않는다. 대신, 순차적으로 번호가 매겨진 섹터들을 파일에 할당하려고 하고, 이건 거의 항상 비슷한 성능을 줄 것이다. 이 문제는 제어기에 달린 캐쉬와 제어기가 하는 자동적인 미리 불러오기에 의해 더 복잡해진다.

각 하드 디스크는 구별되는 장치파일로 나타내어진다. 보통 단지 2개 혹은 4개의 IDE 하드디스크가 있다. 각각 /dev/hda, /dev/hdb, /dev/hdc, /dev/hdd가 된다. SCSI하드디스크는 /dev/sda, /dev/sdb 이런 식으로 된다. 다른 하드디스크 형식에도 비슷하게 이름을 만드는 관례가 있다. 하드디스크를 위한 장치파일은 파티션(다음에 이야기 할 것이다)에 상관없이 전체 디스크에 접근하며, 주의하지 않는다면 전체 디스크안에 있는 파티션이나 데이타가 망가질 것이다. 디스크 장치파일은 보통 오로지 master boot record(역시 다음에 이야기할 것이다)에 접근하기 위해 사용된다.

Notes
[1] 플래터는 알루미늄같은 단단한 물질로 구성되며 그래서 하드디스크라는 이름이 붙었다.

[2] 바이오스란 롬칩에 저장된 작은 내장 소프트웨어이다. 여기서 부팅시 초기단계를 관리한다.

[3] 숫자들은 완전히 상상한 것이다.

플로피
플로피디스크는 하드디스크처럼 자기 물질로 한면 혹은 양면이 둘러싸인 유연한 막으로 구성된다. 플로피디스크 자체는 읽기-쓰기 헤드가 없고, 드라이브에 포함된다. 하나의 플로피는 하드디스크의 한 플래터에 해당하나, 하드디스크는 나눌 수 없는 반면, 플로피는 제거가 가능하고 한 드라이브는 다른 플로피를 사용할 때 사용될 수 있다.

하드디스크처럼, 플로피는 트랙과 섹터로 구분되나(그리고 양면 2개의 대응하는 트랙은 실린더를 이룬다), 하드디스크에 있는 것보단 매우 적다.

플로피드라이브는 몇가지 다른 디스크형식을 사용할 수 있다. 예를 들면 3.5인치드라이브는 720kB 와 1.44MB디스크를 모두 사용할 수 있다. 플로피드라이브는 약간 다르게 작동되어야 하고 운영체제는 디스크의 용량이 얼마나 큰지 반드시 알아야 하므로, 플로피드라이브를 위해 드라이브와 디스크형식의 조합에 하나씩 많은 장치파일이 있다. 그래서, /dev/fd0H1440는 3.5인치, 크기 1440kB(1440)의 고밀도(H)디스크,즉 평범한 3.5인치 HD 플로피를 사용하고, 반드시 3.5인치 드라이브이어야 하는 첫째 플로피드라이브(fd0)이다.

그러나, 플로피드라이브를 위한 이름이 복잡해서, 리눅스에는 드라이브안에 있는 디스크의 형식을 자동으로 알아내는 특별한 플로피장치 형식이 있다. 그건 알맞은 형식을 찾을 때까지 다른 플로피형식을 사용해 새로 집어넣은 디스크의 첫 섹터를 읽는다. 자연히 이건 먼저 플로피를 포맷시키는 것을 요구한다. 자동장치들을 /dev/fd0,/dev/fd1과 같이 부른다.

자동장치가 디스크에 접근하기 위해 사용하는 변수들은 setfdprm을 이용해서 조절할 수도 있다. 만약 디스크 크기가 아닌, 예를 들면, 섹터수가 보통이 아닌, 디스크를 사용할 필요가 있을 때나, 어떤 이유로 자동감지가 실패하고, 알맞은 장치파일이 사라졌다면 사용될 수 있다.

리눅스는 모든 표준 외에도 많은 표준이 아닌 플로피 디스크도 다룰 수 있다. 비표준 중 일부는 특별한 포맷 프로그램을

요구할 것이다. 지금은 이러한 디스크 형식을 다루지 않겠지만, 중간에 당신이 /etc/fdprm 파일을 조사할 수 있다. 그 파일은 setfdprm이 인지하는 설정들을 열거하고 있다.

운영체제는 플로피드라이브 안의 디스크가 바뀌었을 때, 예를 들면, 전 디스크로 부터 캐쉬된 데이타를 사용되는 것을 피하기 위해서 알아야 한다. 불행히도 이걸 위해 사용되는 신호선이 때때로 끊어지거나, 더 나쁘게도, MS-DOS에서 드라이브를 사용할 때는 항상 인지가능하지는 않을 것이다. 만약 플로피를 사용하면서 이상한 문제를 경험한다면, 방금 말한 것이 이유가 될 수도 있다. 그걸 정정하는 유일한 방법은 플로피드라이브를 수리하는 것이다.

CD-ROM

시디롬 드라이브는 광학적으로 읽히는 플라스틱 코팅된 디스크를 사용한다. 정보는 디스크표면 [1] 위에 있는, 중심으로부터 바깥으로 나가는 나선형을 따라 정렬된 조그마한 구멍에 기록된다. 드라이브는 디스크를 읽기 위해 나선형을 따라 레이저빔을 쏜다. 레이저가 구멍에 부딪혔을 때, 레이저는 같은 방향으로 반사되고, 부드러운 표면에 부딪히면, 다른 방향으로 반사된다. 이건 비트, 곧 정보를 코드화하는 것을 쉽게 만든다. 다른 부분은 단지 기계적인 부분으로 쉽다.

시디롬 드라이브는 하드디스크와 비교해서 느리다. 전형적인 하드디스크는 평균적인 탐색시간이 15밀리초 미만일 것이나, 빠른 시디롬 드라이브는 찾는데 영점 몇초 정도 걸릴 것이다. 실제 데이타 전송 비율은 초당 수백 킬로바이트 정도로 꽤 높다. 느리기 때문에, 사용가능하지만 시디롬드라이브를 하드디스크대신 사용하는 건 즐겁지 않다(어떤 리눅스 배포본은 하드디스크에 파일을 복사할 필요없게 해서, 인스톨을 쉽게 그리고 하드디스크 공간을 많이 절약하기 위해 시디롬에 '라이브(live)' 파일시스템을 제공한다.) 프로그램 설치할 때는 최고 속도가 필수적인 것이 아니므로, 새로운 소프트웨어를 설치하기 위해 시디롬을 사용하는 것은 매우 좋다.

시디롬에 데이타를 배열하는 방법은 여러가지가 있다. 가장 대중적인 것은 국제 표준 ISO 9660에 명시되어 있다. ISO 9660은 아주 작은 파일시스템을 명시하고 있는데, MS-DOS가 사용하는 파일시스템보다 훨씬 조잡하다. 반면에 매우 작아서 모든 운영체제들이 자기 고유의 시스템에 ISO 9660을 대응시키는 것이 가능할 것이다.

평범한 유닉스 사용에 ISO 9660 파일시스템은 사용할 수 없어서, 록 릿지 확장(Rock Ridge extension)이라 부르는 표준을 확장한 것이 개발되었다. 록 릿지는 시디롬이 다소간 현재의 유닉스 파일 시스템과 비슷하도록, 긴 파일명, 심볼릭링크와 그외 다른 많은 매력있는 것들을 가능하도록 한다. 훨씬 좋은건, 록 릿지 파일시스템이 여전히 유닉스가 아닌 운영체제에서도 사용가능한 정확한 ISO 9660파일시스템이라는 것이다. 리눅스는 ISO 9660과 록 릿지확장 모두를 지원한다. 록 릿지 확장은 자동적으로 인지되서 사용되어진다.

그러나, 문제는 파일시스템에만 그치는 것이 아니다. 대부분의 시디롬은 접근하기 위해 특별한 프로그램을 요구하는 데이타를 포함하고 있고, 그 프로그램들의 대부분은 리눅스에서 는 돌아가지 않는다(리눅스 MS-DOS 에뮬레이터인 dosemu로 가능한 것은 제외한다).

시디롬 드라이브는 대응되는 장치파일을 통해 접근할 수 있다. 시디롬 드라이브를 컴퓨터에 연결하는 방법은 몇가지가 있다. SCSI를 통해, 사운드카드를 통해, 그리고 EIDE를 통해서이다. 연결하기 위해 하드웨어에 대해 자세히 알아보는 건 이 책이 다루는 범위를 벗어난다.

Notes

[1] 즉, 플라스틱 코팅 안쪽 금속 디스크 위의 표면

테이프

테이프 드라이브는 음악을 위해 사용되는 카세트와 비슷한 [1] 테이프를 사용한다. 테이프는 사실상 시리얼로, 테이프의 어떤 부분에 이르기 위해 먼저 사이의 모든 부분을 통과해서 가야 되는 걸 의미한다. 디스크는 맘대로 접근할 수 있다. 즉 디스크상의 어느 곳이나 바로 갈 수 있다. 테이프가 시리얼 접근을 사용하는 것은 테이프를 느리게 만든다.

반면에, 테이프는 빠를 필요가 없기 때문에 만드는데 비교적 비용이 저렴하다. 뿐만 아니라 테이프는 쉽게 상당히 길게 만들 수 있어서, 많은 양의 데이타를 저장할 수 있다. 이런 이유로 테이프는 큰 속도는 요구하지 않으나 낮은 비용과 큰 저장용량으로 이익을 얻을 수 있는, 파일모으기와 백업같은 일에 매우 적합하다.

Notes
[1] 물론 완전히 다르다.

포맷하기
포맷한다(Formatting)는 것은 자기 매체에 트랙과 섹터를 표시하는 과정이다. 디스크가 포맷되기 전에는 자기 표면(magnetic surface)은 완전히 자기신호의 덩어리이다. 포맷되었을 때, 어디서 트랙이 이루어지고, 섹터가 나누어지는지 필수적인 선을 그림으로써 혼돈상태가 약간의 질서상태로 된다. 실제적인 자세한 것은 이와 같지 않지만, 상관없다. 중요한 것은 디스크가 포맷되지 않는다면 사용하지 못한다는 점이다.

여기서 용어가 약간 헷갈릴 것이다. MS-DOS에서는 포맷한다는 말이 파일시스템을 만드는 과정(나중에 설명된다)도 포함하면서 사용된다. 두 작업이 때때로 합쳐지기도 한다. 특히 플로피의 경우가 그렇다. 구별이 필요할 때, 파일시스템을 만드는 것은 high-level formatting이라고 하고, 진짜 포맷하는 것을 low-level formatting이라고 한다. 유닉스 안에서는 두 가지를 파일시스템 만들기와 포맷하기라고 하고, 이 책에서도 역시 그렇게 사용한다.

IDE 디스크와 약간의 SCSI 디스크는 공장에서 실제적으로 포맷이 되어서 반복할 필요가 없다. 그러므로 대부분의 사람들은 포맷에 대해 거의 걱정할 필요가 없다. 실은 하드디스크를 포맷하는 것은 디스크가 다소 잘 작동하지 않도록 할 수 있다. 예를 들면 자동으로 배드섹터를 교체하도록 하기 위해서는 매우 특별한 방법으로 디스크를 포맷할 필요가 있기 때문이다.

드라이브 내부 포맷 로직 인터페이스가 드라이브마다 다르기 때문에, 포맷할 필요가 있거나 포맷해야 하는 디스크는 때때로 특별한 프로그램을 요구한다. 포맷프로그램은 때때로 바이오스에 있기도 하고, 혹은 MS-DOS프로그램으로 제공되기도 하지만 그 어느 것도 리눅스에서 쉽게 사용할 수 없다.

포맷하는 동안 배드블럭(bad blocks)이나 배드섹터(bad sectors)라고 불리는 디스크상의 잘못된 곳을 만날 수 있다. 때때로 드라이브 자체적으로 처리되지만, 만약 더 많이 나타난다면 디스크의 배드난 부분을 사용하는 것을 피하기 위해 다른 일을 해야한다. 배드난 곳을 피하는 논리는 파일시스템에 포함된다. 파일시스템 안에 정보를 어떻게 첨가하는지는 다음에 설명한다. 대안으로, 배드난 부분을 포함하는 작은 파티션을 만들 수 있다. 파일시스템은 매우 큰 배드가 있으면 때때로 문제를 일으키므로 만약 배드난곳이 매우 넓다면 작은 파티션을 만드는 것이 좋은 생각이다.

플로피는 fdformat으로 포맷한다. 사용할 플로피 장치파일은 매개변수로 주어진다. 예를 들어, 첫번째 플로피드라이브 안에 있는 고밀도 3.5인치 플로피를 포맷하는 경우를 보자. $ fdformat /dev/fd0H1440
Double-sided, 80 tracks, 18 sec/track. Total capacity 1440 kB.
Formatting ... done
Verifying ... done
$

만약 자동감지 장치 (예를 들어 /dev/fd0)를 사용한다면 먼저 setfdprm을 이용해서 장치의 매개변수를 지정해주어야 한다는 점을 주의해라. 명령과 똑같은 효과를 얻으려면 다음과 같이 해야할 것이다. $ setfdprm /dev/fd0 1440/1440
$ fdformat /dev/fd0
Double-sided, 80 tracks, 18 sec/track. Total capacity 1440 kB.
Formatting ... done
Verifying ... done
$

플로피의 형식에 맞는 정확한 장치파일을 고르는 것이 보통 더 필요하다. 디자인 된 것보다 더 많은 정보를 담도록 플로피를 포맷하는 것은 현명하지 않다는 점을 주의해라.

fdformat 시 플로피를 확인한다, 즉 배드 블럭이 있는지 체크한다. 배드블럭을 몇차례 확인하려고 할 것이다(이과정을 들을 수 있다. 드라이브에서 극적으로 소리가 바뀔 것이다.) 만약 플로피가 오로지 부분적으로 배드가 났다면(읽기/쓰기 헤드의 먼지때문에, 몇개의 에러는 잘못된 신호이다), fdformat는 불평하지 않을 것이나, 진짜 에러는 플로피 확인 작업을 중지시킬 것이다. 커널은 발견한 I/O 에러를 로그메시지에 기록할 것이다. 메시지는 콘솔로 가거나, 만약 syslog가 사용

된다면 /usr/log/messages 파일로 갈것이다. fdformat 자신은 에러가 어디서 일어났는지 말하지 않을 것이다(보통 염려하지 않는데, 플로피는 배드난 것은 던저버려도 될만큼 충분하게 싸다). $ fdformat /dev/fd0H1440
Double-sided, 80 tracks, 18 sec/track. Total capacity 1440 kB.
Formatting ... done
Verifying ... read: Unknown error
$

badblocks 명령은 배드블럭을 찾기 위해 어떤 디스크나 파티션(플로피를 포함해서)을 탐색하는데 사용될 수 있다. badblocks는 디스크를 포맷하지 않아서, 존재하는 파일시스템을 체크하는데 사용할 수 있다. 아래 예제는 배드블럭 2개를 가지고 있는 3.5인치 플로피를 체크한다. $ badblocks /dev/fd0H1440 1440
718
719
$

badblocks는 발견한 배드블럭의 블럭 번호를 출력한다. 대부분의 파일시스템은 그런 배드블럭을 피할 수 있다. 파일시스템은 알려진 배드블럭 목록을 관리하는데, 그 목록은 파일시스템이 만들어질 때 초기화되고 나중에 수정할 수 있다. 배드블럭을 처음에 찾는 것은 mkfs 명령(파일시스템을 초기화하는)에 의해 행해질 수 있으나, 나중에 체크하는 것은 반드시 badblocks에 의해 행해져야 하며, 새로운 블럭은 fsck로 첨가되어야 한다. mkfs와 fsck는 나중에 설명할 것이다.

최근의 많은 디스크는 자동적으로 배드블럭을 알아차리고, 대신에 특별히 확보된 좋은 블럭으로 배드블럭을 고치려고 시도할 것이다. 이 과정은 운영체제에는 보이지 않는다. 만약 디스크가 배드블럭을 자동을 고치는지 알고 싶다면 그런 특징은 디스크매뉴얼에 문서로 있을 것이다. 만약 배드블럭의 수가 매우 많이 증가하게 된다면, 디스크가 녹슬어 사용하지 못할 때까지 기회는 있겠지만, 자동으로 고치는 기능을 지니는 디스크조차도 실패할 수 있다.

파티션
하드디스크는 몇개의 파티션(partitions)으로 나누어질 수 있다. 각 파티션은 마치 다른 하드디스크처럼 동작한다. 만약 하나의 디스크를 가지고 있는데 두개의 운영체제를 사용하고 싶다면 디스크를 두개의 파티션으로 나눌 수 있다. 각 운영체제는 자신의 파티션을 원하는 대로 사용하고 다른 쪽을 건들지 않는다. 이런 방식으로 두개의 운영체제가 같은 디스크 안에 평화적으로 공존할 수 있다. 파티션이 없다면 다른 운영체제를 위해 하드디스크를 하나 사야할 것이다.

플로피는 파티션으로 나누지 않는다. 이것을 막는 기술적인 이유는 없으나 플로피는 너무 작아서, 파티션으로 나누는 것은 쓸모있는 경우가 매우 드물것이다. CD-ROM도 역시 보통 파티션을 나누지 않는다. 시디롬을 한 큰 디스크로 사용하는 것이 더 쉽고, 몇개의 운영체제를 시디롬에 설치할 필요가 좀처럼 없기 때문이다.

MBR, 부트섹터, 파티션 테이블
하드디스크가 어떻게 나누어져 있는가에 대한 정보는 하드의 첫번째 섹터에 저장된다(즉 첫번째 디스크 표면위에 있는 첫번째 트랙의 첫번째 섹터). 이 첫번째 섹터가 바로 master boot record (MBR)이다. MBR은 컴퓨터가 처음 부팅될 때 바이오스가 읽어들이고 시작하는 섹터이다. master boot record는 파티션 정보를 읽어들이고, 어떤 파티션이 부팅 가능한 파티션인지, 각 파티션의 boot sector인 첫번째 섹터(MBR도 역시 부트섹터이나 MBR은 특별한 상태여서 특별한 이름을 가지고 있다.)를 읽어들이는 조그마한 프로그램을 포함하고 있다.

파티션 설계는 하드웨어에 내장되는 것도 아니고 바이오스에 있는 것도 아니다. 파티션은 많은 운영체제들이 따르는 관습일 뿐이다. 모든 운영체제들이 파티션 설계를 따르는 것은 아니지만, 그런 운영체제는 예외일 뿐이다. 어지간한 운영체제는 파티션을 지원하나, 그 운영체제들은 하드디스크의 한 파티션을 차지하고 그 파티션안에서 그 운영체제 내부의 파티션 방법을 사용한다. 나중 형식이 다른 운영체제(리눅스를 포함하는)와 평화스럽게 공존하고, 다른 특별한 수단을 요구하지 않으나, 파티션을 지원하지 않는 운영체제는 같은 디스크상에 다른 운영체제와 공존할 수 없다.

안전책으로, 종이에 파티션 정보를 적어두는 것이 좋다. 만약 파티션이 망가졌을 경우 모든 파일들을 날리지 않아도 되기 때문이다.(망가진 파티션은 fdisk로 고칠 수 있다.). 관련 정보는 fdisk -l 명령으로 얻을 수 있다. $ fdisk -l /dev/hda

Disk /dev/hda: 15 heads, 57 sectors, 790 cylinders

```
Units = cylinders of 855 * 512 bytes

   Device Boot  Begin   Start     End  Blocks   Id  System
/dev/hda1           1       1      24   10231+  82  Linux swap
/dev/hda2          25      25      48   10260   83  Linux native
/dev/hda3          49      49     408  153900   83  Linux native
/dev/hda4         409     409     790  163305    5  Extended
/dev/hda5         409     409     744  143611+  83  Linux native
/dev/hda6         745     745     790   19636+  83  Linux native
$
```

확장파티션과 논리 파티션

PC하드 디스크의 본래 파티션 설계는 오로지 4개의 파티션만 허용한다. 4개만 허용하는 것은 실생활에서 너무 작다는 것이 빠르게 알려졌는데, 상당수의 사람들이 4개의 운영제체 이상을 (Linux, MS-DOS, FreeBSD, NetBSD, Windows/NT, 그외 약간의 운영체제들) 사용하길 원한다는 것이 부분적인 이유이나, 주된 이유는 때때로 한 운영체제가 몇개의 파티션을 가지는 것이 좋기 때문이다. 예를 들어, 스왑공간은 속도문제 때문에 리눅스의 주된 파티션에 있는 대신 스왑공간 고유의 파티션에 있는 것이 가장 좋다.(다음에 설명한다.)

이 설계문제를 극복하기 위해 확장파티션(extended partitions)이 개발되었다. 확장파티션을 통해 primary partition을 하위 파티션들로 나눌 수 있다. 나뉘어지는 primary partition이 확장파티션이고 하위파티션이 논리파티션(logical partition)이다. 논리파티션은 primary [1] partition처럼 행동하나 다르게 만들어진다. primary partition과 논리파티션 사이에는 속도차이는 없다.

하드디스크의 파티션 구조는 Figure 4-2와 같이 보일 수도 있다. 디스크는 3개의 primary partition으로 나누어져있고, primary partition 중 2번째는 2개의 논리파티션으로 나누어져있다. 디스크의 일부분은 파티션되어 있지 않다. 디스크 전체적으로, 그리고 각각 primary partition은 부트섹터를 가지고 있다.

Figure 4-2. 하드디스크 파티션의 한 예

파티션 형식

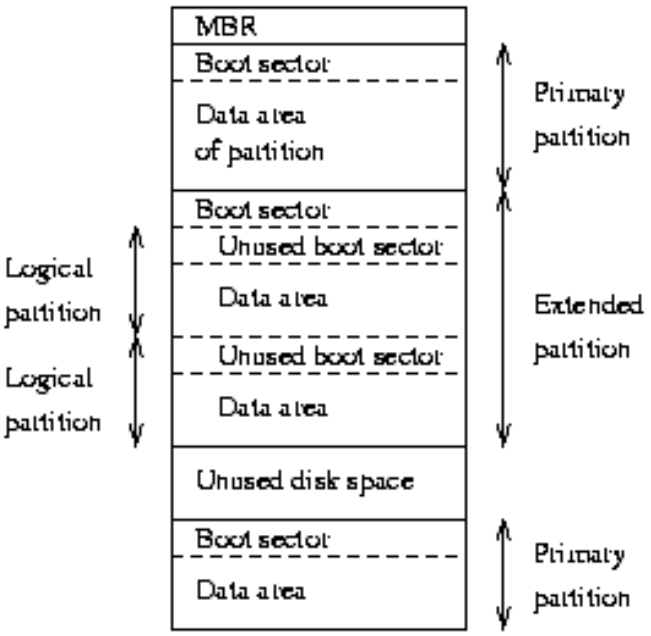
파티션 정보(MBR에 하나, 확장파티션에 하나씩 있는)에는 각 파티션의 형식을 확인하는 1바이트가 파티션당 하나씩 있다. 그 1바이트로 파티션을 사용하고 있는 운영체제를 확인하거나, 운영체제가 어떤 목적으로 그 파티션을 사용하는지 확인하려고 할 것이다. 우연히 같은 파티션을 사용하는 2개의 운영체제를 피하는 것을 가능하게 하기 위해서이다. 그러나 실제로 운영체제들은 파티션형식 바이트에 대해 걱정하지 않는다. 예를 들면, 리눅스는 파티션형식 바이트가 무엇인지 걱정하지 않는다. 나쁘게도, 약간의 운영체제들은 파티션형식 바이트를 부정확하게 사용한다. 예를 들면, 적어도 DR-DOS의 어떤 버전들은 파티션형식 바이트의 가장 중요한 비트를 무시한다.

각 바이트 값이 뜻하는 것을 명시한 표준단체가 없으나, 상당히 일반적으로 받아들여지는 값들이 Table 4-1에 포함되어 있다. 같은 값들이 리눅스 fdisk에서 사용가능하다.

Table 4-1. 파티션 형식 (리눅스의 fdisk 프로그램에서 따옴).

```
0 Empty 40 Venix 80286 94 Amoeba BBT
1 DOS 12-bit FAT 51 Novell? a5 BSD/386
2 XENIX root 52 Microport b7 BSDI fs
3 XENIX usr 63 GNU HURD b8 BSDI swap
4 DOS 16-bitf <32M 64 Novell c7 Syrinx
5 Extended 75 PC/IX db CP/M
6 DOS 16-bit >=32M 80 Old MINIX e1 DOS access
7 OS/2 HPFS 81 Linux/MINIX e3 DOS R/O
8 AIX 82 Linux swap f2 DOS secondary
```

9 AIX bootable 83 Linux native ff BBT
a OS/2 Boot Manag 93 Amoeba



하드디스크 파티션하기
파티션을 만들고 삭제할 수 있는 많은 프로그램들이 있다. 대부분의 운영체제는 그들 자신의 프로그램을 가지고 있고, 다른 운영체제에서 할 수 없는 특이한 것을 할 경우는 운영체제 고유의 프로그램을 사용하는 것이 좋은 생각일 것이다. 리눅스에 있는 것을 포함해서 많은 프로그램들을 fdisk라 하거나 약간 변종들도 있다. 리눅스 fdisk의 자세한 사용법은 man 페이지에 나와있다. cfdisk명령은 fdisk와 비슷하나, 좀더 좋은(전체화면) 사용자 인터페이스를 가지고 있다.

IDE디스크를 사용할 때, 부트 파티션(부팅가능한 커널 이미지 파일이 있는 파티션)은 반드시 첫 1024실린더 안에 완전히 있어야 한다. 디스크는 부팅중(시스템이 프로텍티드 모드로 가기 전) 바이오스를 통하여 사용되기 때문인데, 바이오스는 1024실린더 이상을 처리할 수 없다. 첫 1024실린더에 부분적으로 있을 뿐인 부트파티션을 사용하는 것이 때때로 가능하다. 이건 바이오스가 읽는 모든 파일들이 첫 1024실린더안에 있는 한 작동한다. 그렇게 정렬하는 것이 힘들기 때문에, 부분적으로 첫 1024 실린더에 부트파티션이 오게하는 것은 매우 나쁜 생각이다. 커널업데이트나 디스크 조각모음이 부팅할수 없는 시스템을 언제 초래할지 모르는 일이다. 그러므로, 부트 파티션이 첫 1024실린더 안에 완전히 있는지 확실히 해야 한다.

실은, 바이오스나 IDE 디스크의 몇몇 새 버전에서는 1024실린더 이상되는 디스크를 처리할 수 있다. 만약 그런 시스템이라면 1024 실린더문제를 잊어버려도 된다. 만약 시스템이 1024 실린더를 처리할 수 있는지 완전히 확신할 수 없다면, 첫 1024 실린더안에 부트 파티션을 집어넣어라.

리눅스 파일시스템은 1kB 블럭크기, 즉 2섹터를 사용하기 때문에, 각 파티션들은 짝수개의 섹터를 가져야 한다. 홀수로 섹터를 가지면 마지막 섹터를 사용 못하게 될 것이다. 문제를 일으키지는 않겠지만, 보기 흉하고, 버전에 따라 그것에 대해 경고하는 fdisk도 있을 것이다.

파티션 크기를 바꾸는 것은 보통 첫번째 그 파티션(단 경우에 따라서는 오히려 전체 디스크)에서 남기고 싶은 모든 것을 백업하고, 파티션을 삭제하고, 새로운 파티션을 만든 후, 새로운 파티션으로 모든 것을 다시 저장하는 과정이 필요하다. 파티션을 늘리는 거라면, 인접한 파티션 역시 크기를 조절하는(그리고 백업하고 다시 저장하기)것이 필요할 지도 모른다.

파티션 크기를 바꾼다는 것은 괴로운 일이기 때문에, 처음에 파티션을 적절히 하거나, 효율적이고 사용하기 쉬운 백업시스템을 가지는 것이 바람직하다. 만약 사람의 간섭이 필요없는 매체(플로피가 아니라 시디롬)로 설치하는 거라면, 때때로 처음에 다른 설정으로 설치하는 것이 쉽다. 백업할 데이타를 가지고 있지 않기 때문에, 여러번 파티션 크기를 수정하는 것이 고통스럽지 않다.

fips라는 MS-DOS프로그램이 있는데, 백업과 다시 저장할 필요 없이 MS-DOS 파티션의 크기조정을 하나, 다른 파일시스템을 위해서도 여전히 필요하다.

장치파일과 파티션
각 파티션과 확장파티션은 자신만의 장치파일을 가지고 있다. 전체 디스크의 이름에 1-4는 primary partition(얼마나 많은 primary partition이 있는지에 상관없이), 5-8은 논리파티션(논리파티션이 어떤 primary partition에 있는지 상관없이)으로 파티션 번호를 붙이는 것이 관습이다. 예를 들면, /dev/hda1은 첫번째 IDE 하드 디스크에 있는 첫번째 primary partition이고, /dev/sdb7은 두번째 SCSI 하드디스크에 있는 세번째 논리 파티션이다.

Notes
[1] Illogical?

파일시스템
파일시스템이란 무엇인가?
파일시스템(filesystem)이란 운영체제가 파티션이나 디스크에 파일들이 연속되게 하기 위해 사용하는 방법들이고 자료 구조이다. 즉, 파일들이 디스크상에서 구성되는 방식이다. 파일시스템이라는 말은 파일을 저장하는 데 사용되는 파티션이나 디스크를 가리킬 때나, 파일시스템의 형식을 가리킬 때 사용되기도 한다. 그래서 파일을 저장하는 2개의 파티션을 가지고 있다는 의미에서 어떤 사람이 "난 2개의 파일시스템을 가지고 있다."고 말할지도 모르고, 파일시스템의 형식을 의미해서 "extended filesystem"을 그 사람이 사용하고 있을 것이다

디스크나 파티션과, 디스크나 파티션이 포함하고 있는 파일시스템의 차이는 중요하다. 약간의 프로그램들(합리적으로 충분히 파일시스템을 만드는 프로그램을 포함해서)은 디스크나 파티션의 원시 섹터를 직접 조정한다. 만약 디스크나 파티션에 파일시스템이 존재한다면 그 파일시스템은 파괴되거나 심하게 망가질 것이다. 대부분의 프로그램들은 파일시스템 위에서 작동하며, 파일시스템이 없는(혹은 다른 형식의 파일시스템이 있는) 파티션에서는 작동하지 않을 것이다.

파티션이나 디스크가 파일시스템으로서 사용될 수 있게 되기 전에, 초기화되어야 하며, 파일정보 기록을 위한 자료구조를 디스크에 만들 필요가 있다. 이 과정을 파일시스템 만들기(making a filesystem)라고 한다.

정확한 세부사항은 상당히 다르지만, 대부분의 유닉스 파일시스템은 비슷한 전반적인 구조를 지닌다. superblock, inode, data block, directory block, indirection block이 중심 개념이다. 슈퍼블럭은 파일시스템 크기같은 전체적인 파일시스템에 대한 정보를 포함한다(여기에 들어가는 정보는 파일시스템에 의존한다). inode는 이름을 제외한 파일에 대한 모든 정보를 포함한다. 파일이름은 inode 번호와 함께 디렉토리안에 저장된다. 디렉토리 입구는 파일이름과 파일을 나타내는 inode 번호로 구성된다. inode는 몇개의 데이터블럭 번호를 포함하는데, 데이터블럭은 파일에서 데이타를 저장하기 위해 사용된다. 하지만 inode에는 오로지 약간의 데이터블럭 번호들을 위한 공간이 있어서, 만약 더 많이 필요하면 데이타블럭을 가리키는 포인터를 위한 더 많은 공간이 동적으로 할당된다. 이런 동적으로 할당된 블럭들은 간접적인 블럭들이다. 이름은 데이타블럭을 찾기 위해, 먼저 간접적인 블럭안에서 블럭의 번호를 찾아야한다고 가리킨다.

유닉스 파일시스템은 보통 파일안에 홀(hole)을 만들도록 하는데(홀을 만드는 건 lseek로 행해진다. 메뉴얼페이지를 조사해라), 파일시스템이 파일안의 특정한 장소에 단지 0바이트가 있는체 한다는 것을 의미하나, 파일안에서 그 곳을 위해 실제적인 디스크섹터는 없다(이건 파일이 디스크 공간을 다소 적게 사용할 것이라는 것을 의미한다). 특히 이런 일이 때때로 작은 바이너리, 리눅스 공유 라이브러리, 약간의 데이타베이스와 약간의 다른 특별한 경우에 일어난다. (홀은 inode나 간접적인 블럭안에 데이타 블럭의 주소로 특별한 값을 저장하므로 이루어진다. 이 특별한 주소는 그 파일의 그 부분에 할당된 데이타블럭이 없다는 것, 즉 파일안에 홀이 있다는 것을 의미한다.)

홀은 보통 쓸모있다. 저자의 시스템에서, 간단한 측정을 통해 약 200메가바이트 총용량의 하드에서 홀을 통해 약 4메가바이트의 절약이 있을 수 있음을 볼 수 있었다. 그러나 측정에 사용된 시스템은 비교적 프로그램이 거의 없고 데이타베이스 파일이 없다.

풍부한 파일시스템
리눅스는 몇가지 파일시스템을 지원한다. 이 글을 쓰고 있는 시점에서 중요한 파일시스템은 다음과 같다.

minix
가장 오래되었고 가장 신용할만 하다고 가정되나, 특징에서 다소 제한이 있고(몇몇 time stamp가 유실되고, 파일이름은 최대 30문자이다), 성능에 제한이 있다(파일시스템당 최대 64메가바이트).

xia
파일이름과 파일시스템 크기 한계를 끌어올린 minix 파일시스템을 수정한 버전이나, 새로운 특징은 없다. 매우 유명하지는 않으나 매우 잘 작동한다고 보고된다.

ext2

리눅스 파일시스템 본연의 대부분의 기능을 가지고 있고, 현재 가장 유명한 파일시스템. 쉽게 호환되면서 업되게 설계되어 있어서, 새 파일시스템 버전때문에 존재하는 파일시스템을 다시 만들 필요가 없다.

ext
상위 호환성이 없던 ext2의 구 버전. 설치시에 거의 사용하지 않고, 대부분의 사람들은 ext2로 전환했다.

여기에, 다른 운영체제와 파일 교환을 쉽게 하기 위해, 몇가지 외부의 파일시스템을 지원한다. 이 외부 파일시스템들은 유닉스 특징이 부족하다던가, 심각한 제한이 있다던가, 아니면 다른 특별한 점이 있는 경우를 제외하고 리눅스 파티션처럼 작동한다.

msdos
MS-DOS(OS/2와 Windows NT) FAT파일시스템과 호환

usmdos
msdos파일시스템을 리눅스상에서 긴 파일명, 소유자, 접근권한, 링크와 장치파일들을 지원하도록 확장한 것. umsdos는 보통의 msdos파일시스템이 리눅스 파일시스템처럼 사용되도록 하기 때문에, 리눅스를 위해 파티션을 나눌 필요를 없앤다.

iso9660
CD-ROM 표준 파일시스템. 시디롬 표준에 좀더 긴 파일명을 쓸 수 있는 확장한 유명한 록 릿지(Rock Ridge)가 자동으로 지원된다.

nfs
많은 컴퓨터들이 컴퓨터들의 파일에 서로 쉽게 접근하기 위해 컴퓨터들이 서로 파일시스템을 공유하도록 하는 네트웍 파일시스템(Nework FileSystem)

hpfs
OS/2 파일시스템

sysv
SystemV/386과 SystemV/386에서 나온 것들과 Xenix의 파일시스템

파일시스템의 선택은 상황에 따라 다르다. 호환성과 다른 이유로 리눅스 본래의 파일시스템이 아닌 것 중 하나가 필요하다면, 그것은 반드시 사용되어야 한다. 만약 자유롭게 고를 수 있다면 아마도 ext2를 사용하는 것이 가장 현명할 것이다. ext2는 모든 특성을 가지고 있고 수행능력이 부족해서 고생하지 않기 때문이다.

proc파일시스템이라는 것도 존재하는데, 보통 /proc 디렉토리로 접근할 수 있다. proc파일시스템은 파일시스템같이 보일지라도 실제로 전혀 파일시스템이 아니다. proc파일시스템은 프로세스 리스트(process list, proc파일시스템의 이름의 유래)같은 일정한 커널 데이타 구조에 접근하기 쉽게 한다. proc파일시스템은 이런한 데이타 구조를 파일시스템처럼 만들어 버리고, 이러한 파일시스템은 모든 평범한 파일도구로 다룰 수 있다. 예를 들어 모든 프로세스 리스트를 얻기 위해 다음 명령을 내릴 수 있다. $ ls -l /proc

```
total 0
dr-xr-xr-x   4 root     root            0 Jan 31 20:37 1
dr-xr-xr-x   4 liw      users           0 Jan 31 20:37 63
dr-xr-xr-x   4 liw      users           0 Jan 31 20:37 94
dr-xr-xr-x   4 liw      users           0 Jan 31 20:37 95
dr-xr-xr-x   4 root     users           0 Jan 31 20:37 98
dr-xr-xr-x   4 liw      users           0 Jan 31 20:37 99
-r--r--r--   1 root     root            0 Jan 31 20:37 devices
-r--r--r--   1 root     root            0 Jan 31 20:37 dma
```

```
-r--r--r--   1 root     root            0 Jan 31 20:37 filesystems
-r--r--r--   1 root     root            0 Jan 31 20:37 interrupts
-r--------   1 root     root      8654848 Jan 31 20:37 kcore
-r--r--r--   1 root     root            0 Jan 31 11:50 kmsg
-r--r--r--   1 root     root            0 Jan 31 20:37 ksyms
-r--r--r--   1 root     root            0 Jan 31 11:51 loadavg
-r--r--r--   1 root     root            0 Jan 31 20:37 meminfo
-r--r--r--   1 root     root            0 Jan 31 20:37 modules
dr-xr-xr-x   2 root     root            0 Jan 31 20:37 net
dr-xr-xr-x   4 root     root            0 Jan 31 20:37 self
-r--r--r--   1 root     root            0 Jan 31 20:37 stat
-r--r--r--   1 root     root            0 Jan 31 20:37 uptime
-r--r--r--   1 root     root            0 Jan 31 20:37 version
$
```

(하지만, 프로세스완 관련이 없는 약간의 파일들이 있을 것이다. 위 예는 실제보이는 것을 편집한 것이다.)

파일시스템이지만 proc파일시스템의 어느 것도 디스크를 건드리지 않는다는 것을 유의해라. proc파일시스템은 오로지 커널의 상상속에서만 존재한다. 누군가가 proc 파일시스템의 어떤 부분을 보려고 한다면, 커널은 실제로 존재하지는 않지만, 마치 어딘가에 존재하는 것처럼 보이게 한다. /proc/kcore 파일이 있을지라도, 디스크 공간을 차지하지는 않는다.

어떤 파일시스템을 사용할 것인가?
보통 많은 다른 파일시스템을 사용하는데는 조그만 이유가 있을 것이다. 현재는 ext2fs가 가장 유명한 파일시스템이고, ext2fs가 가장 현명한 선택일 것이다. 파일구조를 기록하기 위한 부하, 속도, (파악된) 안정성, 호환성과 여러가지 다른 이유에 의해서, 다른 파일시스템을 사용하는 것도 추천할만 할지도 모른다. 파일시스템을 고르는 것은 각각의 경우에 따라 결정될 필요가 있다.

파일시스템 만들기
파일시스템은 mkfs 명령으로 만들어진다. 즉 초기화되는 것이다. 실제로 각 파일시스템마다 다른 프로그램이 있다. mkfs는 단지 원하는 파일시스템의 형식에 따라 적절한 프로그램을 돌리는 전위 프로그램이다. 파일시스템 형식은 -t fstype 옵션으로 선택되어진다.

mkfs라 불리는 프로그램들은 약간 다른 명령어 인터페이스를 가진다. 일반적이고 가장 중요한 옵션들은 아래에 요약되어 있다. 더 자세한 것은 메뉴얼 페이지를 보아라.

-t fstype
파일시스템의 형식을 선택한다.

-c
배드블럭을 조사하고 조사한 결과에 따라 배드블럭 리스트를 초기화한다.

-l filename
filename이라는 파일로부터 초기의 배드블럭리스트를 읽어들인다.

ext2파일시스템을 플로피에 만들기 위해, 다음과 같은 명령을 내릴 것이다. $ fdformat -n /dev/fd0H1440
Double-sided, 80 tracks, 18 sec/track. Total capacity 1440 kB.
Formatting ... done
$ badblocks /dev/fd0H1440 1440 $>$ bad-blocks
$ mkfs -t ext2 -l bad-blocks /dev/fd0H1440
mke2fs 0.5a, 5-Apr-94 for EXT2 FS 0.5, 94/03/10

```
360 inodes, 1440 blocks
72 blocks (5.00%) reserved for the super user
First data block=1
Block size=1024 (log=0)
Fragment size=1024 (log=0)
1 block group
8192 blocks per group, 8192 fragments per group
360 inodes per group

Writing inode tables: done
Writing superblocks and filesystem accounting information: done
$
```

먼저, 플로피가 포맷된다. (-n 옵션을 확인, 즉 배드블럭 조사를 막는다.). 그리고 bad-blocks이라는 파일로 결과를 리다이렉트하면서 배드블럭이 badblocks로 조사된다. 마지막으로 badblocks 명령이 찾아내어 초기화시킨 배드블럭리스트를 이용해 파일시스템이 만들어진다.

badblocks와 배드블럭리스트 대신에 -c 옵션이 mkfs와 함께 사용될 수도 있을 것이다. 예는 아래와 같다. $ mkfs -t ext2 -c /dev/fd0H1440

```
mke2fs 0.5a, 5-Apr-94 for EXT2 FS 0.5, 94/03/10
360 inodes, 1440 blocks
72 blocks (5.00%) reserved for the super user
First data block=1
Block size=1024 (log=0)
Fragment size=1024 (log=0)
1 block group
8192 blocks per group, 8192 fragments per group
360 inodes per group

Checking for bad blocks (read-only test): done
Writing inode tables: done
Writing superblocks and filesystem accounting information: done
$
```

badblocks를 따로 사용하는 것보다 -c가 더 편리하지만, badblocks는 파일시스템이 만들어진 후 배드블럭을 체크하기 위해 필요하다.

포맷하는 것이 불필요한 것을 제외하고, 하드디스크나 파티션에 파일시스템을 만드는 과정은 플로피와 같다.

마운트하기와 마운트 풀기

파일시스템을 사용하기 전에, 마운트되어야 한다. 그리고나서, 운영체제는 모든 것이 잘 작동하는지 확실히 하기 위해 여러가지 기록하는 작업을 한다. 유닉스안의 모든 파일들은 단일 디렉토리 트리안에 있으므로, 마운트 작업은 새로운 파일시스템의 내용이 이미 어딘가에 마운트된 파일시스템의 존재하는 하위디렉토리의 내용으로 보이게 할 것이다.

예를 들어, Figure 4-3은 각각 고유의 루트 디렉토리를 지니는 세개의 다른 파일시스템을 보여준다. 마지막 두 파일시스템이 첫째 파일시스템의 /home과 /usr에 각각 마운트되었을 때, Figure 4-4처럼 단일 디렉토리 트리를 얻을 수 있다.

Figure 4-3. 각각 분리된 세개의 파일시스템.

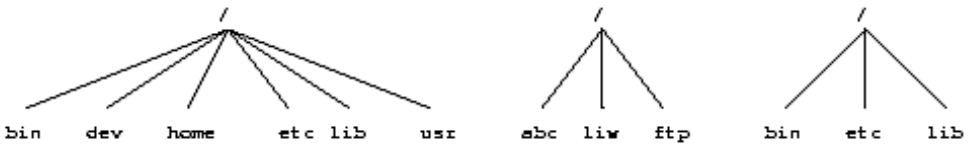
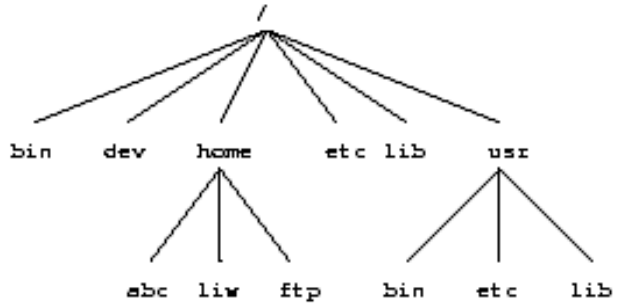


Figure 4-4. 마운트된 /home과 /usr.



마운트는 다음과 같이 행해질 수 있다. $ mount /dev/hda2 /home
$ mount /dev/hda3 /usr
$

mount 명령은 2개의 인수를 취한다. 첫번째 인수는 파일시스템을 포함하고 있는 디스크나 파티션에 해당되는 장치파일이다. 두번째 인수는 마운트될 디렉토리이다. 위 명령 후, 두 파일시스템의 내용은 각각 /home과 /usr 디렉토리 내용으로 보인다. ``/dev/hda2가 /home에 마운트된다''라고 말할 수 있을 것이고, /usr의 경우도 비슷하다. 어느 파일시스템을 보기 위해선, 어떤 다른 디렉토리인양, 파일시스템이 마운트되어있는 디렉토리의 내용을 보면 될 것이다. 장치파일 /dev/hda2와 마운트한 디렉토리 /home의 차이점을 유의해라. 장치파일은 디스크의 원시 내용을 접근하게 하고, 마운트한 디렉토리는 디스크의 파일에 접근하게 한다. 마운트한 디렉토리를 mount point라 한다.

리눅스는 많은 파일시스템 형식을 지원한다. mount는 파일시스템의 형식을 추측하려고 할 것이다. 형식을 바로 지정하기 위해 -t fstype 옵션을 사용할 수 있다. -t fstype은 때때로 필요하다. mount가 사용하는 추론이 항상 동작하는 것은 아니기 때문이다. 예를 들어, MS-DOS플로피를 마운트하기 위해, 다음 명령을 내릴 수 있다. $ mount -t msdos /dev/fd0 /floppy
$

마운트할 디렉토리는 반드시 존재해야 하지만 비어있을 필요는 없다. 그러나, 그 안에 있는 어떤 파일이라도 파일시스템이 마운트되어 있는 동안은 이름으로는 접근할 수 없을 것이다.(이미 열려있던 어떤 파일들은 여전히 접근 가능할 것이다. 다른 디렉토리에 하드링크되어 있는 파일들은 그 이름을 가지고 접근할 수 있을 것이다.) 그렇게 한다고해서 해가 되지 않고, 심지어 필요할 수도 있다. 예를 들어, 어떤 사람들은 /tmp와 /var/tmp를 같게 사용하는 것을 좋아해서, /tmp를 /var/tmp로 심볼릭링크시킨다. 시스템이 부팅될 때, /usr 파일시스템이 마운트되기 전, 루트 파일시스템에 들어있는 /var/tmp 디렉토리가 대신 사용된다. /usr이 마운트되었을 때, 루트 파일시스템에 있는 /var/tmp 디렉토리는 접근불가능이 될 것이다. 만약 /var/tmp가 루트파일시스템에 존재하지 않는다면 /var을 마운트하기 전에는 임시파일들은 사용하는 것이 불가능할 것이다.

만약 파일시스템에 어떤 것도 기록할 생각이 없다면, 읽기전용 마운트를 하기 위해 mount에 -r 스위치를 사용해라. 읽기전용 마운트는 커널이 파일시스템에 기록하려고 하는 어떤 시도도 중지하도록 할 것이고, 커널이 inode안에 있는 파일 접근 시간을 갱신하는 것도 방해할 것이다. 읽기전용 마운트는 쓸 수 없는 미디어, 예를 들어 시디롬에 필요하다.

기민한 독자들은 벌써 약간의 논리적인 문제가 있다는 것을 눈치챘다. 분명 다른 파일시스템에 마운트될 수 없는데, 첫번째 파일시스템(루트 디렉토리를 포함하기 때문에, root 파일시스템이라 불린다.)은 어떤게 마운트되는가? 글쎄 답은 마술에 의해 이루어진다이다. [1] 루트 파일시스템은 마술같이 부트타임에 마운트되고, 루트 파일시스템이 항상 마운트될 것이라고 믿을 수 있다. 루트 파일시스템이 마운트될 수 없다면, 시스템은 부팅되지 않는다. 루트로 마술처럼 마운트되는 파일시스템의 이름은 커널에 컴파일되어 들어가거나, LILO나 rdev를 이용해서 지정한다.

보통 루트 파일시스템은 처음에 읽기만 되도록 마운트된다. 그리고나서,시작 스크립트는 루트 파일시스템의 타당성을 검증하기 위해 fsck를 실행할 것이고, 만약 문제가 없다면, 시작스크립트는 루트 파일시스템을 쓰기가 허용되도록 루트 파일시스템을 다시 마운트할 것이다. fsck는 마운트된 파일시스템에서는 행해지면 안된다. fsck가 돌아가는 동안에 파일시

스템에 어떤 변화가 있으면 문제를 일으킬 것이기 때문이다. 루트 파일시스템이 체크되는 동안에 루트파일시스템은 읽기 전용으로 마운트되어 있기 때문에, fsck는 걱정없이 어떤 문제라도 고칠 수 있다. 다시 마운트하는 작업은 파일시스템이 메모리에 저장했던 어떤 중간에 생긴 데이타라도 방출해 버릴 것이다.

많은 시스템에는 부팅시간에 자동으로 마운트되어야할 다른 파일시스템이 있다. 그런 파일시스템들은 /etc/fstab 파일에 명시되어 있다. 형식에 대한 자세한 것을 위해서는 fstab메뉴얼페이지를 봐라. 여분의 파일시스템이 마운트될 때 정확한 세부사항들은 많은 인수에 의존하고, 필요하다면 각 관리자에 의해 설정될 수 있다. 이에 대한 자세한 내용은 Chapter 6 을 보기 바란다.

파일시스템이 더 이상 마운트될 필요가 없을 때, umount라는 명령으로 마운트를 풀 수 있다. [2] umount는 한개의 인수를 취한다. 장치파일이나 마운트된 곳이다. 예를 들어 전 예에서 마운트한 디렉토리들의 마운트를 풀고 싶다면, 다음 명령을 사용할 수 있다. $ umount /dev/hda2
$ umount /usr
$

명령을 어떻게 사용하지는 더 많은 지시들을 원하면 메뉴얼페이지를 봐라. 항상 마운트된 플로피의 마운트를 풀어야하는 것은 꼭 해야할 일이다. 드라이브에서 플로피를 그냥 꺼내지 마라! 디스크 캐쉬때문에 플로피를 마운트 풀기 전까지 데이타가 플로피에 기록될 필요는 없어서, 드라이브에서 플로피를 너무 빨리 제거하는 것은 플로피 내용이 왜곡되게 할지도 모른다. 만약 플로피에서 읽기만 했다면, 그렇지 않겠지만, 만약 기록했다면, 우연일지라도, 결과는 재앙일지도 모른다.

마운트하기와 마운트 풀기는 슈퍼유저 권한을 필요로 한다. 즉 오로지 root만 할 수 있다. 만약 어떤 유저가 플로피를 어떤 디렉토리에 마운트할 수 있다면, /bin/sh이나 어떤 때때로 사용되는 다른 프로그램으로 위장된 트로이의 목마를 넣어 플로피를 만드는 것이 다소 쉬워지기 때문이다. 하지만 때때로 사용자들에게 플로피를 사용하도록 허가하는 것이 필요하고, 몇가지 방법이 있다.

사용자들에게 루트패스워드를 알려준다. 분명 보안상 나쁘지만, 가장 쉬운 방법이다. 어쨌든 보안이 필요없다면 잘 작동할 것이고, 많은 네트웍에 연결이 안되어 있는 개인 시스템들의 경우이다.

사용자들이 마운트를 할 수 있도록 sudo같은 프로그램을 사용한다. 역시 보안상 나쁘지만, 슈퍼유저 권한을 모든 사람들에게 직접 주지 않는다. [3]

사용자들에게 mtools를 사용하게 한다. 플로피를 마운트하지 않고 MS-DOS파일시스템을 다루는 패키지이다. 만약 MS-DOS플로피가 필요한 모든 것이라면 잘 작동하지만, 그렇지 않다면, 다소 곤란하다.

/etc/fstab 안에 적당한 옵션과 함께 플로피 장치와 허용가능한 마운트 지점을 함께 적어둔다.

마지막 대안은 /etc/fstab 파일에 다음과 같은 줄을 추가해서 적용할 수 있다. /dev/fd0 /floppy msdos user,noauto 0 0

각 열들은 이렇다. 마운트할 장치파일, 마운트할 디렉토리, 파일시스템 형식, 옵션들, 백업 주기(dump에 의해 사용된다), fsck에 넘겨주는 값(어떤 파일시스템들이 부팅시 체크되는가 명시하기 위해. 0은 체크를 안하는 것을 뜻한다)이다.

noauto 옵션은 시스템이 시작할 때 마운트가 자동으로 되는 것을 막는다(즉, mount -a로 마운트하려고 하는 것을 막는다.). user 옵션은 어떤 사용자라도 파일시스템을 마운트하게 하지만, 보안 때문에, 프로그램(보통 프로그램이나 setuid된 프로그램)의 실행과 마운트된 파일시스템에서 장치파일들을 해석하는 것을 막는다. 위와같이 하고나면, 어떤 사용자라도 다음 명령으로 msdos파일시스템을 가지고 있는 플로피를 마운트 할 수 있다. $ mount /floppy
$

플로피는 대응되는 umount 명령으로 마운트를 풀수 있다(물론 마운트를 풀 필요가 있다.).

만약 몇가지 형식의 플로피에 접근을 제공하길 원한다면, 몇개의 마운트 지점을 줄 필요가 있다. 설정은 각 마운트 지점

마다 다를 수 있다. 예를 들어, MS-DOS와 ext2 플로피 모두에 접근하게 하려고 한다면, /etc/fstab에 다음과 같은 줄을 첨가할 수 있다. /dev/fd0 /dosfloppy msdos user,noauto 0 0
/dev/fd0 /ext2floppy ext2 user,noauto 0 0

MS-DOS 파일시스템 때문에(단지 플로피가 아니라), 아마도 uid, gid, umask 파일시스템 옵션들을 이용해서 MS-DOS 파일시스템에 접근을 제한하기를 원할 수 있다. 자세한 것은 mount 매뉴얼페이지에 설명된다. 조심하지 않는다면, MS-DOS 파일시스템을 마운트하는 것은 모든 사람들이 그 안에 있는 파일들을 적어도 읽을 수 있도록 하는데, 좋은 생각이 아니다.

fsck로 파일시스템 완전성(integrity) 체크하기
파일시스템은 복잡한 창조물이고, 창조물이 그렇듯이, 어딘지 문제를 일으키는 경향이 있다. 파일시스템의 정확성과 타당성은 fsck를 통해 체크될 수 있다. fsck가 발견하는 어떤 작은 문제들을 해결하고 , 수리할 수 없는 어떤 문제가 있으면 사용자에게 경고하기 위해 명령을 내릴 수 있다. 다행히도, 파일시스템을 이루는 코드는 다소 효율적으로 디버깅되어서, 좀처럼 어떤 문제도 없고, 전원이 꺼진다던가, 하드웨어가 잘못되었던가, 운영자가 실수했다던가 하는 이유로 문제가 발생한다. 예를 들어 시스템을 적절히 종료시키지 않으면 문제가 발생한다.

대부분의 시스템들은 fsck를 부팅할 때 자동적으로 실행하도록 설정되어, 시스템이 사용되기 전에 어떤 에러라도 발견된다(그리고 다행히도 고쳐진다.). 망가진 파일시스템을 사용하는 것은 일을 더 나쁘게 만드는 경향이 있다. 만약 자료구조가 한번 뒤집히면, 파일시스템을 사용하는 것은 아마도 더 많은 자료 손실을 일으키며, 파일시스템을 더욱더 뒤집어지게 만들 것이다. 그러나 fsck는 큰 파일시스템에서 돌아가는데 약간 시간이 걸릴 수 있으며, 만약 시스템이 적절히 종료되었다면 문제는 거의 절대 일어나지 않기 때문에, 다음과 같은 경우에 체크를 피하기 위해 몇가지 트릭이 사용된다. 첫째로 /etc/fastboot라는 파일이 있다면, 체크를 하지 않는다. 둘째로 ext2파일시스템은 파일시스템의 슈퍼블럭안에 파일시스템이 이전 마운트 후에 적절히 마운트를 풀었는지 알려주는 특별한 표시를 가지고 있다. 만약 표시가 마운트가 풀어졌음을 가리킨다면(적절하게 마운트를 푼다는 것은 문제가 없음을 가리킨다라고 가정), 이 표시는 e2fsck(ext2 파일시스템을 위한 fsck버전)가 파일시스템을 점검하는 것을 피하게 한다. /etc/fastboot 방법이 시스템에서 작동하는지 않하는지는 시작 스크립트에 달려있지만, ext2방법은 e2fsck를 사용하는 모든 경우에 작동한다. 피하려면 e2fsck를 옵션을 주어 명백하게 통과해야 한다.(어떻게 하는지 자세한 것을 원하면 e2fsck 매뉴얼페이지를 봐라.)

자동 체크는 부팅시에 자동으로 마운트되는 파일시스템에서만 작동한다. 다른 파일시스템들, 예를 들어 플로피를 체크하려면 fsck를 수동으로 사용해라.

만약 fsck가 복구할 수 없는 문제를 발견하면, 파일시스템이 일반적으로 동작하는 방법과 특히 망가진 파일시스템의 형식에 대한 깊은 지식이 필요하거나, 백업을 잘 하는 것이 필요하다. 후자는 해결하기 쉽고(비록 때때로 지겹지만), 전자는 만약 당신 자신이 하는 방법을 모른다면, 때때로 친구, 리눅스 뉴스그룹, 메일링리스트나 다른 지원책을 통해 해결될 수 있다. 더 말해주길 원하지만, 교육과 경험의 부족으로 힘들다. Theodore T'so가 만든 debugfs 프로그램이 유용할 것이다.

fsck는 마운트가 안된 파일시스템에서만 행해져야 하고, 마운트된 파일시스템에서는 해서는 안된다(시작시 읽기전용으로 마운트된 root를 제외하고). fsck가 원시디스크를 건드려서, 운영체제의 인지없이 파일시스템을 수정할 수 있기 때문이다. 만약 운영체제가 혼동한다면 문제가 있을 것이다.

badblocks로 디스크 에러를 검사하기
주기적으로 배드블럭을 검사하는 것은 좋은 생각일 수 있다. badblocks 명령으로 행해진다. badblocks는 찾아낼 수 있는 모든 배드블럭의 번호 리스트를 결과로 내놓는다. 배드블럭리스트는 파일시스템 데이타 구조안에 저장되기 위해 fsck로 입력될 수 있어서 운영체제는 데이타를 저장하기 위해 배드블럭을 사용하려고 하지 않을 것이다. 다음 예는 어떻게 행해지는지 보여줄 것이다. $ badblocks /dev/fd0H1440 1440 > bad-blocks
$ fsck -t ext2 -l bad-blocks /dev/fd0H1440
Parallelizing fsck version 0.5a (5-Apr-94)
e2fsck 0.5a, 5-Apr-94 for EXT2 FS 0.5, 94/03/10
Pass 1: Checking inodes, blocks, and sizes
Pass 2: Checking directory structure
Pass 3: Checking directory connectivity
Pass 4: Check reference counts.

```
Pass 5: Checking group summary information.

/dev/fd0H1440: ***** FILE SYSTEM WAS MODIFIED *****
/dev/fd0H1440: 11/360 files, 63/1440 blocks
$
```

만약 badblocks가 이미 사용되고 있는 블럭을 보고한다면, e2fsck는 그 블럭을 다른 곳으로 옮기려고 할 것이다. 만약 그 블럭이 단지 부분적이 아니라 정말 망가졌다면, 파일의 내용들은 아마 망가질 것이다.

디스크가 조각나는 것과 싸우기
디스크에 한 파일이 쓰여질 때, 파일이 항상 연속되는 블럭에 쓰여질 수는 없다. 연속적인 블럭에 저장되지 않은 파일은 조각난(fragmented) 것이다. 조각난 파일을 읽는 것은 약간 시간이 더 걸린다. 디스크의 읽기쓰기 헤드가 더 많이 움직여야 할 것이기 때문이다. 미리 읽기 기능을 가진 좋은 버퍼캐쉬를 지닌 시스템안에서는 문제가 작아지지만, 조각나는 것을 피하는것이 바람직하다.

블럭들이 연속되는 섹터안에 저장되지 못할지라도, 파일안의 모든 블럭이 같이 가까이 있도록 하면서, ext2파일시스템은 조각나는 것을 최소로 유지하려고 시도할 것이다. ext2는 효율적으로 항상 파일의 다른 블럭에 가장 가까운 여분의 블럭들 할당할 것이다. 그래서 ext2를 위해선 좀처럼 조각나는 것에 대해 걱정할 필요가 없다. ext2파일시스템 조각모으기를 위한 프로그램이 있기는 하다.

조각난 것을 제거하기 위해 블럭들을 파일시스템 둘레로 옮기는 많은 MS-DOS 조각모으기 프로그램들이 있다. 다른 파일시스템을 위해서는 조각모으기는 파일시스템을 백업하고, 다시 만들고, 백업한 것에서 파일들을 다시 저장하는 과정을 통해 이루어져야 한다. 조각모으기 전에 파일시스템을 백업하는 것은 모든 파일시스템에 좋은 생각이다. 조각모으기를 하는 동안 많은 것들이 잘못될 수 있기 때문이다.

모든 파일시스템들을 위한 다른 도구들
약간의 다른 도구들 역시 파일시스템들을 다루는데 쓸모있다. df는 하나 혹은 더 많은 파일시스템들의 여분의 디스크공간을 보여준다. du는 얼마나 많은 디스크공간이 디렉토리와 디렉토리안의 파일들이 포함하고 있는가를 보여준다. 이런 것들은 디스크공간을 낭비하는 것들을 잡아낼 때 사용할 수 있다.

sync는 버퍼캐쉬(the section called 버퍼 캐쉬 in Chapter 5을 보라.) 안의 모든 기록되지 않은 블럭들이 디스크에 기록되도록 한다. 수동으로 하는 것은 좀처럼 필요치 않다. 데몬 작업인 update가 자동으로 해준다. 큰 문제가 있을 경우, 예를 들어 update나 update를 도와주는 작업인 bdflush가 죽었다거나, 전원을 당장 꺼야 하는데 update가 돌아갈 시간까지 기다릴 수 없다면, 쓸모 있을 것이다.

ext2파일시스템을 위한 다른 도구들
직접적 혹은 파일시스템 형식에 독립적인 전위 프로그램을 통해서 접근할 수 있는 파일시스템 만드는 도구(mke2fs)와 파일시스템을 검사하는 도구(e2fsck) 외에도 ext2파일시스템은 사용할 수 있는 약간의 추가되는 도구를 가지고 있다.

tune2fs는 파일시스템 매개변수를 조절한다. 재미있는 매개변수들 중 일부는 다음과 같다.

최대 마운트 수. e2fsck는 슈퍼블럭에 있는 표시가 깨끗하더라도(역자 주: 즉 지난 번 마운트한 후 마운트가 적절히 풀어졌더라도) 파일시스템이 너무 많이 마운트되었으면 검사하도록 한다. 개발이나 시스템을 테스트하기 위해 사용되는 시스템이라면, 이 제한을 줄이는 것이 좋을 지도 모른다.

검사 사이의 최대 시간. e2fsck는 표시가 깨끗하고, 파일시스템이 매우 가끔씩 마운트되지 않는다면, 두번의 검사사이의 최대 시간을 요구한다. 하지만 못하게 할 수도 있다.

root를 위해 남겨둔 블럭 수. 파일시스템이 다 찬다면 어떤 것들을 지울 필요없이 시스템관리가 여전히 가능하게 하기 위해 ext2는 루트를 위해 약간의 블럭을 남겨둔다. 남겨둔 양은 기본적으로 5%인데, 대부분의 디스크에서 낭비하기에 충분

하지 않다. 그러나, 플로피에는 남겨둘 블럭이 아주 조금도 없다.

더 많은 정보를 위해선 tune2fs 메뉴얼페이지를 보라.

dumpe2fs는 대개 슈퍼블럭으로부터, ext2파일시스템에 대한 정보를 보여준다. Figure 4-5는 한가지 실례이다. 실행 결과 안의 어떤 정보는 기술적이고 파일시스템이 어떻게 작동하는지에 대한 이해가 필요하지만, 많은 양이 쉽게 이해할 수 있다.

Figure 4-5. dumpe2fs가 보여주는 출력의 한 예

```
dumpe2fs 0.5b, 11-Mar-95 for EXT2 FS 0.5a, 94/10/23
Filesystem magic number:  0xEF53
Filesystem state:         clean
Errors behavior:          Continue
Inode count:              360
Block count:              1440
Reserved block count:     72
Free blocks:              1133
Free inodes:              326
First block:              1
Block size:               1024
Fragment size:            1024
Blocks per group:         8192
Fragments per group:      8192
Inodes per group:         360
Last mount time:          Tue Aug  8 01:52:52 1995
Last write time:          Tue Aug  8 01:53:28 1995
Mount count:              3
Maximum mount count:      20
Last checked:             Tue Aug  8 01:06:31 1995
Check interval:           0
Reserved blocks uid:      0 (user root)
Reserved blocks gid:      0 (group root)

Group 0:
  Block bitmap at 3, Inode bitmap at 4, Inode table at 5
  1133 free blocks, 326 free inodes, 2 directories
  Free blocks: 307-1439
  Free inodes: 35-360
```

debugfs는 파일시스템 디버거이다. 디스크에 저장된 파일시스템 데이타구조에 직접 접근하는 것을 허용해서 너무 깨져서 fsck가 자동으로 수리할 수 없는 디스크를 수리하는데 사용될 수 있다. 지워진 파일들을 복구하는데에도 사용되는 것으로도 알려져 있다. 그러나, debugfs는 하는 작업을 이해할 것을 너무 많이 요구한다. 이해하지 못하는 것은 모든 데이타를 파괴할 수 있다.

dump와 restore는 ext2파일시스템을 백업하는데 사용될 수 있다. dump와 restore는 전통적인 UNIX 백업툴들의 ext2 특유의 버전들이다. 백업에 대해 더 많은 정보를 원하면 Chapter 10를 보기 바란다.

Notes

[1] 더 많은 정보를 원하면 커널소스나 `Kernel Hackers Guide'를 봐라.

[2] umount는 물론 unmount이어야겠지만, 이상하게도 70년대에 n이 사라졌고, 그 이후로 보이지 않았다. 만약 찾는다면 New Jersey의 벨 연구소로 돌려주기 바란다.

[3] 사람들의 행동에 대해 고심할 몇 초가 필요하다.

파일시스템 없는 디스크
모든 디스크나 파티션이 파일시스템으로 사용되는 것은 아니다. 예를 들어 스왑 파티션은 파일시스템을 가지지 않을 것이다. 많은 플로피들이 테이프드라이브를 에뮬레이트하는 형식으로 사용되기에, tar나 다른 파일들을 파일시스템이 없이 원시디스크에 직접적으로 쓰는 것이 가능하다. 리눅스 부트 플로피는 파일시스템을 포함하지 않고 오로지 커널만이 있다.

파일시스템은 항상 파일정보기록을 위해 낭비를 하기 때문에, 파일시스템을 피하는 것은 더 많은 디스크를 사용가능하게 하는 장점이 있다. 그리고 디스크가 다른 시스템과 더 쉽게 호환할수 있게 하기도 한다. 예를 들어, 파일시스템은 대부분의 시스템에서 다르지만 tar 파일 형식은 모든 시스템에서 같다. 필요하다면 바로 파일시스템이 없이 디스크를 사용할 수 있을 것이다. 부팅가능한 리눅스 플로피 역시 파일시스템을 가지는 것도 가능하지만 파일시스템이 필요한 것은 아니다.

원시디스크를 사용하는 한 이유는 디스크의 이미지 복사본을 만들기 위해서이다. 예를 들어, 디스크가 부분적으로 파손된 파일시스템을 포함하고 있다면, 고칠려고 하기 전에 정확한 디스크의 복사본을 만드는 것이 좋다. 수리작업이 더 디스크를 망가뜨린다면 다시 시작할 수 있기 때문이다. 디스크 수리를 하기 위해 디스크 이미지를 복사하는 한 방법은 dd를 사용하는 것이다. $ dd if=/dev/fd0H1440 of=floppy-image
2880+0 records in
2880+0 records out
$ dd if=floppy-image of=/dev/fd0H1440
2880+0 records in
2880+0 records out
$

첫 dd 명령은 플로피의 정확한 이미지를 floppy-image라는 파일로 만들고, 두번째 dd 명령은 이미지를 플로피에 기록한다. (아마도 두번째 명령을 내리기전에 플로피를 바꿨을 것이다. 그렇지 않으면 위 두 명령은 의심스러운 사용법이다.)

디스크 공간 할당하기
디스크 분할 계획
디스크를 가능한 가장 좋은 방법으로 분할한다는 것은 쉽지 않다. 나쁘게도, 디스크를 분할하는 일반적인 정확한 방법이 없다. 고려해야할 요인들이 너무 많다.

전통적인 방법은, /bin, /etc, /dev, /lib, /tmp와 시스템이 돌아가는데 필요한 다른 것들을 포함하는 (상대적으로) 작은 루트 파일시스템을 만드는 것이다. 이 방법으로, 루트 파일시스템(고유한 파티션이나 디스크에 있는)은 시스템을 작동시키는데 필요한 모든 것이 된다. 작은 루트 파일시스템을 만드는 이유는 만약 루트파일시스템이 작고 많이 사용되지 않으면, 시스템이 망가졌을 때, 파일시스템이 망가질 가능성이 적고, 그래서 시스템이 망가져서 생긴 문제들을 고치는 것이 쉽다는 것을 알 수 있을 것이다. 그리고, /usr, 사용자 홈 디렉토리(종종 /home)와 스왑 공간을 위해 다른 파티션을 만들던지 아니면 다른 디스크를 사용한다. 프로그램들(/usr밑에 있는)을 백업하는 건 보통 필요없기 때문에, 홈디렉토리(사용자의 파일들과 함께)를 홈디렉토리만의 파티션으로 떼어 놓는 것은 백업을 더 쉽게 한다. 네트웍 환경에서는 컴퓨터 대수만큼 수십에서 수백메가바이트씩에 달하는 양을 줄이면서, /usr을 몇대의 컴퓨터들이 공유하는 것도 가능하다(예를 들어 NFS를 이용해서).

많은 파티션을 가져서 생기는 문제는 안쓰고 있는 디스크 총량을 많은 조각으로 나눈다는 점이다. 디스크와 (희망차게도) 운영체제가 더 신뢰할 수 있는 요즈음, 많은 사람들은 모든 파일들을 포함하는 오로지 한 파티션을 선호한다. 반면에, 작은 파티션을 백업하는(그리고 복구하는)것이 고통이 덜할지도 모른다.

작은 하드디스크(커널 개발을 하지 않는다고 가정하고)는 아마도 한 파티션을 가지는 것이 가장 좋은 방법일 것이다. 큰 하드디스크는 단지 뭔가 정말 잘못될 경우, 상당수의 큰 파티션으로 나누는 것이 아마도 좋을 것이다.(작고 크다는 것이 여기에서 상대적인 감각으로 사용된다는 것에 유의하라. 디스크 공간의 필요성이 어떻게 반응할 지 결정한다.)

만약 여러개의 하드디스크를 가지고 있다면, 루트 파일시스템(/usr을 포함하는)을 한 디스크에 넣고, 홈디렉토리를 다른 디스크에 넣기를 원할지도 모른다.

다른 디스크 할당 계획을 가지고 약간 실험할 준비를 하는 것은 좋은 생각이다(첫번째 인스톨할 동안만이 아니라 시간에 걸쳐). 그렇게 하는 것은 필수적으로 시스템을 몇차례 인스톨하는 것을 필요로 하기 때문에 다소 일이 될 것이나, 디스크 분할을 정확히 했다고 확신할 수 있는 유일한 방법이다.

공간 요구량
설치하려는 리눅스 배포본에서 여러 설정에 따라 얼마나 많은 디스크량이 필요한지 약간의 지시가 있을 것이다. 따로 설치한 프로그램도 역시 같은 지시가 있을 것이다. 그런 지시들이 디스크 사용을 계획하는데 도움을 줄 것이나, 미래를 준비해야 하고 나중에 필요하다고 느낄 것들을 위해 약간의 여분의 공간을 확보해야 한다.

사용자 파일에 필요한 양은 사용자가 뭘 하길 바라는지에 의존한다. 대부분의 사람들이 가능한한 그들의 파일을 위해 많은 공간을 필요로 하는 것처럼 보이긴 하지만, 사용자들이 행복하게 살아가기 위해 필요한 공간은 저마다 다양하다. 어떤 사람들은 오로지 간단한 텍스트 작업을 하고 몇 메가바이트로 잘 살아갈 것이고, 다른 사람들은 힘든 이미지 처리를 하고 몇 기가바이트가 필요할 것이다.

그런데, 킬로바이트나 메가바이트로 주어지는 파일크기나 메가바이트로 주어지는 디스크 공간을 비교할 때, 두 단위가 다르게 사용될 수 있다는 것을 아는 것이 중요하다. 어떤 디스크 제조업체는 1 킬로바이트가 1000바이트고 1메가바이트는 1000킬로바이트인 척하는 것을 좋아한다. 나머지 모든 컴퓨터세계에서는 모든 단위에 1024를 사용하지만 말이다. 그래서 345MB 하드디스크는 실제적으로 330MB 하드디스크이다. [1]

스왑디스크 할당은 the section called 스왑 공간 할당하기 in Chapter 5에서 설명한다.

하드디스크 할당의 예
난 109MB하드디스크를 가진 적이 있다. 지금 난 330MB하드디스크를 사용하고 있다. 이 디스크들을 어떻게 왜 분할했는지 설명할 것이다.

내 필요와 내가 사용하는 운영체제가 바뀔 때, 많은 방법들로 109MB디스크를 분할했다. 2개의 전형적인 시나리오를 설명할 것이다. 첫번째, 리눅스와 함께 MS-DOS를 돌렸다. 도스를 위해 약 20MB의 하드디스크, 다시 말하면 MS-DOS, C 컴파일러, 편집기, 약간의 다른 유틸리티, 내가 작업하는 중이던 프로그램을 갖기에 충분할 정도인 양과, 밀실공포증을 느끼지 않을 충분한 디스크 여유 공간를 필요로 했다. 리눅스를 위해 10MB의 스왑파티션을 가졌고, 나머지, 즉 79MB는 리눅스에 있는 모든 파일들을 담는 단일 파티션이었다. 루트파티션, /usr, /home을 분리한 적도 있으나, 흥미있는 일을 할 파티션에서 결코 충분한 여분의 디스크가 없었다.

더 이상 MS-DOS를 사용할 필요가 없어졌을 때, 디스크를 다시 분할해서 12MB의 스왑파티션과 나머지를 다시 단일 파일시스템으로 했다.

다음과 같이 330MB는 몇개의 파티션으로 나눴다.

5 MB root 파일시스템
10 MB 스왑 파티션
180 MB /usr 파일시스템
120 MB /home 파일시스템
15 MB 막 쓰는 파티션

막 쓰는 파티션은 고유한 파티션을 요구하는, 예를 들어 다른 리눅스 배포본을 시도하기나 파일시스템의 속도를 비교하기 같은 일을 수행하기 위해 있다. 다른 것에 필요로 하지 않을 때는 스왑공간으로 사용했다(난 윈도우를 많이 열어놓는 것

을 좋아한다).

리눅스에 디스크공간을 더 추가하기
적어도 하드웨어가 적절히 설치된 후라면(하드웨어 설치는 이 책의 범위에서 벗어난다), 리눅스에 디스크공간을 더 추가하는 것은 쉽다. 필요하다면 포맷하고, 위에서 설명한 대로 파티션과 파일시스템을 만들고, /etc/fstab에 적절하게 줄을 추가시켜 줘서 파티션이 자동적으로 마운트되게 한다.

디스크 공간을 절약하기 위한 팁
디스크 공간을 절약하는 가장 좋은 팁은 필요없는 프로그램들을 설치하는 것을 피하는 것이다. 대부분의 리눅스 배포본은 배포본들이 포함하고 있는 패키지들의 일부만을 설치할 옵션을 지니고, 필요를 분석하면 패키지들중 대부분이 필요없다는 것을 알아낼 지도 모른다. 많은 프로그램들이 다소 크기 때문에, 불필요한 패키지를 설치하지 않는 것은 많은 디스크 공간을 절약하는데 도움을 줄 것이다. 특정한 패키지나 프로그램이 필요할 지라도, 그 패키지나 프로그램 모두가 필요하지는 않을 것이다. 예를 들어, 어떤 온라인 문서, GNU Emacs의 Elisp파일들의 일부, X11 폰트의 일부, 프로그래밍을 위한 라이브러리의 일부는 불필요할 것이다.

만약 패키지를 삭제할 수 없다면, 압축을 알아볼 수도 있을 것이다. gzip이나 zip같은 압축 프로그램들은 각각의 파일들이나 파일의 묶음들을 압축(그리고 압축풀기)을 할 것이다. gzexe 시스템은 사용자에게 보이지 않게 프로그램들을 압축하고 압축을 풀 것이다(사용되지 않는 프로그램은 압축되고, 프로그램이 사용되면 압축을 푼다). 실험적인 Double 시스템은 파일시스템에 있는 그 파일들을 사용하는 프로그램 모르게 모든 파일들을 압축할 것이다. (MS-DOS에 있는 Stacker같은 제품에 친숙하다면 원리는 같다.)

Notes
[1] Sic transit discus mundi.. 세상의 디스크들은 이렇게 물건너갔다..

Chapter 5. 메모리 관리
Table of Contents
가상 메모리란?
스왑 공간 생성하기
스왑 공간 사용하기
다른 운영체제와 스왑 공간을 공유하기
스왑 공간 할당하기
버퍼 캐쉬
"Minnet, jag har tappat mitt minne, ? jag svensk eller finne, kommer inte ih?..." (Bosse ?terberg)

"기억, 나는 내 기억을 잃어버렸어, 내가 스웨덴 사람인지 핀란드 사람인지, 생각나지 않아... " (Bosse 賴terberg)

여기서는 리눅스 메모리 관리에 대하여 설명한다. 즉, 가상 메모리와 디스크 버퍼 캐쉬와 같은 내용에 대해 다룬다. 그리고 메모리 관리가 필요한 이유와 그에 필요한 작업들, 그밖에 시스템 관리자로서 관심을 가져야 할 여러 주제들을 설명할 것이다.

가상 메모리란?
리눅스는 가상 메모리(virtual memory)란 것을 지원한다. 이것은 메모리 사용량이 늘어남에 따라, 디스크의 일부를 마치 확장된 RAM처럼 사용할 수 있게 해주는 기술이다. 이 기술에 따르면, 커널은 실제 메모리(RAM)에 올라와 있는 메모리 블록들 중에 당장 쓰이지 않는 것을 디스크에 저장하는데, 이를 통해 사용가능한 메모리 영역을 훨씬 늘릴 수 있게 된다. 만일 디스크에 저장되었던 메모리 블록이 다시 필요하게 되면 그것은 다시 실제 메모리 안으로 올려지며, 대신 다른 블舅?디스크로 내려가게 된다. 그러나 이런 과정이 일어나고 있다는 것이 사용자에게는 전혀 보이지 않으며, 프로그램들에게도 그저 많은 양의 메모리가 있는 것처럼 보일 뿐이어서, 점유하고 있는 메모리가 디스크에 있는지 실제 메모리에 있는지 전혀 신경쓸 필요가 없게 된다. 그러나, 하드디스크를 읽고 쓰는 시간은 RAM보다 훨씬 느리기 때문에(보통 천배쯤 느리다), 프로그램의 실행은 그만큼 더디게 된다. 이렇듯 가상적인 메모리로 쓰이는 하드디스크의 영역을 `스왑 영역(swap space)'이라고 한다(swap은 바꿔치기를 한다는 뜻).

리눅스는 스왑 영역으로 일반적인 파일을 사용할 수도 있고 별도의 스왑을 위한 파티션을 사용할 수도 있다. 스왑 파티션은 속도가 빠른 반면에, 스왑 파일은 그 크기를 자유롭게 조절할 수 있다(또한 스왑 파일을 사용하면, 리눅스 설치시에 파티션을 다시 해야 할 필요없이 모든 것을 그냥 설치할 수 있다). 스왑 영역이 얼마나 많이 필요한지를 미리 알고 있다면 그만큼 스왑 파티션을 잡으면 된다. 그러나 스왑 영역이 얼마나 필요할지 확실히 모른다면, 우선 스왑 파일을 사용해서 시스템을 가동해 보고 필요한 공간이 얼마인지 파악한 후에 스왑 파티션을 잡도록 하자.

또한 리눅스에서는 여러개의 스왑 파티션과 스왑 파일을 섞어서 사용할 수 있다. 이 방법을 이용하면, 언제나 큰 용량의 스왑 영역을 잡을 필요없이 그때 그때 필요한 만큼만 스왑을 늘려줄 수 있으므로 편리하다.

운영체제 용어에 관한 이야기 : 컴퓨터 과학에서는 스와핑(해당 프로세스 전체를 스왑 영역으로 내보냄)과 페이징(몇 킬로바이트의 작은 단위로 내보냄)을 구별하는 것이 일반적이다. 이 중에서 페이징이 좀더 효율적인 방법이며, 리눅스에서도 이 방법을 쓴다. 그러나 전통적인 리눅스 용어로는 이 두가지를 모두 뭉뚱그려서 스와핑이라고 흔히 불러왔다. [1]

Notes
[1] 그러니 이런 문제로 쓸데없이 컴퓨터 과학자들을 괴롭히지 말자.

스왑 공간 생성하기
스왑 파일은 평범한 파일이다. 즉, 커널이 보기엔 일반 파일과 다를 바가 없다. 다만 다른 점이라면 스왑 파일에는 빈틈(holes)이 없으며, mkswap과 함께 사용하게 되어 있다는 점 정도이다. 그리고 스왑 파일은 꼭 자신의 파일시스템(local filesystem)에 있어야 하며, NFS를 통해 마운트된 파일시스템에 있어선 안 된다.

스왑 파일 안에 홀(hole)이 없어야 한다는 점은 중요하다. 스왑 파일은 디스크의 일부를 미리 점유하고 있는데, 이렇게 하면 디스크 섹터를 일일이 할당하는 과정을 거치지 않고서도 메모리 페이지를 파일로 빠르게 스왑시킬 수 있다. 즉, 커널은 파일에 미리 할당되어 있는 섹터를 곧바로 사용하기만 하면 되는 것이다. 스왑 파일 안에 빈틈이 있다는 것은 아무 섹터도 할당되지 않은 공간이 파일 안에 있다는 뜻인데, 이렇게 되면 커널이 스왑을 사용하는데 곤란을 겪게 된다.

홀이 없는 스왑 파일을 생성하기 위한 좋은 방법은 다음과 같다. $ dd if=/dev/zero of=/extra-swap bs=1024 count=1024
1024+0 records in
1024+0 records out
$

위에서 /extra-swap이란 것은 스왑 파일의 이름이며, bs= 뒤에 오는 숫자는 입출력 단위의 크기를 지정한 것이고(1024 byte, 즉 1 kilobyte), count= 뒤의 숫자는 입출력 단위의 몇배 크기의 파일을 만들 것인지를 지정하기 위한 것이다(즉, 여기서는 1024 kilobyte 크기의 파일을 만든 것이 되겠다). count는 꼭 4의 배수로 지정해 주는 것이 좋은데, 그 이유는 커널이 스왑하는 메모리 페이지(memory page)의 단위가 4 kilobyte이기 때문이다. 만일 파일의 크기를 4 kilobyte의 배수로 하지 않는다면, 파일 끝에 남는 몇 킬로바이트는 아예 사용되지 않을 것이다.

스왑 파티션도 사실 특별한 것은 없다. 만드는 것도 다른 보통 파티션과 다를 것이 없지만, 특별한 점이라면 스왑파티션에는 어떤 파일시스템도 사용되지 않으며 날것(raw partition) 그대로 쓴다는 점이다. 스왑용으로 쓸 파티션은 type 82로 지정해 두는 것이 좋은데, 이렇게 해두면 파티션의 용도가 명확해진다. 그러나 사실 커널은 이런 것에 그다지 구애받진 않는다.

스왑 파일이나 스왑 파티션을 만들고 나면, 그 앞부분에 일종의 인식표를 달아두어야 한다. 여기에는 커널이 사용하는 몇 가지 정보가 위치하게 된다. 이것을 해주는 명령어는 mkswap인데, 다음과 같이 쓰인다. $ mkswap /extra-swap 1024
Setting up swapspace, size = 1044480 bytes
$

이렇게 했다고 해서 이 스왑 공간을 사용하게 된 것은 아니다. 다만 커널이 이것을 가상 메모리로 사용할 수 있도록 준비만 마친 것이다.

mkswap 명령은 사용에 주의가 필요하다. 이 명령은 파일이나 파티션이 사용 중인지 아닌지를 판별해 주지 않기 때문이다.

따라서 mkswap을 부주의하게 사용하면 중요한 파일과 파티션을 간단히 날려버릴 수 있다! 그러나 다행히도, mkswap 명령은 주로 시스템 설치시에만 사용된다는 점이 우리를 안심시켜 주긴 한다.

리눅스의 메모리 관리자는 각각의 스왑 공간의 크기를 약 127MB로 제한하고 있다(몇가지 기술적인 이유로 인해 실제 한계치는 (4096-10) * 8 * 4096 = 133890048 bytes 즉 127.6875 megabytes이다). 대신, 최대 8개의 스왑 공간을 연결해 사용하면 스왑을 대략 1GB까지 확장할 수가 있다. [1]

Notes
[1] 요즘은 어디서나 기가바이트 단위를 들먹인다. 이제는 실제 메모리에도 기가바이트 단위를 사용하게 될 날이 멀지 않았다.

스왑 공간 사용하기
스왑 공간을 초기화하는 데는 swapon 명령을 사용한다. 이 명령은 커널에게 해당 공간을 스왑으로 사용할 수 있다는 점을 알려준다. 이 명령에게는 추가하고자 하는 스왑 공간의 경로를 인수로 전달해 주어야 한다. 임시 스왑 파일을 스왑 공간에 추가하고자 한다면 다음과 같이 한다. $ swapon /extra-swap
$

스왑 공간들은 /etc/fstab 파일에 의해서 자동적으로 사용될 수도 있다. /dev/hda8 none swap sw 0 0
/swapfile none swap sw 0 0

시스템이 시작될 때, 스크립트를 통해서 swapon -a 명령이 실행되는데 이 명령은 /etc/fstab에 나열되어 있는 스왑 공간들을 모두 사용하게 해 준다. 그래서 흔히 swapon 명령은 추가적인 스왑이 필요할 때만 사용되는 것이 보통이다.

free 명령을 쓰면 스왑의 사용 상황을 모니터 할 수 있다. 이것은 현재 얼마나 많은 용량의 스왑이 사용되고 있는지 알려준다. $ free

	total	used	free	shared	buffers
Mem:	15152	14896	256	12404	2528
-/+ buffers:		12368	2784		
Swap:	32452	6684	25768		

$

여기서 Mem: 이라고 쓰여진 첫째줄은 실제 물리적 메모리의 상황을 보여주는 것이다.커널은 물리적 메모리를 약 1 megabyte 정도 사용하는데, total이라고 쓰여진 세로줄에서 보여주는 전체메모리 양에는 이 커널이 차지하는 공간이 빠져있다. used라는 세로줄은 현재 사용중인 메모리 양을 보여주고 있으며(두번째 가로줄은 버퍼 로 사용되는 부분을 제외하고 계산한 양이다), free란 세로줄에서는 전혀 사용되지 않은 양을 보여주고 있다. 또한 shared란 부분은 프로세스간에 공유되고 있는 메모리를 나타내고 있는 것이므로, 그 양이 많은 것은 기쁜 일이다. buffers는 현재 디스크 버퍼 캐쉬로 사용되는 메모리 양을 보여주고 있다.

마지막 줄인 Swap:은 위와 같은 항목을 스왑 공간에 똑같이 적용시킨 내용이다. 이 항목이 모두 제로라면, 스왑 공간이 아예 동작하고 있지 않다는 뜻이다.

같은 정보를 top 명령이나 /proc/meminfo 파일을 통해 얻을 수 있다. 그러나 어느 경우든, 특정한 스왑 공간에 대한 정보를 얻는 것은 좀 어렵다.

스왑 공간은 swapoff 명령으로 기능을 멎게 할 수 있다. 그러나 임시로 잡은 스왑 공간이 아니라면, 스왑을 끌 필요는 없다. 만약 스왑을 끄게되면, 스왑 공간에 들어있던 메모리 페이지들이 먼저 실제 메모리로 들어가야 되는데, 실제 메모리에 여유가 없는 경우에는 또 다른 스왑 공간으로 방출되게 된다. 그런데 이 메모리 페이지들을 모두 수용하기에 가상메모리마저도 부족하다면, 그때부터는 리눅스 시스템이 무진장 버벅대기 시작할 것이다. 시간이 아주 많이 걸린 후에는 좀 잠잠해지겠지만, 여전히 시스템은 사용불능 상태에 있게 된다. 따라서 스왑을 끄기 전에, 충분한 여유 메모리가 있는지 꼭 확인해 보아야만 한다(free 같은 것으로).

swapon -a 명령으로 자동적으로 사용되는 스왑 공간들은, 마찬가지로 swapoff -a 명령을 써서 끌 수 있다. 이것도 역시 /etc/fstab 파일에 나열되어 있는 스왑 공간만을 끄기 때문에, 나머지 수동으로 추가시킨 스왑들은 영향을 받지 않는다.

때때로, 실제 메모리가 많이 비어 있는데도 불구하고 스왑을 아주 많이 쓰고 있는 경우를 보게 될 수가 있다. 보통 이런 일이 발생하는 경우는 이렇다. 어떤 덩치 큰 프로세스가 실제 메모리를 많이 점유하는 바람에 시스템이 스왑을 많이 사용하게 되었다고 하자. 이 프로세스가 종료되면 실제 메모리엔 여유 공간이 많이 남게 되지만, 스왑으로 한번 내려간 데이터는 그것이 당장 필요하지 않는 한 실제 메모리로 불려지지 않는다. 따라서 스왑 영역을 많이 사용하면서도 실제 메모리가 많이 비어있는 현상이 꽤 오래 지속될 수 있는 것이다. 그러므로 이런 현상에 특별히 신경쓸 필요는 없다. 하지만, 최소한 그 원리는 이해하고 있어야 나중에 불안하지 않을 것이다.

다른 운영체제와 스왑 공간을 공유하기
가상 메모리 기술은 이미 많은 운영체제에 내장되어 있다. 그런데, 운영체제는 단지 그것이 실행 중일 때만 스왑을 필요로 한다. 따라서 한 컴퓨터에서 다양한 운영체제를 사용한다면, 각각의 운영체제마다 따로 스왑 공간을 마련해 주는 것은 낭비일 것이다. 실제로 서로 다른 운영체제가 스왑 공간을 공유하는 것이 가능한데, 다만 그렇게 하기 위해서는 조금의 해킹이 필요하다. 실제로 이것을 어떻게 구현할 수 있는가에 대한 정보는 각종 Tip이나 HOWTO를 참고하기 바란다.

스왑 공간 할당하기
보통, 스왑 공간을 잡을 때는 그 크기를 물리적인 메모리의 두 배 정도로 하는 것이 적당하다고 말하는 사람들이 많은데, 사실 이것은 좀 근거없는 이야기이다. 여기서 좀더 합리적인 방법을 알아보도록 하자.

우선, 필요한 메모리의 총량을 어림잡아 본다. 필요한 메모리의 총량이라는 것은, 한순간에 필요한 메모리의 최대 크기, 즉 한꺼번에 돌리고 싶은 모든 프로그램들이 필요로 하는 메모리의 총량을 말하는 것이다. 실제로 원하는 모든 프로그램을 한번에 다 띄워놓는다면 바로 이런 상태를 만들 수 있다.

예를 들어, X를 띄우고 싶다면 여기엔 대략 8MB 정도의 메모리 할당이 필요하다. gcc를 돌리고 싶다면 보통 4MB 정도가 필요하지만, 특별히 큰 프로그램을 컴파일한다면 아마 수십 메가바이트 이상이 필요할 것이다. 또한 커널은 그 자체가 대략 1MB 정도 차지하며, 일반적인 쉘들과 작은 유틸리티들은 각각 수백 킬로바이트 정도 잡아먹는다(다 합치면 1 메가 정도 될 것이다). 이런 어림짐작들이 꼭 정확해야 할 필요는 없지만, 좀 더 비관적인 태도로 계산해 볼 필요는 있다.

한가지 명심할 것은, 만약 같은 프로그램을 동시에 돌리고 있는 여러 사람이 있다면, 이들도 모두 각각 메모리를 잡아먹는다는 점이다. 다만, 예를 들어, 같은 프로그램을 두사람이 돌린다고 했을 때 그 메모리 점유량이 원래의 꼭 두배가 되는 것은 아닌데, 왜냐면 프로그램의 실행 코드와 공유 라이브러리들은 메모리에 한벌씩만 올려두고 공유해 사용할 수 있기 때문이다.

free와 ps 명령을 적절히 사용하면 메모리 필요량을 알아보는 데 유용하다.

이제, 앞에서 나온 값에다 안전보장용 여유분을 좀 더하자. 이렇게 하는 이유로서는, 우선 프로그램의 크기 계산이 틀렸을 수도 있고, 또한 실행하고자 하는 프로그램을 몇개 빠뜨렸을 수도 있기 때문이다. 이런 경우를 대비하여 좀 여유 공간을 두어야 하는데, 필요하다고 계산된 크기의 10% 정도를 여유로 잡아두면 충분할 것이다.(스왑을 너무 적게 잡는 것보다는 너무 많이 잡는 것이 낫지만, 그래도 지나치게 많은 양을 스왑으로 잡는 것은 낭비일 뿐이다. 스왑이 더 필요하다면 나중에 추가할 수도 있다.) 이렇게 필요한 크기가 계산되었으면, 편의를 위해 반올림을 해서 메가바이트 단위로 만들자.

위의 계산에 근거하면, 총 얼마 정도의 메모리가 필요한지 파악이 될 것이다. 이제, 필요한 스왑 공간을 계산하기 위하여, 총 필요 메모리량에서 실제 물리적 메모리량을 빼자.(어떤 UNIX 버전에서는, 실제 물리적 메모리의 이미지를 위한 공간까지 스왑에 포함시켜주어야 한다. 이런 경우에는 위의 두번째 단계에서 산출된 필요 메모리 크기 전부를 스왑 공간으로 할당해야 하며, 빼기를 해서는 안된다.)

만일 산출된 스왑 공간이 실제 메모리의 두배를 넘는다면, 실제 메모리를 좀 더 확충하는 방안을 고려해 보자. 그러지 않는다면 시스템이 아주 기어가게 될 것이다.

계산 결과 스왑 공간이 전혀 필요없다고 해도, 약간의 스왑을 잡아두는 것이 좋다. 리눅스는 메모리에 될 수 있는 대로 많은 여유공간을 확보하려 하는데, 이를 위해 스왑을 아주 적극적으로 사용한다. 즉, 실제 메모리에 여유가 많이 있다 하더라도, 사용되지 않고 있는 메모리 페이지가 있다면 그 부분은 스왑 영역으로 내려진다. 이처럼 디스크가 쉬고 있을 때 미리 스왑을 해두기 때문에, 스왑으로 인한 지연 시간을 많이 줄일 수 있다.

또한 스왑 공간은 여러개의 디스크에 나누어져 있을 수도 있는데, 이렇게 하면, 디스크의 속도와 그 액세스 방식에 따라서 스왑 성능이 향상되기도 한다. 그 밖에도 여러 방식이 있을 수 있으므로 그것을 시험해 보고 싶겠지만, 보통 그런 방식들은 제대로 시험해 보기가 쉽지 않다. 특히, `어떤 방식이 다른 것보다 훨씬 월등하다'는 식의 말은 절대로 믿지 마라. 그런 것들은 거의 언제나 사실이 아니다.

버퍼 캐쉬

디스크를 읽는 일은 (진짜) 메모리를 읽는 것보다 아주 느리다. [1] 더구나, 디스크의 동일한 영역을 짧은 시간 동안 반복해서 계속 읽는 일은 아주 빈번하다. 예를 들어, 누군가 e-mail 메시지를 읽고, 답장을 하기 위해 편집기로 불러들이고, 그걸 보내기 위해 메일프로그램에게 다시 읽게 하는 과정을 생각해 보자. 또한 ls 명령어 같은 것을 시스템의 모든 사용자들이 얼마나 자주 사용할지 생각해 보자. 따라서, 디스크로부터 한번 읽어들인 정보를 메모리에 상당시간 보관한다면, 첫번째로 읽을 때만 시간이 좀 걸릴 뿐 속도가 전반적으로 빨라질 것이다. 바로 이런 것을 가리켜 디스크 버퍼링(disk buffering)이라고 하며, 이런 목적으로 쓰이는 메모리를 버퍼 캐쉬(buffer cache)라고 부른다.

그러나 메모리는 아쉽게도 한정된, 아니, 아주 귀중한 자원이기 때문에, 버퍼 캐쉬는 보통 큰 크기를 가질 수 없다(즉, 우리에게 필요한 모든 데이터를 담아둘 수 있을 정도로 크지는 않다). 따라서, 캐쉬가 다 차게 되면 오랫동안 쓰이지 않은 데이터는 버려지며 그 빈 공간을 새로운 데이터가 메우게 된다.

이런 디스크 버퍼링은 쓰기에도 똑같이 적용된다. 보통, 데이터들은 쓰여지자 마자 또 곧바로 다시 읽어들여지므로(예를 들어, 소스 코드 파일은 일단 파일로 저장된 후, 컴파일러에 의해 다시 읽어들여진다), 이런 데이터들을 캐쉬에 넣어둔다면 확실히 효율적일 것이다. 또한, 쓰기 작업을 디스크에 즉시 하지 않고 캐쉬에 넣어두면, 프로그램들이 그만큼 출력을 빨리 끝낼 수 있기 때문에 전반적인 시스템 성능향상에도 도움이 된다.

대부분의 운영체제들이 버퍼 캐쉬를 갖고 있긴 하지만(좀 다른 이름으로 불릴 수도 있다), 모두가 위와 같은 원리로 동작하는 것은 아니다. 한가지 방법은 write-through라는 것인데, 이 방법은 쓰기를 할 때면 언제나 디스크에도 즉시 기록하는 것이다(물론 캐쉬에도 남겨둔다). 또 다른 방법은 write-back이라 불리는 것으로, 쓰기를 일단 캐쉬에 해 두었다가 나중에 한꺼번에 디스크에 기록하는 방식이다. 효율적이기는 write-back 방식이 뛰어나지만, 대신 약간의 에러가 발생할 소지가 있다. 즉, 시스템이 갑자기 멈춰버린다거나, 전원이 갑자기 나가버린다면, 또는 캐쉬 내용을 미처 써 넣기 전에 플로피를 빼 버린다면, 캐쉬에 담겨 있던 내용들은 고스란히 날아가 버리고 만다. 특히, 손실된 정보가 파일시스템 유지에 필요한 중요 데이터였다면, 자칫 전체 파일시스템을 망가뜨리고 마는 결과를 초래할 수도 있다.

이렇기 때문에, 컴퓨터를 끄기 전엔 반드시 적절한 셧다운 절차를 밟아야만 하는 것이고(Chapter 6 참조), 마운트한 플로피를 빼기 전엔 꼭 언마운트를 해야하는 것이다. 한편, 캐쉬를 디스크로 내보내기(flush) 위한 명령으로 sync가 있는데, 이 명령을 쓰면 아직 기록되지 않고 캐쉬에 남아있는 데이터들을 모두 디스크에 써넣게 되므로, 모든 내용이 안전하게 기록되었다는 점을 보장받을 수가 있다. 또한 전통적인 UNIX에는 update란 백그라운드 프로그램이 있어서, sync가 해주는 것과 같은 일을 30초에 한번씩 자동으로 해준다. 그러므로 사실 sync를 별로 사용할 필요는 없는 없는 셈이다. 특히, 리눅스에는 추가적인 데몬으로 bdflush란 것이 있는데, 이 것은 sync에 비해선 상당히 불충분하게 flush 작업을 하지만 대신 좀더 자주 실행하도록 되어 있다. 이런 방식이 고안된 이유는, sync가 디스크 입출력을 순간적으로 과도하게 일으키면서 시스템이 멈춰버리는 현상이 종종 있어 왔기 때문이다.

리눅스에서는, update에 의해 bdflush가 구동된다. 보통 때는 이 데몬들에 별로 신경 쓸 필요가 없지만, 만일 bdflush가 어떤 이유로 죽어버린다면 커널이 이 사실을 바로 알려줄 것이다. 이럴 때는 수동으로 실행시켜 주면 된다(/sbin/update).

그런데, 사실 캐쉬는 파일을 버퍼링하는 것은 아니고, 실제로는 디스크 입출력의 가장 작은 단위인 블록을 버퍼링한다(리

눅스에서는 보통 1KB 크기이다). 그렇기 때문에, 디렉토리라든가, 수퍼 블록들, 다른 파일시스템의 유지 데이터, 심지어 파일시스템이 없는 디스크까지도 캐쉬될 수가 있는 것이다.

캐쉬의 효율성은 기본적으로 그 크기에 좌우된다. 캐쉬의 크기가 너무 작으면, 다른 데이터를 캐쉬하기 위해서 캐쉬된 데이터를 계속 내보내야 하므로, 사실상 작은 캐쉬는 별 쓸모가 없는 셈이다. 캐쉬가 어느 정도 쓸모있기 위한 최소한의 크기는, 얼마나 많은 데이터가 읽고 씌여지는지와, 같은 데이터가 얼마나 자주 액세스되는지에 달려있는데, 이것을 알아보기 위한 단 하나의 방법은 그저 실험해보는 것 뿐이다.

만일 캐쉬의 크기가 고정되어 있다면, 그 크기가 너무 큰 것도 곤란한 일일 것이다. 캐쉬가 너무 크면 여유 메모리는 그만큼 줄어들 것이고, 많은 스와핑을 일으켜서 시스템은 느려지게 된다. 리눅스는 자동적으로 모든 RAM의 빈공간을 버퍼 캐쉬로 사용하여 메모리의 효율성을 높이려 하는데, 프로그램들이 많은 메모리를 필요로 할 때는 자동적으로 캐쉬를 크기를 줄여 준다.

그래서, 리눅스에서는 캐쉬를 사용하는 데 대해서 아무것도 신경쓸 필요가 없다. 완벽하게 자동적이기 때문이다. 다만, 셧다운 할 때와 플로피를 빼낼 때의 절차는 꼭 지켜 주어야 한다. 이것만 빼면, 걱정할 것은 하나도 없다.

Notes
[1] RAM 디스크의 경우는 제외이다. 이것은 RAM에 만들어진 가상의 디스크이므로, 당연히 램만큼이나 속도가 빠르다.

Chapter 6. 부팅과 셧다운
Table of Contents
부팅과 셧다운 과정의 개괄
부팅의 세부 과정
셧다운의 세부 과정
리부팅
단일 사용자 모드
응급 부팅 플로피

Start me up
Ah... you've got to... you've got to
Never, never never stop
Start it up
Ah... start it up, never, never, never
 You make a grown man cry,
  you make a grown man cry
(Rolling Stones)

나 일으켜 줘요
앙... 해줘요... 해줘요
절대로, 절대 절대 그만두지 말고
일으켜줘요
앙... 일으켜줘요, 진짜루, 진짜루, 진짜루
다큰 사람 울리지 말구.
다큰 사람 울리지 말구
(롤링 스톤즈 : 미국의 록 그룹)

여기서는 리눅스 시스템이 시작될 때와 멈춰질 때 어떤 일이 진행되는지를 설명할 것이며, 또한 그것이 제대로 진행되려면 어찌 해야하는지에 대해서도 알아볼 것이다. 만일, 이 때 적절한 과정이 수행되지 못한다면, 파일들이 손상을 입거나 지워질 수도 있다.

부팅과 셧다운 과정의 개괄
컴퓨터 시스템에 전원을 넣고 운영체제를 불러들이는 과정 [1] 을 가리켜 부팅(booting)이라고 한다. 이 용어는 컴퓨터가 남의 도움없이 스스로 신발끈(bootstrap)을 질끈 동여매고 일어서는 모습을 연상시키는데, 사실 실제과정이 이렇게 단순하지는 않다.

신발을 신고 일어서기 위해선 우선 신발끈을 동여매야 하듯이, 운영체제가 부팅을 하기 위해선 우선 부트스트래핑(bootstrapping) 과정을 거쳐야 한다. 우선 컴퓨터는 부트스트랩 로더(bootstrap loader)라는 작은 기계어 코드를 불러들이게 되는 데, 이 프로그램은 다시 운영체제를 불러들여서 그것을 시동시킨다. 바로 이 과정이 부트스트래핑이며, 부트스트랩 로더는 보통 하드디스크나 플로피의 특정 영역에 위치하고 있다. 이런 두 단계의 과정을 거치는 이유는 컴퓨터가 맨 처음 읽어들일 수 있는 코드의 크기에 제한이 있기 때문이다(대략 몇백 바이트 정도). 만일 크고 복잡한 운영체제를 바로 읽어들일 수 있도록 하려면, 펌웨어를 괜히 복잡하게 만들어야만 할 것이다.

이런 부트스트래핑 과정은 컴퓨터의 종류에 따라 다 다르다. PC의 경우, 컴퓨터(BIOS)는 플로피나 하드디스크의 섹터(boot sector라고도 함)를 읽어들이게 되어 있으며, 이곳에 바로 부트스트랩 로더가 들어 있다. 불려진 부트스트랩 로더는 디스크의 다른 부분에서 운영체제를 읽어들이게 된다.(디스크말고 다른 곳에서 운영체제를 불러들일 수도 있다.)

일단 리눅스가 불려지게 되면 우선 하드웨어와 장치 드라이버들이 초기화되며, 그 다음에 init가 실행된다. init는 사용자들이 로그인해 작업을 할 수 있도록 기타 다른 프로세스들을 시동시켜 준다. init에 대한 자세한 이야기는 뒤에서 하도록 하자.

리눅스 시스템을 셧다운시키기 위해서는, 먼저 모든 프로세스들에게 종료하라는 지시를 내려야한다(이 지시를 받으면, 각 프로세스들은 그들이 사용하던 파일을 닫고 기타 작업들을 깔끔히 정리하게 된다). 그 다음에는 파일시스템과 스왑 공간을 언마운트 해야 하며, 이 모든 작업이 끝나야 비로소 콘솔에 전원을 내려도 좋다는 메시지를 출력하게 된다. 만일 이런 과정이 제대로 수행되지 않는다면, 아주 끔찍한 일이 벌어질 수도 있다. 특히, 파일시스템의 버퍼 캐쉬가 제대로 비워지지 않는다면, 데이터들은 다 날아가고 파일시스템이 불안해져서 결국 못쓰게 되는 사태가 발생할 수도 있다.

Notes
[1] 초창기 컴퓨터들은 단지 전원을 켜는 것만으로는 부팅이 되지 않았고, 직접 수동으로 운영체제를 불러들여야만 했다. 또한 이 최신 유행의 장난감들은 뭐든지 혼자 힘으로 해내야만 했다.

부팅의 세부 과정
리눅스는 플로피나 하드디스크로부터 부팅될 수 있다. 리눅스를 설치하고 부팅하는 방법에 관해서는 "Installation and Getting Started guide"를 참고하기 바란다.

일단 PC에 전원이 들어오게 되면, BIOS는 우선 시스템의 하드웨어에 문제가 없는지 다양한 테스트를 해보게 된다. [1] 그리고 문제가 없다면 부팅을 시작시킨다. BIOS는 먼저 어느 디스크 드라이브로부터 부팅을 시작할 것인지 선택하는데, 보통 첫번째 플로피 드라이브에 플로피가 들어있다면 플로피로부터 부팅하려 할 것이고, 그렇지 않다면 첫번째 하드디스크로부터 부팅을 시도할 것이다.(이 순서는 다르게 설정할 수도 있다.) 그리고 디스크의 첫번째 섹터를 읽어 들이는데, 이것이 바로 부트 섹터(boot sector)이다. 또한 하드디스크가 여러 파티션을 갖고 있는 경우에는 부트 섹터를 각각 따로 갖게되는데, 이때는 디스크의 첫번째 섹터를 마스터 부트 레코드(master boot record) 라고 부르기도 한다.

부트 섹터에는 작은 프로그램(섹터 하나에 들어갈 수 있을만큼 작은)을 넣어두는데, 이 프로그램이 운영체제를 읽어들이고 실행을 시키게 된다. 플로피 디스크로부터 리눅스를 부팅할 때는, 이 프로그램이 디스크의 첫번째 몇백 블록(물론 커널의 크기에 따라 달라진다)을 메모리의 특정장소로 읽어들인다. 리눅스 부트 플로피에는 파일시스템이 없어서, 커널은 그저 연속적인 섹터들 안에 그대로 저장된다. 이렇게 하는 이유는 부팅 과정을 좀더 간단하게 하기 위해서이다. 하지만, LILO 즉 리눅스 로더(LInux LOader)를 사용하면 파일시스템이 있는 플로피에서도 부팅을 할 수가 있다.

하드 디스크에서 부팅할 때는, 우선 마스터 부트 레코드의 프로그램이 파티션 테이블(이것도 역시 마스터 부트 레코드 안에 있는 정보이다)을 검사한다. 그리고 이 과정을 통해 어느 파티션이 활성화된 파티션(즉 부팅이 가능하다고 표지된 파티션)인지를 알아본 후에, 그 파티션의 부트 섹터를 읽어서 그 코드를 실행시킨다. 그러나 이 부트 섹터의 역할은 플로피의 경우와 좀 달라서 이것은 커널을 파티션으로부터 읽어들이고 실행시켜야 한다. 그런데, 각 파티션에는 파일시스템이

존재하므로 플로피의 경우처럼 디스크를 단순히 순차적으로 읽을 수는 없다. 이 문제를 해결하기 위한 여러 방법들이 있는데, 그 중에 가장 많이 쓰는 것이 바로 LILO이다. LILO는 커널이 어느 섹터에 위치하는 지를 미리 파악해 두었다가, 부팅때 이 정보를 가지고 커널을 읽어들이는 방법을 쓴다. 이 방식은 파일시스템이 없는 파티션을 따로 만들어서 커널을 저장하는 것보다 훨씬 효율적이다. (LILO의 동작에 관해 더욱 자세한 내용은 관련 문서를 참조하기 바란다.)

LILO로 부팅을 하게 되면, 보통 기본 설정된 커널로 부팅이 된다. 그러나 설정을 바꿔주면 몇가지 다른 커널을 사용해 부팅할 수도 있고, 심지어 아예 다른 운영체제로도 부팅이 가능하다. 그래서 부팅시에 어떤 커널이나 운영체제로 부팅을 할 것인지 사용자가 직접 고를 수 있다. 즉, 부팅시 LILO가 떴을 때, alt, shift 또는 ctrl 키를 누른 후 선택을 입력하게 할 수도 있고, 아예 언제나 입력을 요구하도록 설정할 수도 있다. 선택을 하지 않는다면, 지정된 대기 시간이 지난후 기본 설정으로 부팅이 될 것이다.

또한 LILO는 커널에 명령행 인자(kernel command line argument)를 전달하는 데도 유용하게 쓰인다.

플로피로부터의 부팅이든 하드 디스크로부터의 부팅이든 각자 장단점이 있지만, 번거로운 플로피 부팅보다는 하드 디스크 부팅이 보통 더 빠르고 산뜻한 방법이다. 다만, 시스템을 설치한 후 바로 하드 디스크로 부팅을 시도하는 것은 문제를 발생시킬 소지가 많으므로, 보통은 일단 플로피로 부팅을 해보고 시스템에 문제가 없는 것을 확인한 후, LILO를 설치하고 하드 디스크 부팅을 하게 되는 일이 많다.

일단 리눅스 커널이 메모리 속으로 읽혀지고나면, 진짜 부팅 과정이 시작된 것이라 볼 수 있다. 이제부터는 대략 다음과 같은 일이 일어나게 된다.

리눅스 커널은 압축된 형태로 설치되어 있다. 따라서 우선 압축을 풀어야 한다. 그래서 압축된 커널 이미지의 첫부분은 압축을 풀기 위한 작은 프로그램으로 되어 있다.

만약 특별한 텍스트 모드를 지원하는 super-VGA 카드가 설치되어 있다면, 리눅스가 어떤 모드를 사용해야 하는지 물어 볼 수 있다. 그러나 보통 커널 컴파일시에 미리 설정되므로, 그 이상 묻지는 않는다. 텍스트 모드의 선택은 LILO나 rdev를 통해서도 할 수 있다.

이런 과정이 끝나면, 커널은 어떤 하드웨어들이 장착되어 있는지 체크하고(하드 디스크, 플로피, 네트워크 어댑터 등), 적절한 장치드라이버를 설정한다. 이 동안에 어떤 장치가 인식되었는지를 보여주는 메시지가 출력된다. 예를 들면 다음과 같다. LILO boot:
```
Loading linux.
Console: colour EGA+ 80x25, 8 virtual consoles
Serial driver version 3.94 with no serial options enabled
tty00 at 0x03f8 (irq = 4) is a 16450
tty01 at 0x02f8 (irq = 3) is a 16450
lp_init: lp1 exists (0), using polling driver
Memory: 7332k/8192k available (300k kernel code, 384k reserved, 176k data)
Floppy drive(s): fd0 is 1.44M, fd1 is 1.2M
Loopback device init
Warning WD8013 board not found at i/o = 280.
Math coprocessor using irq13 error reporting.
Partition check:
  hda: hda1 hda2 hda3
VFS: Mounted root (ext filesystem).
Linux version 0.99.pl9-1 (root@haven) 05/01/93 14:12:20
```

텍스트의 세부적인 내용은 시스템의 하드웨어와 리눅스 버전에 따라, 또 그것이 어떻게 설정되었느냐에 따라 달라진다.

이제 커널은 루트 파일시스템(root filesystem)을 마운트하려 할 것이다. 이 위치는 컴파일시에 지정될 수도 있고, rdev

나 LILO를 통해 정해줄 수도 있다. 또한 파일시스템 타입은 자동적으로 감지된다. 만일, 적합한 파일시스템 드라이버를 커널에 포함시키지 않았다든지 하는 이유로, 파일시스템을 마운트하는 데 실패한다면 커널은 공황상태(panic)에 빠져들고 시스템은 그저 꺼지는 수 밖에 없다.(루트 파일시스템이 마운트되지 않으면 아무것도 할 수가 없다.)

루트 파일시스템은 흔히 읽기 전용으로만 마운트된다.(역시 위와 같은 방법으로 설정할 수 있다) 이렇게 하면 마운트한 상태에서도 파일시스템을 안전하게 점검할 수 있게 된다. 읽고 쓰기 가능하도록 마운트를 하고서 파일시스템을 점검하다가는 파일시스템이 손상을 입을 수도 있다.

그리고 나서, 커널은 init 프로그램을 백그라운드로 실행시킨다(/sbin/init). init는 가장 먼저 실행되는 프로세스이므로, 그 프로세스 번호는 1이 된다. init는 시스템 시작을 위한 다양한 작업을 수행하는데, 최소한 몇가지 필수적인 백그라운드 데몬을 실행하도록 되어 있다. init가 정확히 어떤 일을 하느냐는 설정에 따라 달라진다. init에 대한 더 자세한 정보는 Chapter 7에 설명하였다.

그 다음, init는 다중사용자 모드로 전환되며, getty를 가상 콘솔과 시리얼 라인 터미널들에 띄운다. getty는 사용자들이 가상 콘솔이나 시리얼 라인 터미널을 통해 로그인 할 수 있도록 해주는 프로그램이다. 또한 init가 어떻게 설정되느냐에 따라, 여기서 몇가지 다른 프로그램들을 실행하기도 한다.

이렇게 해서, 부팅은 완료되었다. 이제 시스템은 정상적으로 가동된다.

Notes
[1] 이것을 POST(power on self test)라고 부른다.

셧다운의 세부 과정
리눅스 시스템을 셧다운시킬 때, 적절한 절차를 밟아야 한다는 점은 아주 중요하다. 이렇게 하지 못한다면, 파일시스템이 망가지거나 파일들이 손상을 받을 것이다. 이렇게 되는 이유는, 리눅스가 디스크에 쓰기를 바로 하지 않고 디스크 캐쉬를 거치기 때문이다. 이 방식은 시스템의 성능을 향상시켜 주지만, 만일 캐쉬의 내용이 디스크에 미처 기록되기 전에 전원을 내린다면 파일시스템이 망가지고 마는 위험도 갖고 있다.(왜냐면 디스크의 중요내용이 결손된 상태로 남게 되기 때문이다).

전원을 함부로 내려서는 안되는 또한가지 이유로는, 많은 백그라운드 작업들이 멀티 태스킹 환경에서 돌아가고 있다는 점을 들 수 있다. 이런 상태에서 그대로 전원을 내린다는 것은 상당히 위험한 일이다. 그러나 적절한 셧다운 과정을 거친다면, 모든 백그라운드 작업들이 데이터를 안전하게 저장하도록 할 수 있다.

리눅스 시스템을 안전하게 셧다운 시키는 명령이 바로 shutdown이다. 이 명령은 흔히 두가지 방식으로 사용한다.

만일 시스템을 혼자서만 사용한다면, 우선 모든 프로그램의 실행을 끝내고 모든 가상 콘솔에서 로그 아웃한 뒤, 다시 루트로 로그인하여 shutdown -h now 명령을 내려야한다.(이미 루트로 작업하고 있었다면, 루트 디렉토리나 루트의 홈디렉토리로 이동한 뒤에 명령을 내려야 한다. 이러지 않으면 언마운트시 문제가 생길 수 있다) 여기서 now라는 말대신, +기호와 함께 숫자를 넣어주면 그만큼 시간(분 단위)이 흐른 뒤에 시스템이 종료된다. 그러나 대부분의 단일 사용자 시스템에서는 이럴 필요가 없을 것이다.

그러나 사용자들이 많은 시스템이라면, shutdown -h +time message 이런 방식으로 명령을 내리도록 한다. time 부분은 시스템이 종료되기까지 남은 시간을 써 넣는 부분이며, message 부분은 사용자들을 위한 안내문을 써 넣는 부분이다. # shutdown -h +10 'We will install a new disk. System should
> be back on-line in three hours.'
#

위 명령은 모든 사용자들에게 `시스템이 10분 후에 종료하니 각자 작업을 정리하세요'라는 경고를 담고 있다. 이 내용은 사용자들이 로그인한 모든 터미널(xterm 을 포함하여)에 출력된다. Broadcast message from root (ttyp0) Wed Aug 2 01:03:25 1995...

We will install a new disk. System should
be back on-line in three hours.
The system is going DOWN for system halt in 10 minutes !!

이 경고는 자동적으로 반복 출력되는데, 셧다운 시간이 가까워질 수록 점점 자주 출력되도록 되어 있다.

좀 있다 진짜로 셧다운이 시작되면, 우선 루트를 제외한 모든 파일시스템이 언마운트되며, 로그아웃하지 않은 사용자들의 프로세스들은 죽여진다. 그리고 데몬들까지 종료되고 나면, 마지막으로 루트 파일시스템도 언마운트되면서 모든 것이 종료된다. 이 모든 과정이 끝나고 나면, init는 전원을 꺼도 좋다는 메시지를 화면에 뿌려주는데, 비로소 이때가 되어야 전원에 손을 댈 수가 있는 것이다.

좋은 시스템에서는 드문 일이지만, 가끔 셧다운 절차를 제대로 밟을 수 없는 경우가 있다. 예를 들어, 커널이 패닉 상태에 빠졌다든지 시스템이 먹통이 되어 꼼짝할 수 없게 되면 더 이상 어떤 명령도 입력할 수가 없기 때문에 시스템을 적절히 셧다운시키기 어렵게 된다. 이럴 때 할 수 있는 일이라고는, 그저 별일 없기를 바라면서 전원을 내리는 수 밖에 없다. 만일, 문제가 좀 덜 심각한 경우(즉, 누가 키보드를 도끼로 내리쳤다는가 하는 경우..-.-)로서 커널과 update 프로그램이 제대로 동작한다면, update가 버퍼 캐쉬를 디스크로 내보낼 수 있도록 2분 정도 기다린 후 전원을 끄는 것이 좋은 방법이다.

어떤 사람들은 셧다운을 한답시고 sync [1] 명령을 세번 정도 두들긴 후, 디스크 입출력이 멈추면 그대로 전원을 내려버리기도 한다. 만일 돌고 있는 프로그램이 아무것도 없다면, 이것은 shutdown과 같은 효과를 낼 수도 있다. 그러나, 이 방법은 디스크 언마운트를 전혀 하지 않기 때문에, ext2 파일시스템의 `clean filesystem' 플래그와 문제를 일으키게 된다. 따라서 이 `세번 sync' 방법은 삼가해야 한다.

(궁금한 사람들을 위해: UNIX 초창기에는, 명령어를 몇번 타이핑하는 정도의 시간이면 디스크 입출력이 완료되는데 충분한 것으로 간주하였다. 이것이 sync를 세번두들기는 이유이다.)

Notes
[1] sync는 버퍼 캐쉬를 디스크로 내보내는 명령어이다.

리부팅
리부팅이란, 시스템을 다시 부팅하는 것이다. 이것은 우선 셧다운을 적절히 하고, 전원을 내린 뒤, 다시 전원을 올리는 과정으로 이루어진다. 간단한 방법은, 직접 껐다 켜는 대신 shutdown에게 리부팅을 시키는 것이다. 이렇게 하려면, shutdown에게 -r 옵션을 주면 된다. 즉, shutdown -r now 요렇게 한다.

또한 대부분의 리눅스 시스템은 키보드의 ctrl-alt-del을 함께 눌렀을 때 shutdown -r now 명령을 실행한다. 따라서 리부팅이 당장 이루어지게 된다. ctrl-alt-del이 눌려졌을 때, 어떤 동작을 하게 할 것인지는 설정이 가능한데, 다중 사용자 시스템에서는 일정 시간 지연이 있은 후 리부팅하게 하는 것이 좋다. 또한 아무나 시스템에 물리적으로 접근이 가능한 환경이라면, 이 기능은 아예 꺼두어야 할 것이다.

단일 사용자 모드
단일 사용자 모드일 때는 아무도 로그인 하지 못하며, 단지 루트 사용자만 콘솔 상에서 작업할 수 있다. shutdown 명령은 이 단일 사용자 모드에서 시스템을 끌 때도 사용된다. 단일 사용자 모드는 시스템 가동시엔 하기 어려운 여러가지 시스템 관리 작업을 할 때 유용하다.

응급 부팅 플로피
사실 언제나 하드 디스크로부터 부팅이 가능한 것은 아니다. 예를 들어, LILO의 설정이 잘못되었다면 부팅은 이루어지지 않을 것이다. 이런 상황에 대비해서, 언제나 시스템을 부팅시킬 수 있는 다른 방법이 필요하다.(물론 이런 경우라도 하드웨어는 제대로 동작해야 한다.) 대부분의 일반적인 PC들은, 플로피로부터 부팅하는 방법으로 이런 문제를 해결할 수 있다.

대부분의 리눅스 배포판들은, 설치시 응급 부팅 플로피(emergency boot floppy) 하나를 만들 수 있도록 해준다. 위와 같은 상황에서, 이런 응급 부팅 플로피는 아주 유용할 것이다. 하지만, 보통 이런 부팅 디스크들은 단지 커널만을 포함하고 있으며, 문제 해결에 필요한 프로그램들은 배포판의 설치 디스크에 있는 것을 쓰도록 되어 있는 경우가 많다. 그런데 어떨 때는 이 프로그램들만으로는 부족할 수가 있다. 즉, 만일 백업 프로그램을 설치 디스크에서 제공되지 않는 것으로 사용했다면, 파일을 복원하기 위해서 설치 디스크에 있는 프로그램을 쓸 수는 없을 것이다.

그래서, 독자적으로 만든 루트 플로피가 필요할 수 있다. 이런 플로피를 만드는 방법은 Graham Chapman이 쓴 Bootdisk HOWTO를 참고하기 바란다. 그리고 응급 부팅 플로피와 루트 플로피는 언제나 최신으로 유지해야 한다는 점을 명심하도록 하자.

루트 플로피를 마운트하고 있는 플로피 드라이브는 다른 플로피를 마운트할 수 없다. 이 점은 플로피 드라이브가 하나뿐인 상황에서 아주 불편한 일이다. 그러나 메모리가 충분하다면 루트 디스크를 램디스크(ramdisk)로 읽어들이도록 설정할 수가 있다(이렇게 하려면 부팅 플로피의 커널에 특별한 설정이 필요하다). 램디스크란, 메모리 상에 가상의 파일시스템을 만들어 디스크처럼 사용하는 기법이다. 일단 루트 플로피가 램디스크 안으로 읽혀지기만 하면, 그 다음부터는 다른 디스크를 맘대로 마운트할 수가 있다.

Chapter 7. init
Table of Contents
init가 먼저 나타난다
getty를 실행하기 위한 init 설정: /etc/inittab 파일
실행 레벨
/etc/inittab에서의 특수 설정
단일 사용자 모드에서의 부팅

"Uuno on numero yksi" (Slogan for a series of Finnish movies.)

"뭐든지 첫번째에 달려 있다" (핀란드의 한 영화 시리즈 슬로건)

여기서는 커널에 의해 실행되는 첫번째 프로세스인 init에 대해 다룬다. init는 여러가지로 중요한 역할을 하는데, getty를 띄운다거나(사용자들이 로그인 할 수 있게 해준다), 실행 레벨을 구현하고, 고아가 된 프로세스들을 돌봐주는 등의 일이 모두 init 몫이다. 이제 init가 어떻게 설정되며, 다른 실행 레벨로는 어떻게 전환시킬 수 있는지 알아보자.

init가 먼저 나타난다

init는 리눅스 시스템의 작동에 있어서 절대적으로 필수적인 프로그램이지만, 그럼에도 흔히 init에 대해 무지한 경우가 많다. 아마 좋은 리눅스 배포본이라면 init가 대부분의 시스템에서 잘 돌아가도록 설정되어 있을 것이고, 따라서 init에 대해선 별로 신경쓸 필요가 없었을 것이다. 다만, 시리얼 터미널들이나 다이얼-인 모뎀(다이얼-아웃은 아님)들을 연결해야하는 경우, 또는 기본 설정된 실행 레벨을 전환시켜야하는 경우에는 init를 주의깊게 살펴보아야 한다.

커널이 자신의 부팅을 진행할 때(즉, 자기 자신을 메모리에 올리고, 그것을 실행시키고, 장치 드라이버들과 데이터 스트럭쳐들을 초기화시키는 등의 일을 진행할 때), 부트 프로세스로서 커널이 마지막으로 해야할 일은 init를 실행시키는 것이다. 즉, 사용자 레벨의 프로세스인 init를 실행시킴으로 해서, 커널은 부트 프로세스로서의 역할을 마치게 된다. 그래서, init는 언제나 첫번째 프로세스가 되는 것이다(따라서, init의 프로세스 번호도 언제나 1이 된다).

커널은 우선 init가 어디 있는지 찾아본다. 역사적으로 init가 있었던 장소는 몇군데 되지만, 리눅스 시스템에서의 적절한 장소는 /sbin/init이다. 만일 커널이 init를 찾지 못한다면, /bin/sh를 실행시키려 하는데, 이마저도 찾지 못한다면 시스템의 시동은 실패하고 만다.

init가 실행되면, 몇가지 관리 작업을 처리하고 부트 프로세스를 마치게 된다. 즉, 파일시스템을 검사하고, /tmp를 청소하며. 그 밖에 다양한 서비스들을 시작시킨다. 또한 getty를 각각의 터미널과 가상 콘솔에 띄워서 사용자들이 로그인 할 수 있도록 한다(Chapter 8 참조).

부팅이 되어 시스템이 정상적으로 가동되면, init는 사용자들이 로그 아웃한 터미널에 다른 사용자들이 다시 로그인 할 수 있도록 getty를 재실행시킨다. 또한 init는 고아 프로세스들을 거두어 양자로 삼는다. 즉, 한 프로세스가 자식 프로세스를 생성하고서 자식 프로세스보다 먼저 죽어 버렸을 경우, 그 자식 프로세스는 즉시 init의 자식 프로세스가 되는 것이다. 이것은 여러가지 기술적 이유로 해서 무척 중요한데, 간단하게 말하자면 모든 프로세스의 리스트와 그 트리 구조를 알기 쉽게 유지하기 위해서라고 할 수 있다. [1] init에는 몇가지 종류가 있는데, 대부분의 리눅스 배포본들은 System V init 디자인에 기반한 sysvinit(Miquel van Smoorenburg이 만듦)를 사용한다. 반면에 BSD 버전의 유닉스들은 다른 init를 사용하는데, 이 둘간의 가장 큰 차이점은 실행 레벨의 유무에 있다. 즉, System V는 실행 레벨이란 개념이 있지만, BSD는 이런 개념이 없다(최소한 전통적으로는 없다). 그러나 이런 차이점은 별로 본질적인 것은 아니다. 여기서는 sysvinit에 대해서만 살펴보기로 하겠다.

Notes
[1] init 자신은 죽음이 허락되지 않는다. 심지어 init에 SIGKILL 시그널을 보낸다하더라도 그것을 죽일 수는 없다.

getty를 실행하기 위한 init 설정: /etc/inittab 파일
시스템이 부팅될 때, init는 /etc/inittab 설정파일을 읽어들이도록 되어 있다. 또한, 시스템이 가동 중일 때도 HUP 시그널을 받으면 이 설정파일을 다시 읽어들인다. [1] 따라서 init의 설정을 변경했다고 해서 그것을 적용시키기 위해 시스템을 리부팅시킬 필요는 없다.

/etc/inittab 파일은 좀 복잡하다. 일단 여기서는 getty에 관한 부분만을 한가지 예로서 살펴보기로 하자. /etc/inittab은 다음과 같이 콜론으로 나눠진 네 부분으로 구성된다. id:runlevels:action:process

이제 각각의 영역을 자세히 살펴보자. 참고로, /etc/inittab은 빈 줄을 포함할 수 있으며, #로 시작하는 줄은 무시된다.

id
이것은 해당 라인을 다른 부분과 식별시켜 준다. getty를 설정하는 부분에서는, 이곳에 터미널 번호를 지정해주도록 되어 있다(이것은 장치 파일 이름에서 /dev/tty뒤에 따라오는 숫자이다). 물론 getty 설정 부분이 아닌 곳에서는 이렇지 않다. id는 길이 제한이 있으며, 파일내에서는 유일한 것이어야 한다.

runlevels
이것은 해당 라인이 어떤 실행 레벨(run level)에서 유효한지를 지정하는 부분이다. 실행 레벨은 한자리 숫자로 표현되며, 특별한 구분기호 없이 연속적으로 여러 실행 레벨을 써넣을 수 있다(실행 레벨은 다음 섹션에서 설명한다).

action
여기서는, 해당 라인이 어떻게 동작해야 하는지를 지정한다 예를 들어, 이곳에 respawn이라고 쓰게 되면 그 다음 영역에 있는 명령이 종료될 때마다 그것을 재실행 하게 된다. once라고 쓰게 되면 실행을 딱 한번만 하게 된다.

process
이곳에 실행시킬 명령이 들어간다.

일반적인 다중 사용자 실행 레벨(multi-user run level, 2번부터 5번까지임)에서 getty를 첫번째 가상 터미널에 띄우고 싶다면, 다음과 같이 해야 한다. 1:2345:respawn:/sbin/getty 9600 tty1

첫번째 부분은 이 라인이 /dev/tty1을 위한 것이라는 점을 알려준다. 두번째 부분은 이것이 실행 레벨 2,3,4,5번에 적용되도록 지정한 것이며, 세번째 부분은 명령의 실행이 종료될 때마다 다시 실행되도록 하는 부분이다(즉, 로그아웃했다가 다시 로그인 , 로그아웃 할 수 있도록). 마지막 부분은 명령으로서, 첫번째 가상 터미널에 getty를 띄우라고 지시하고 있다. [2]

만일 터미널이나 다이얼 인 모뎀 라인을 시스템에 추가하고 싶다면, 그들 각각을 위한 설정 라인을 /etc/inittab에 추가하여야 한다. 이것에 관해 더욱 자세한 내용은 init, inittab, getty의 매뉴얼 페이지를 참고하기 바란다.

어떤 명령이 실행에 실패한다면, init는 그것을 다시 재실행하게 된다. 그러나, 재실행하고 실패하고 다시 재실행하고 실패하고.. 이와 같이 끝없이 반복된다면, 이것은 시스템의 자원을 굉장히 많이 소비하게 된다. 이런 일을 막기위해서 init는 명령이 얼마나 자주 재실행되는지를 점검하고 있다가, 어떤 명령이 지나치게 자주 반복되면 그것을 5분간 다시 실행하지 않는다.

Notes
[1] 예를 들면, 루트로 kill -HUP 1 명령을 내리면 된다.

[2] getty의 버전이 다르면, 그 실행 명령도 다르다. 매뉴얼 페이지를 참조하되, 그것이 getty의 버전에 맞는 것인지를 꼭 확인하기 바란다.

실행 레벨
init는 시스템이 제공할 여러 서비스들을 실행시키는데, 이것을 어떤 수준으로 실행시킬지 등급을 나눠 정의한 것이 실행 레벨(run level) 이라는 개념이다. Table 7-1에 나타낸 바와 같이, 실행 레벨은 숫자로 나타내어 진다. 사용자 정의 실행 레벨(2에서 5까지)에 대해서는 이것을 어떻게 정의할 것인지 합의된 것이 없다. 그래서, 이 부분은 어떤 시스템 구성요소를 사용할 것인지 선택하는데 쓰이기도 한다. 즉, X를 실행시킬 것인지, 네트워크를 작동시킬 것인지 등의 선택을 실행 레벨을 통해 할 수 있다. 그러나 실행 레벨을 통해 시스템을 세부적으로 통제하기란 어려운 일이므로, 실행 레벨에 관계없이 모든 시스템 구성요소들을 개별적으로 실행시키기도 한다. 이중 어떤 방법을 사용한 것인지는 스스로 결정할 문제이지만, 현재 사용중인 리눅스 배포본에서 취하고 있는 방법을 따르는 것이 아마도 가장 손쉬운 방법일 것이다.

Table 7-1. 번호로 나타낸 실행 레벨

0 시스템 종료.
1 단일 사용자 모드 (특별한 시스템 관리 작업용).
2-5 일반적인 시스템 가동 상태(사용자 정의 가능).
6 리부팅.

실행 레벨은 /etc/inittab 파일에서 다음과 같이 설정된다. l2:2:wait:/etc/init.d/rc 2

첫번째 부분은 임의로 붙인 식별용 라벨이다. 두번째 부분은 이것이 실행 레벨 2번에서 적용되도록 지정한다. 세번째 부분은, 실행 레벨에 진입할때 init가 네번째 부분의 명령을 실행하되, 그것이 완료될때까지 기다리라는 뜻이다. 그리고 네번째 부분은 /etc/init.d/rc가 실행 레벨 2번에 해당되는 명령들을 실행하도록 지시하고 있다.

따라서 해당 실행 레벨의 구현에 필요한 모든 일은 네번째 부분의 명령이 담당한다. 실행 레벨이 전환되면, 이 명령은 필요한 모든 서비스들을 시작시키며, 필요없는 서비스들은 종료시킨다. 어느 실행 레벨에서 어떤 명령들이 실행되는지는 리눅스 배포본에 따라 다르다.

시스템이 시작될때, init는 /etc/inittab 파일에서 기본 실행 레벨이 몇번으로 지정되었는지 찾는다. id:2:initdefault:

부팅할 때, 커널 명령행 인자로 single 이나 emergency라고 써넣어 주면, init를 기본 실행 레벨이 아닌 다른 레벨로 실행할 수가 있다. 커널 명령행 인자는 LILO에 의해 전달되며, 위와 같은 인자를 넣어주면 단일 사용자 모드로 진입하게 된다(실행 레벨 1번).

시스템이 가동 중일 때는 telinit 명령으로 실행 레벨을 전환시킬 수가 있다. 이렇게 하면, init는 /etc/inittab에서 그에 해당되는 명령을 찾아 실행시킨다.

/etc/inittab에서의 특수 설정
/etc/inittab에는 init가 특별한 상황에 반응할 수 있도록 해주는 특수 키워드들이 있다. 이 키워드들은 설정 라인의 세번째 부분에 넣어준다. 아래에 몇가지 예를 들었다.

powerwait
정전을 감지하였을 때 어떤 행동을 취할 것인지 지정한다. 보통은 init가 시스템을 자동으로 셧다운하도록 한다. 이렇게 할 수 있으려면, 일단 UPS가 시스템에 장착되어 있어야 하며, UPS에서 정전이 감지되었을 때 이것을 init에게 알려주는 소프트웨어가 작동중이어야 한다.

ctrlaltdel
콘솔 키보드의 ctrl-alt-del 키가 함께 눌려졌을 때의 동작을 지정하는데, 보통은 시스템이 리부팅되도록 한다. 그러나 이때 다른 동작을 하도록 설정할 수도 있다. 만일 시스템이 개방된 장소에 있다면, ctrl-alt-del을 단순히 무시하도록 할 필요가 있을 것이다(혹은 넷핵 - nethack, 오랜 전통의 유닉스 게임 - 이 실행되도록 지정할 수도 있다 -.-).

sysinit
시스템이 부팅될 때 실행할 명령을 지정한다. 예를 들면, /tmp 디렉토리를 청소하도록 하는 경우가 많다.

위에 나열한 것이 전부는 아니다. 사용 가능한 모든 키워드와 그 자세한 사용법을 알고 싶으면 inittab 매뉴얼 페이지를 보기 바란다.

단일 사용자 모드에서의 부팅
단일 사용자 모드(single user mode, 실행 레벨 1)는 중요한 실행 레벨이다. 이 상태에서는 단지 관리자만이 시스템을 사용할 수 있으며, login 같이 시스템 가동에 필수적인 최소한의 서비스만이 실행된다. 단일 사용자 모드는 몇몇 시스템 관리 작업을 하기 위해서 필요한데, [1] 예를 들자면 /usr 파티션에 fsck를 실행시키는 일 같은 것들이다. fsck를 실행시키기 위해서는 해당 파티션을 언마운트시켜야 하는데, /usr 같은 파티션을 언마운트시키자면 거의 모든 시스템 서비스들을 종료시켜야 한다.

가동 중인 시스템을 단일 사용자 모드로 전환하려면, telinit를 사용해 실행 레벨 1로 전환하면 된다. 부팅시에는, 커널 명령행에 single이나 emergency라고 적어주면 커널이 이것을 init에 전달해 주게 되며, init는 이것을 알아듣고 기본 설정된 실행 레벨 대신 레벨 1번을 사용하게 된다. (커널 명령행 인자를 넣는 방법은 시스템을 부팅하는 방법에 따라 좀 다를 수 있다. 보통은 LILO에서 boot: 프롬프트가 떴을 때, "boot:linux single"과 같이 하는 방법을 쓴다.)

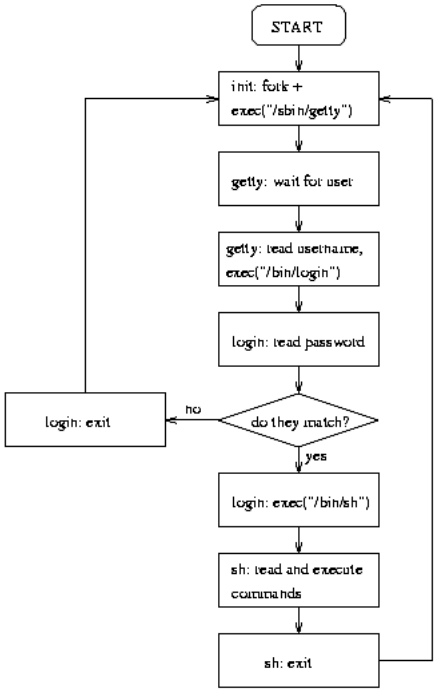
단일 사용자 모드는, 주로 손상된 파일시스템이 마운트되기 전에 fsck 명령을 수동으로 실행하기 위해서 사용된다. 손상된 파일 시스템을 그대로 다시 마운트하면 더욱 큰 손상을 입힐 수 있기 때문에, 손상된 파일시스템은 마운트한다거나 기타 다른 조작을 해선 안되며 가능한 빨리 fsck로 복구를 시도하여야 한다.

손상된 파일시스템이 발견되면 init가 자동으로 fsck를 실행하는데, 이 자동 복구가 실패하게 되면 init 스크립트는 자동으로 시스템을 단일 사용자 모드로 진입시킨다. 이렇게 하면, 손상이 심각하여 fsck가 자동으로 복구할 수 없는 파일시스템이 그대로 마운트되는 일을 막을 수 있다. 물론 이럴 정도로 심하게 손상되는 일은 상당히 드물며, 보통 하드디스크가 손상되었거나 실험적인 커널을 사용했을 경우에 가끔 발생할 수 있는 일이다. 그러나, 이런 사태에 대비하고는 있어야 하겠다.

보안상의 이유로, 제대로 설정되어 있는 시스템이라면 단일 사용자 모드에서 쉘을 실행시키기 전에 루트 패스워드를 물어올 것이다. LILO에서 커널 명령행 인수로 single 을 적어 주는 경우도 이와 같다.(그러나 /etc/passwd가 들어있는 파일시스템이 깨졌다면 단일 사용자 모드로도 들어 올 수가 없다. 결국 이럴 때는 부팅 플로피를 사용해야만 할 것이다)

Notes
[1] nethack을 플레이하기 위해서 단일 사용자 모드를 사용하진 않을 것이다.

Chapter 8. 로그인과 로그아웃
Table of Contents
터미널을 통한 로그인
네트워크를 통한 로그인
login 프로그램이 하는 일
X와 xdm



접근 제어
쉘의 시작
"I don't care to belong to a club that accepts people like me as a member." (Groucho Marx)

"나같은 사람을 멤버로 받아주는 클럽에 몸담고 있지만 뭐 별로 걱정은 안됩니다. " (Groucho Marx: 미국의 코미디언)

여기서는 사용자가 로그인하고 또 로그아웃할 때 어떤 일이 일어나는 지를 살펴보도록 하겠다. 또한, 각종 백그라운드 프로세스들의 다양한 상호 작용과, 로그 파일, 설정 파일 등에 대해서 상세히 알아보게 될 것이다.

터미널을 통한 로그인
Figure 8-1은 터미널을 통한 로그인이 어떻게 이루어지는 지를 보여주고 있다. 우선, init는 getty 프로그램을 각각의 터미널(혹은 콘솔)에 실행시킨다. getty는 터미널에서 로그인하려는 사용자가 있는지 살펴보면서 기다린다(즉, 사용자가 뭔가를 타이핑하지 않는지 살펴본다). 사용자가 있다면, getty는 환영 메시지를 출력하고(이 메시지는 /etc/issue에 들어있다) login: 같은 프롬프트를 띄운 뒤 마지막으로 login 프로그램을 실행시킨다. login 프로그램은 username을 매개변수로 전달받고, 해당 password를 묻기 위해 password: 같은 프롬프트를 띄운다. password가 정확하면, login은 설정되어 있는 쉘을 실행시킨다. password가 틀리다면, login 프로그램은 단순히 종료된다(보통은 몇번 정도 기회를 더 준 뒤에 종료된다). init는 login 프로그램이 종료된 것을 감지하고, 터미널에 새로운 getty를 띄워 놓는다.

Figure 8-1. 터미널을 통한 로그인: init, getty, login, 그리고 쉘의 상호작용.

위에서, 새로운 프로세스는 오직 init에 의해서만 생긴다는 점을 주목하기 바란다(fork 시스템 호출을 사용한다). getty와 login은 단지 실행 중인 프로그램과 교대를 할 뿐이다(exec 시스템 호출을 사용해서).

각각의 시리얼 라인에는 그 라인만을 전담하는 개별적인 getty를 미리 띄워 놓는데, 이렇게 하는 이유는 사용 중인 터미널만 감지해서 getty를 띄우는 일이 좀 복잡하기 때문이다. 또한, 각각의 연결은 그 설정과 속도가 제각각일 수 있기 때문에, getty는 그 각각에 알맞게 적응하도록 되어 있다. 이것은 특히, 각 전화 접속마다 그 설정과 매개변수들이 바뀔 수 있는 다이얼-인 연결인 경우에 중요한 기능이다.

getty와 init에는 여러가지 버전이 있는데, 각각 장단점이 있다. 현재 자신의 시스템에 설치된 버전 뿐만 아니라, 다른 버전들에 대해서도 알아두면 좋을 것이다(이런 것들은 Linux Software Map에서 찾을 수 있다). 만일 시스템에서 다이얼-인 연결 서비스를 제공하지 않을 계획이라면, 아마도 getty에 대해선 별로 신경쓸 필요가 없을 것이다. 그러나 init에 대해서는 언제나 주의를 기울여야 한다.

네트워크를 통한 로그인
같은 네트워크 안에 있는 두 대의 컴퓨터는 물리적인 하나의 케이블로 연결되어 있는 것이 보통이다. 그런데, 이 컴퓨터의 프로그램들이 네트워크를 통해 통신을 한다면, 이 프로그램들도 일종의 가상적인 케이블을 통해 각각 하나씩의 가상 연결(virtual connection)을 이루고 있는 셈이다. 즉, 프로그램들이 가상 연결을 이루고 있는 동안 만큼은, 그들은 자신들의 케이블을 갖고 있기라고 한 것처럼 단순히 동작할 수 있는 것이다. 그러나, 이 케이블은 어디까지나 실제가 아닌 가상의 케이블이므로, 두 컴퓨터의 운영체제는 하나의 물리적인 케이블을 여러 가상 연결들이 나누어 쓸 수 있도록 해주어야 한다. 이렇게 되면, 단지 하나의 케이블을 쓰면서도 많은 프로그램들이 서로 통신을 할 수가 있으며, 다른 프로그램들의 통신 상태에는 신경 쓸 필요가 없다. 더구나 이 방법을 통하면 같은 케이블을 여러대의 컴퓨터가 나누어 쓰는 것도 가능하다. 즉, 케이블 상에 많은 가상 연결이 이미 존재한다하더라도, 자신과 관계없는 것은 그저 무시해버리면 되는 것이다.

실제로 이런 연결은 무척 복잡한 방법을 통해 이루어지며, 위의 내용은 아주 간략화한 설명이다. 그러나 여기서는, 왜 네

트워크를 통한 로그인이 일반적인 로그인과 다른 점이 있을 수 밖에 없는지를 이해하는 정도면 충분하겠다. 가상 연결은 통신하기를 원하는 두 프로그램이 서로 다른 컴퓨터에 있을 때 성립되며, 이것은 다른 컴퓨터에서 네트워크를 통해 로그인하려 하는 경우에도 마찬가지이다. 또한 가상 연결은 동시에 많은 수가 이루어질 수 있으므로, 모든 가능한 login 연결마다 미리 getty를 띄워 놓을 수는 없다.

바로 이런 문제에 대처하기 위해서, 모든 네트워크 로그인을 다룰 수 있는 inetd(getty에 상응하는 것이다)라는 단일 프로세스가 있다. 네트워크 로그인 요청이 하나 들어올 때마다, inetd는 그에 대응할 프로세스를 역시 하나씩 새로 실행시킨다(즉, inetd는 다른 컴퓨터로부터 가상 연결을 통한 로그인 요청이 들어오는지 항상 감지하고 있다). 그리고 원래의 inetd 프로세스는 그대로 남아 다시 새로운 로그인 시도가 있는지 살펴보고 있게 된다.

그런데 네트워크 로그인에 쓰이는 통신 프로토콜은 한가지 뿐이 아니어서 일이 좀더 복잡하게 되는데, 그 중에 가장 많이 쓰이는 것은 telnet과 rlogin이다. 또한 네트워크 로그인 이외에도 다른 많은 가상 연결들이 있다(FTP, Gopher, HTTP 기타 등등). 이런 많은 연결들에 대응하기 위해 모두 각각의 프로세스를 띄운다면 그것은 매우 비효율적인 일이 될 것이다. 그래서, 연결이 어떤 종류의 연결인지를 파악하여 그에 해당되는 서비스를 제공하는데 알맞는 프로그램을 시작시켜주는 단 하나의 프로세스를 사용하게 되는데, 이것이 바로 inetd이다. 좀더 자세한 내용은 `Linux Network Administrators ' Guide를 참고하기 바란다.

login 프로그램이 하는 일
login 프로그램은 우선 사용자를 인증하며(즉, username과 password가 맞는지 확인한다), 시리얼 라인에 퍼미션을 주고 쉘을 시작시켜 사용자의 초기 환경을 만들어 준다.

또한 초기 설정의 일부로서, /etc/motd(`오늘의 메시지' 같은 짧은 정보를 넣어둔다)의 내용을 화면에 뿌려주며 또한 전자 우편이 도착하였는지를 확인시켜준다. 만일 이런 것들을 보고 싶지 않다면, 사용자의 홈디렉토리에 .hushlogin 파일을 만들어 두면 된다.

그런데 만일 /etc/nologin 파일이 있다면, 로그인이 아예 봉쇄된다. 이 파일은 shutdown과 같은 명령이 주로 만드는데, login은 이 파일이 있는지 검사해서 만일 있다면 그 내용을 화면에 뿌려주고 로그인은 받아들이지 않는다.

login은 모든 실패한 로그인에 대한 기록을 시스템 로그 파일에 기록하여 둔다(이 일은 syslog를 통해서 이루어진다).

현재 로그인해 있는 사용자는 /var/run/utmp에 나열되어 있다. 이 내용은 시스템이 부팅될 때 지워지므로, 단지 시스템이 가동 중일 때만 유효하다. 이 파일에는 현재 로그인한 사용자의 이름과 사용중인 터미널 등의 정보가 수록되어 있는데, who나 w 같은 명령들이 바로 이 utmp 파일을 들여다 보고 누가 로그인해 있는지 알아낸다.

모든 성공적인 로그인은 /var/log/wtmp에 기록된다. 이 파일은 끝없이 크기가 커지므로 주기적으로 그 내용을 지워 주어야 하는데, 예를 들면 cron을 사용해서 일주일에 한번 정도 지워주는 것이 좋다. [1] wtmp 파일의 내용은 last 명령을 사용해 살펴볼 수 있다.

utmp와 wtmp는 모두 바이너리 파일이므로(utmp 매뉴얼 페이지 참조), 이 파일의 내용을 살펴 보려면 위와 같이 알맞은 프로그램을 사용하여야만 한다.

Notes
[1] 좋은 리눅스 배포본이라면 이미 이렇게 되어 있을 것이다.

X와 xdm
xdm을 사용하면 곧바로 X을 띄운 상태에서 로그인을 할 수가 있다. xterm -ls 명령도 이와 같은 일을 해준다.

접근 제어
사용자들에 대한 데이터베이스는 전통적으로 /etc/passwd 파일에 담겨 있다. 그러나 어떤 시스템들은 섀도우 패스워드(shadow password)를 사용하며, 이런 경우에는 패스워드들을 /etc/shadow 파일에 따로 담아놓는다. 많은 컴퓨터들이 함께 돌아가는 큰 사이트에서는 사용자 데이터베이스를 관리하기 위해 NIS 같은 기술을 쓰는데, 이를 통하면 사용자들의 계정

정보를 공유할 수가 있다. 즉, 하나의 중앙 컴퓨터에서 다른 컴퓨터들에게 데이터베이스 정보를 제공해 주도록 되어 있다.

사용자 데이터베이스에는 단지 패스워드만이 들어 있는 것이 아니다. 이곳에는 사용자들의 실제 이름, 홈 디렉토리의 위치, 로그인때 실행시킬 쉘 등의 정보가 담겨 있다. 이런 정보들은 공개되어 있는 것으로서 누구나 이 정보들을 읽을 수가 있지만, 패스워드는 그 자체가 암호화(encrypt)되어 있으므로 단순히 읽어보는 것만으로는 원래 패스워드를 알아낼 수 없도록 되어있다. 그러나 암호화된 패스워드를 이렇게 아무나 읽어볼 수 있다면, 각종 암호 해독 방법을 동원하여 원래 암호를 알아내는 것이 가능해지며, 더구나 이런 방법을 통하면 추측한 암호가 맞는지 확인하기 위하여 시스템에 직접 로그인해 볼 필요도 없어진다. 섀도우 패스워드는 바로 이런 문제를 피해가기 위해 고안된 것으로서, 루트만이 읽을 수 있는 파일에 패스워드를 따로 보관해 두는 방식을 사용한다(역시 암호화된 형태로 저장된다). 다만 한가지 걸림돌은, 일반 패스워드로 설치한 시스템을 다시 섀도우 패스워드 시스템으로 전환하는 일이 무척 어렵다는 점이다.(그러나 요즘 배포본들은 PAM이란 기술을 사용하고 있어서 비교적 손쉽게 이런 전환을 할 수 있다.)

섀도우 패스워드를 사용하건 하지 않건 간에, 시스템의 모든 패스워드들을 추측하기 힘든 형태로 유지하는 것은 아주 중요한 일이다. crack이란 프로그램은 패스워드를 알아내기 위해 사용되는 프로그램인데, 이런 프로그램에 의해 추측되어질 수 있는 패스워드는 모두 좋지않은 패스워드로 간주하면 된다. 즉, 이 프로그램은 시스템을 뚫고 들어오려는 침입자들에 의해서도 사용되지만, 이것을 역이용하면, 반대로 나쁜 패스워드를 가려내는 데 유용하게 쓰일 수 있다. 이것을 이용해서, passwd 프로그램은 사용자의 패스워드를 입력받을 때 그것이 나쁜 패스워드로 인식되면 다른 패스워드를 사용하도록 요구할 수 있다. 원래의 패스워드 crack 프로그램은 굉장히 많은 연산을 요구하는데 비해, passwd가 나쁜 패스워드를 가려내는 연산은 아주 효율적이어서 시스템에 무리를 주지 않는다.

사용자 그룹에 대한 데이터베이스는 /etc/group에 저장된다. 만일 섀도우 패스워드 시스템이라면 /etc/shadow.group이 된다.

보통 루트 사용자는 네트워크를 통해 로그인 할 수 없으며, 단지 /etc/securetty 파일에 나열된 터미널을 통해서만 로그인 할 수 있다. 따라서 루트로 직접 로그인하려는 사용자는 위 파일에 나열된 터미널 중의 하나에 물리적으로 접근할 수 있어야 한다. 다만 그 밖의 터미널에서도, su 명령을 사용한다면 루트 권한을 획득할 수 있긴 하다.

쉘의 시작
로그인 쉘이 실행될 때, 쉘은 미리 설정된 파일들을 자동적으로 실행시킨다. 쉘이 달라지면 실행시키는 파일의 종류도 달라지므로, 각각의 쉘에 대한 자세한 내용은 해당 문서들을 참고하기 바란다.

대부분의 쉘들은 모든 사용자에게 공통적으로 적용되는 파일을 갖고 있는데, 예를 들어 본 쉘(Bourne shell, /bin/sh)과 이 본 쉘에서 파생된 쉘들은 우선 /etc/profile을 공통적으로 실행시킨 후, 사용자의 홈디렉토리에 있는 .profile을 덧붙여 실행시킨다. /etc/profile에서는 시스템관리자가 각 사용자들에게 공통적으로 적용시키고 싶은 환경 설정이 들어가는데, 특히 명령들의 경로명 같은 것이 일반적인 경우와 좀 다를 때 그것을 지정해 주는 경우가 많다. 반면에, .profile은 각 사용자들이 기본 설정 대신에 자신의 환경을 스스로 설정할 필요가 있을 때 사용하는 파일로서, /etc/profile과 중복되는 내용이 있을 경우 .profile의 내용이 적용된다.

Chapter 9. 사용자 계정의 관리

"The similarities of sysadmins and drug dealers: both measure stuff in K's, and both have users." (Old, tired computer joke.)
"시스템관리자와 약종상의 같은 점: 둘다 모든 것을 K 단위로 재고, 둘다 사용자가 있다는 점. " (컴퓨터에 대한 재미없는 옛날 농담.)

이 장에서는 새로운 사용자 계정을 어떻게 만들고, 그 계정의 속성을 어떻게 변경시키며, 또 어떻게 계정을 지우는가를 설명할 것이다. 각각의 리눅스 시스템은 이러한 일을 하는 데 각기 다른 도구들을 가지고 있다.

계정이란 무엇인가?
다수의 사람들이 한 컴퓨터를 이용할 때에는 각각의 사용자들의 프라이버시(ex.개인 파일들)를 지켜주기 위해 사용자들을

구분시켜주는 것이 필수적이다. 이는 집에서 자신의 컴퓨터를 혼자 사용하더라도 간과할 수 없는 사실이다. [1] 그래서 각각의 사용자에겐 자신만이 접속할 수 있는 사용자이름(username)이 주어진다.

그러나 사용자에겐 단지 username 말고도 주어지는 것이 있다. 계정(account)이란 한 사용자에게 소유되는 모든 파일과 자원, 그리고 정보인 것이다. `계정'이란 은행, 그리고 각각의 계정이 그에 수반되는 돈을 가지며 그 돈이 사용자가 제도에 얼마만큼 영향을 미치느냐에 따라 소비되는 속도가 결정되는 '상업제도'에서 힌트를 얻은 용어이다. 예를 들어 디스크 공간은 용량과 기간에 따라 가치를 가지며, 처리시간도 초마다 가치가 달라질 것이다.

Notes
[1] 만약 당신의 동생이 당신의 연애 편지를 본다면 얼마나 당혹스럽겠는가...-.-

계정 만들기
리눅스 커널은 사용자들을 단순히 숫자로만 다룬다. 각각의 사용자는 독특한 정수로 된 user id 혹은 uid로 구별되는데, 그 이유는 문자로 된 이름보다 컴퓨터가 접근하는데 쉽기 때문이다. 커널 밖의 별도의 데이터베이스는 username을 각각의 user id에 할당한다. 이 데이터베이스는 역시 추가적인 정보를 담고 있다.

계정을 만들기 위해서는 그 사용자의 데이터베이스에 사용자에 관한 정보를 추가시키고 사용자를 위한 home디렉토리를 만들어야 한다. 사용자를 교육시키고 그 사용자에 적합한 초기 환경을 만들어 주는 것 또한 필수적이다.

대부분의 리눅스 배포본은 새로운 계정을 만드는 프로그램을 통해 사용자를 추가시킨다. 거기에는 여러 가능한 프로그램이 있는데 adduser와 useradd 라는 명령 중 택일하면 된다. 또한 GUI 방식으로 사용할 수 있는 도구도 있다. 이런 작업이 이루어지는 세부과정은 좀 까다롭지만, 이런 프로그램들은 모두 일을 자동으로 처리해 준다. 계정을 직접 수동으로 추가하는 방법에 대해서는 the section called 수동으로 계정 만들기에서 설명하겠다.

/etc/passwd와 이외의 정보 파일
유닉스 시스템에 있는 기본적인 사용자 데이터베이스는 /etc/passwd이며(password file이라고도 부른다), 여기에는 모든 유효한 username과 그들과 관련된 정보가 나열되어 있다. 사용자의 정보는 한줄로 이루어져 있으며, 이것은 다시 콜론으로 구분된 7개의 영역으로 나누어진다.

Username.

Password, 암호화되어 있다.

숫자로된 user id.

숫자로된 group id.

사용자의 실제이름이나 계정에 관한 기타 설명.

홈 디렉토리의 위치.

로그인 쉘(또는 로그인시 실행할 프로그램).

이 파일의 형식은 passwd 매뉴얼 페이지에 보다 자세히 설명되어 있다.

어떤 사용자라도 시스템상에서 패스워드 파일을 읽을 수 있을 것이므로, 예를 들어 다른 계정의 이름을 알 수 있다. 이것은 두 번째 필드에 있는 패스워드가 모두에게 이용 가능하다는 것을 의미한다. 패스워드 파일은 패스워드를 암호화하여, 이론적으론 아무런 문제가 없다. 그러나 암호는 깨질 수가 있다. 특히 짧거나 사전에서 찾을 수 있는 쉬운 단어로 된 것이라면 쉽게 깨질 수 있다. 그래서 패스워드 파일에 암호를 가지는 것은 별로 좋은 방법이 아니다.

대부분의 리눅스 시스템은 shadow passwords를 가지고 있다. 이것은 패스워드를 저장하는 다른 방법이다. 암호화된 패스워드는 root만이 읽을 수 있는 /etc/shadow라는 분리된 파일에 저장된다. /etc/passwd 파일은 단지 두 번째 필드에 특별한 표시를 포함하고 있다. 어떤 프로그램은 사용자가 setuid 되었다는 것을 증명할 필요가 있으며 그렇게 함으로써 shadow password 파일에 접근할 수 있다. 패스워드 파일 내의 다른 필드만을 이용하는 보통의 프로그램들은 암호를 얻을 수 없다. [1]

사용자와 그룹 아이디 번호 골라내기
대부분의 시스템에서는 사용자와 그룹 아이디 번호가 무엇인지 중요하지 않다. 그러나 네트워크 파일시스템(NFS)을 사용할 경우 모든 시스템상에서 같은 uid와 gid를 가질 필요가 있다. 이것은 NFS 역시 사용자를 uid 번호와 동일하게 간주하기 때문이다. 만약 NFS를 사용하지 않는다면 당신은 당신의 계정 만들기 도구가 자동적으로 그것들을 고르도록 만들어도 된다.

만약 NFS를 사용한다면 당신은 계정의 정보를 동시성을 가지도록하는 메카니즘을 개발해야 한다. 다른 대안은 NIS 시스템이다. (Olaf kirch의 Linux network administrators' guide를 보라.)

하지만 당신은 uid 번호(그리고 문자로 된 usernames)를 다시 사용하는 것을 피해야 할 것이다. 왜냐하면 uid나 username의 새로운 소유자가 기존의 소유자 파일(혹은 메일, 혹은 무엇이든)에 접근할 지도 모르기 때문이다.

초기 환경: /etc/skel
home 디렉토리에 새 계정이 만들어지면 /etc/skel 디렉토리로부터 파일이 초기화된다. 시스템 운영자는 /etc/skel에서 파일을 만들 수 있으며 그것은 사용자를 위한 멋진 기초 환경을 제공할 것이다. 예를 들어 /etc/skel/.profile을 만들어 에디터 환경을 몇몇 에디터에게 변할 수 있게 설정하여 새 사용자에게 친숙하게 할 수 있다.

그러나 /etc/skel 을 가능한한 작게 유지하는 것이 좋다. 왜냐하면 현존하는 계정들의 파일을 업데이트하는 것이 거의 불가능할 것이기 때문이다. 예를 들어 친숙한 에디터의 이름이 바뀐다면, 모든 현존하는 사용자들은 그들의 .profile을 편집해야 할 것이다. 시스템 운영자는 스크립트를 이용하여 그 과정을 자동적으로 처리하도록 만들 수 있지만, 누군가의 파일에 손상이 가는 것이 불가피할 것이다.

언제라도 가능하면 전체적인 설정은 /etc/profile 같은 전체 파일에 두는 것이 낫다. 이런 방법으로 사용자 자신의 설정을 손상시키는 일 없이 업데이트를 가능케 할 수 있다.

수동으로 계정 만들기
새 계정을 수동적으로 만드려면 다음의 과정을 밟으면 된다 :

/etc/passwd를 vipw로 편집하고 새 계정을 위한 뉴 라인을 추가한다. 형식에 주의하라. 에디터로 직접 편집하지 마라! vipw는 파일을 잠궈서 다른 명령이 동시에 그것을 업데이트하려 하지 않을 것이다. 로그인이 불가능하도록 패스워드 필드를 `*'로 채워야 한다.

역시 새 그룹을 만드려고 한다면, 같은 방법으로 vigr를 이용하여 /etc/group를 편집한다.

mkdir 명령으로 home디렉토리에 계정을 추가한다.

/etc/skel에서 새 home디렉토리에 파일을 카피한다.

소유권한과 접근권한을 chown과 chmod로 수정한다. -R 옵션은 가장 유용하다. 정확한 접근 권한은 한 사이트에서 다른 사이트까지 바뀌나, 보통 다음의 명령은 맞는 명령이다. cd /home/newusername
chown -R username.group .
chmod -R go=u,go-w .
chmod go= .

passwd 명령으로 암호를 설정하라.

마지막으로 암호를 정한 후에 계정은 작동할 것이다. 당신은 다른 모든 과정이 다 수행될 때까지 암호를 설정해서는 안된다. 그렇지 않으면 당신이 파일을 카피하는 동안에도 의도하지 않게 계정은 로그인 될 지도 모른다.

때때로 사람이 사용하지 않는 모조 계정 [2] 을 만들어야 할 때가 있다. 예를 들어 anonymous FTP 서버를 설정하기 위해서는(계정 없이도 누구나 자료를 다운받을 수 있도록), ftp라는 계정을 만들 필요가 있다. 이런 경우, 대개 위의 마지막 과정인 암호 설정을 할 필요가 없다. root는 어떤 사용자도 될 수 있으므로 아무나 root가 되지 않는 한, 그들이 계정을 사용할 수 없도록 암호를 설정하지 않는 것이 좋다.

Notes
[1] 이는 패스워드 파일은 암호를 제외한 모든 정보를 가지고 있다는 의미다.

[2] 비현실적인 사용자라고 할 수 있다.

계정 속성 바꾸기
계정의 다양한 속성을 바꾸는 몇몇 명령어들이 있다. (즉,/etc/passwd의 영역과 관련하여)

chfn
필드의 전체 이름을 바꾼다.

chsh
로그인 쉘을 바꾼다.

passwd
패스워드를 바꾼다.

수퍼유저는 이러한 명령어를 사용해 어떤 계정의 속성이든 바꿀 수 있다. 일반 사용자들은 단지 자신의 계정 속성만 바꿀 수 있다. 가끔 일반 사용자들이 chmod 같은 명령을 사용하지 못하게 할 필요가 있다. 가령 많은 초보 사용자들이 사용하는 환경에선 그렇게 해야 할 것이다.

다른 작업들은 직접 해야 한다. 예를 들어 username을 바꾸려면 직접 /etc/passwd를 vipw(기억해두라)를 이용해 편집하면 된다. group에 user를 추가시키기나 삭제할 때도 유사한 방법으로 vigr을 이용해 /etc/group을 편집하면 된다. 그러나 이러한 작업들은 드문 경우이며 주의해서 해야 한다. 가령 username을 바꾸고 mail을 alias 해 놓지 않으면 e-mail이 올 수 없을 것이다. [1]

Notes
[1] 예를 들자면 외국의 경우, 결혼으로 인해 이름이 바뀔 수도 있다. 이럴 때는 username과 새 이름이 서로 연관성을 갖도록 하고 싶을 것이다.

계정 삭제하기
계정을 삭제하기 위해서는 먼저, 계정 내의 모든 파일들, 우편함, mail aliases, print 작업들, cron과 at 작업들 그리고 그 계정과 관련된 모든 작업들을 제거한다. 그리고나서 /etc/passwd와 /etc/group로부터 관련된 라인을 지운다. (username을 추가된 모든 group으로부터 지우는 것을 잊지 말라.) 내용물 제거를 시작하기 전에 계정을 사용하지 못하도록(아래에 있다.) 조치해 두는 것이 좋다. 그렇게 함으로써 계정을 삭제하는 동안 사용자가 계정을 이용하는 것을 막을 수가 있다.

사용자가 자신의 홈 디렉토리 외부에 파일을 가지고 있을 수도 있다는 것을 염두하라. find 명령어로 그것들을 찾을 수 있다. find / -user username

그러나 용량이 클 경우 위 명령은 오랜 시간이 걸린다는 것을 알아두라. 만약 네트워크 디스크들을 마운트한다면 네트워크나 서버를 못 쓰게 하지 않도록 주의해야 할 것이다.

몇몇 리눅스 배포본은 이를 수행하기 위해 특별한 명령어를 가지고 있다. (deluser 혹은 userdel을 찾아라.) 그러나 명령어는 모든 것을 처리해주지 않으며, 직접 하기에도 쉽다.

일시적으로 계정 사용 금지하기
때때로 계정을 제거하지 않고 일시적으로 사용하지 못하도록 하는 것이 필요할 때도 있다. 예를 들어 사용자가 사용료를 지불하지 않았거나 시스템 운영자가 보기에 크래커가 계정의 암호를 가지고 있다고 의심이 드는 경우, 그런 조치가 필요하다.

계정을 사용하지 못하게 하는 가장 좋은 방법은 쉘을 특별한 프로그램으로 바꾸어 메시지만 출력하도록 하는 것이다. 이 방법으로 그 계정에 접속하려는 사람이라면 누구나 접속에 실패할 것이며 그 이유를 알 수 있게 될 것이다. 메시지로 사용자로 하여금 시스템 운영자에게 연락해 문제를 다루도록 알려줄 수 있다.

username이나 password를 다른 것으로 바꾸는 것 역시 가능하다. 하지만 그러면 사용자는 무슨 일인지 알 수가 없을 것이다. 당황한 사용자는 다른 방법을 계속 시도해 볼 것이다. [1]

위에서 말한 특별한 프로그램을 만드는 간단한 방법은 'tail scripts'를 짜는 것이다. : #!/usr/bin/tail +2
This account has been closed due to a security breach.
Please call 555-1234 and wait for the men in black to arrive.

첫 번째 두 문자(`#!')는 커널에게 나머지 라인이 이 파일을 번역하기 위해 필요한 명령임을 말해준다. 이 경우 tail 명령은 첫 줄만 제외한 나머지 모든 라인을 표준 출력으로 출력하게 된다.

billg가 보안 위반으로 의심된다면, 시스템 운영자는 다음과 같이 할 것이다. : # chsh -s /usr/local/lib/no-login/security billg
su - tester
This account has been closed due to a security breach.
Please call 555-1234 and wait for the men in black to arrive.
#

su의 용도는 물론 바뀐 것이 작동되는지 테스트하는 것이다.

Tail scripts는 그들의 이름이 일반 유저 명령으로부터 간섭받지 않도록 별도의 디렉토리에 두어야 한다.

Notes
[1] 만약 당신이 BOFH(The Bastard Operator From Hell, 엉터리 시스템 관리자를 비꼬는 우스개 소리)라면 혹시나 사용자들이 즐거워 할지도 모르겠다..

Chapter 10. 백업
Table of Contents
지속적인 백업의 중요성에 대해서
백업 매체 선택하기
백업 툴 선택하기
단순 백업
다단계 백업
무엇을 백업해야 할 것인가
압축을 사용한 백업

Hardware is indeterministically reliable.
Software is deterministically unreliable.
People are indeterministically unreliable.
Nature is deterministically reliable.

하드웨어는 조금은 믿을 수가 있다.
소프트웨어를 정말로 믿을 수는 없다.
사람은 조금도 믿을 수가 없다.
자연은 정말로 믿을 수가 있다.

여기서는 백업을 왜, 어떻게, 언제 하여야 하는지에 관해 알아볼 것이다. 그리고 백업을 받아둔 뒤 다시 복구하는 방법에 대해서도 알아보기로 한다.

지속적인 백업의 중요성에 대해서
여러분의 데이터는 가치있는 것이다. 잃어버린 데이터를 다시 살리기 위해서는 노력과 시간, 혹은 돈을 들여야 할 것이다. 만일 그렇지 않더라도 최소한 개인적인 슬픔과 눈물이 뒤따를 것이다. 어떤 경우에, 잃어버린 데이터는 영영 복구 불가능한 것일 수도 있다. 특히 어떤 실험의 결과라면 더욱 그럴 수 있다. 무엇이든지 노력이 들어간 그 순간부터, 여러분은 그 것을 잃지 않도록 준비를 해야만 한다.

기본적으로, 데이터를 잃어버리게 되는 네가지 요인이 있다 : 하드웨어의 망가짐, 소프트웨어의 버그, 사람의 실수(혹은 고의), 그리고 자연 재해로 인한 경우이다. [1] 요즘 하드웨어들은 신뢰도가 높긴 하지만 자연적으로 망가질 수 있다는 점은 예전과 마찬가지다. 중요한 데이터가 보관되는 하드웨어 중 가장 핵심적인 것은 하드디스크일 것이다. 그렇지만 하드디스크는 전자기적 노이즈로 가득한 이 세상에서 홀로 자신을 지키고 있는 불안한 장치이다. 또한 소프트웨어도 별로 믿을 만한 것이 못되어서, 신뢰도 높은 견고한 소프트웨어라는 것은 없다고 보면 된다. 더구나 사람은 정말로 믿어선 안되는 존재이며 언제나 실수를 저지르게 마련인데다가, 그 중에는 아주 고의로 데이터를 망쳐 놓으려는 악질들도 있다는 점을 명심해야 한다. 대자연은 최소한 우리에게 악의를 품고 있지는 않다. 그러나 언제 갑자기 우리에게 재앙을 가져다 줄지 알 수 없는 존재이다. - 이러한 모든 악조건 하에서도 여러분의 시스템이 잘 돌아가고 있다면, 그것은 아마 작은 기적이라고 불러야 할 것이다.

백업이란 것은 데이터가 지닌 가치를 보전하는 작업이다. 데이터를 여러개 복사해 둔다면, 그 중에 하나가 망가지더라도 별 문제가 되지 않을 것이다(단지 백업해둔 복사물로부터 데이터를 복구하는 비용만 들이면 될 것이다).

백업을 평소에 철저히 해두는 것은 무척 중요하다. 그러나 현실의 모든 일이 그러하듯이, 백업 작업 자체도 언젠가는 실패할 수 있다. 백업을 제대로 해내기 위한 방편 중 하나는, 모든 일에 철저를 기하는 것이다; 그렇게 하지 않는다면, 언젠가 여러분의 백업이 더 이상 제 역할을 하지 못하는 심각한 사태에 직면하게 될 것이다. [2] 만일을 위해 심각한 사태의 예를 들어보자. 극단적인 경우, 여러분이 백업을 받고 있는 그 시점에 시스템이 크래쉬 되어버릴 수도 있다; 이렇게 되면 저장되고 있던 백업 데이터들도 손상을 입게 되고, 그 밖에 따로 백업을 받아 둔 것이 없다면.. 여러분은 고된 노동의 흙먼지 속에 버려진 비참한 신세가 될 것이다. [3] 또한 무척 중요한 데이터(1,500명 분의 데이터베이스 같은 것)가 미처 백업되지 못했다는 사실이 복구 도중에야 밝혀진다면, 이 역시 엄청난 비극이 될 것이다. 다행히 이런 일들이 일어나지 않고 백업이 잘 이루어졌다고 해도, 백업테이프를 어제까진 잘 읽던 드라이브 하나가 오늘은 습기에 가득 젖어 있다면..

백업에 관해서라면, 병적인 강박증은 담당자의 필수 자격조건이라고 말할 수 있다.

Notes
[1] 다섯번째 요인은 ``그 밖의 모든 사고''라고 할 수 있겠다.

[2] 그저 웃어넘길 일이 아니다. 언젠가 당신에게 이런 일이 닥칠 것이다.

[3] 이런 상황에 처했다면, 이렇게 될 수 밖에...

백업 매체 선택하기
백업에 있어서 가장 중요한 결정은 어떤 백업 매체를 사용할 것인지를 선택하는 일이라고 할 수 있다. 여기에 고려할 사항으로서는, 비용, 신뢰성, 속도, 사용가능성, 그리고 편리성이 있다.

백업 매체의 용량은 백업할 데이터의 몇배 이상이 되어야 하므로, 그 비용(cost)은 중요한 고려 사항이 된다. 즉, 값싼 매체의 선택이 보통 필수적이다.

신뢰성은 아주 중요하게 고려되어야 한다. 망가져버린 백업 매체 앞에서는 다 큰 어른 들도 엉엉 울 수 밖에 없을 것이기 때문이다. 백업 매체라면 최소한 몇년 정도는 데이터를 보존할 수 있어야 한다. 다만, 백업 매체를 어떻게 사용하느냐에 따라 수명은 좀 달라질 수 있을 것이다. 하드디스크는 보통 신뢰성이 높다고 알려져 있지만, 만일 같은 컴퓨터 안의 하드디스크로 백업을 하는 경우라면 백업 매체로서의 신뢰성이 그다지 높다고 말할 수 없다.

백업이 사람의 간섭없이도 자동적으로 진행될 수 있다면, 속도는 그다지 문제시 되지 않는다. 자동적으로 진행되는 백업이라면 그것이 두시간 쯤 걸린다고 해서 크게 문제되진 않을 것이다. 다만 컴퓨터가 언제나 바쁜 상태여서 오랜 시간 동안 백업을 돌릴만한 여유가 없다면, 속도 문제도 고려해 보아야 한다.

사용가능성은 상당히 중요한 문제이다. 왜냐하면 존재하지도 않는 매체로 백업을 할 수는 없는 일이기 때문이다. 또한 그 매체를 미래에도 계속 쓸 수 있을지, 또한 다른 종류의 컴퓨터에도 사용가능할지 등을 고려해 보아야 한다. 이런 배려를 미리 해 두지 않는다면, 언젠가 재앙이 닥친 후에 복구마저도 할 수 없는 불행한 사태에 직면하게 될 것이다.

편리성은 백업을 얼마나 자주하느냐에 그 중요도가 달려 있다. 즉, 백업 작업을 좀더 쉽게 할 수 있을 수록 좋은 것이다. 백업하기가 지겨울 정도로 쓰기 불편한 매체여서는 곤란하다.

전형적인 두가지 백업 매체로서 플로피와 테이프가 있다. 플로피 디스켓은 아주 값싸고 상당히 신뢰성이 좋으며 사용가능성도 높지만, 다만 좀 느리고 많은 양의 데이터를 백업하기에는 적당하지 못하다. 테이프는 값이 적당하고 상당히 신뢰성이 좋으며 속도도 상당히 빠르면서 사용가능성도 높은데다가, 편리하기까지 하다(편리성은 테이프의 크기에 따라 좀 다를 수 있다).

그 밖에도 몇가지 다른 대안들이 있을 수 있다. 보통 이런 것들은 그다지 사용가능성이 높지 않지만, 어떤 경우에는 좀더 나은 성능을 발휘할 수도 있다. 예를 들어 광자기 디스크는 플로피(랜덤 억세스 능력과 간단한 파일의 신속한 복구 능력)와 테이프(많은 양의 데이터 저장 능력)의 좋은 측면을 모두 가지고 있는 백업 매체이다.

백업 툴 선택하기
백업에 사용되는 툴들은 그 종류가 무척 다양하다. 백업에 사용되는 전통적인 유닉스 툴로서는 tar, cpio 그리고 dump가 있으며 그 밖에도 많은 외부 업체들이 만든 패키지(third party package : 프리웨어이거나 상용일 수 있다)들이 사용될 수 있다. 이 중에 어떤 툴을 쓸 것이냐하는 것은 백업 매체의 종류에 상당 부분 영향을 받는다.

tar와 cpio는 백업 툴로서는 상당히 유사한 점이 많다. 둘 다 테이프를 사용하는 백업과 복구에 적합하지만, 그 밖에 다양한 백업 매체에서도 사용할 수 있다. 이것이 가능한 이유는, 사용자 레벨의 프로그램들이 커널의 장치 드라이버를 통해 다양한 하드웨어를 일관적으로 다룰 수 있기 때문이다. tar와 cpio의 어떤 유닉스 버전들은 심볼릭 링크나 장치 파일, 아주 긴 이름 파일 등 특별한 파일들을 제대로 다루지 못하는 경우가 있는데, 리눅스 버전이라면 모든 파일을 제대로 인식하므로 걱정할 필요가 없다.

dump는 파일시스템 서비스를 사용하지 않고 파일시스템 자체를 직접 읽어낸다는 점에서 상당히 특별하다. 더구나 dump는 특별히 백업만을 위해서 만들어진 프로그램이다. 그 반면, tar와 cpio는 백업도 잘해내지만 원래는 파일을 한데 묶어내기(archiving) 위한 프로그램이었다.

파일시스템 자체를 직접 읽어내는 방법에는 상당한 잇점이 있다. 이 방법을 쓰면 파일에 손댄 시각(time stamp)을 변경시키지 않고서도 백업을 할 수 있다. 반면에 tar와 cpio는 반드시 파일시스템을 읽기 전용으로 마운트하고 나서야 백업을 할 수 있다. 또한 cpio는 디스크 헤드에 부하를 적게 주기 때문에, 많은 양의 백업을 하여야 할 때 좀더 효율적이다. 그러나 이 방식의 단점은, 한가지 종류의 파일시스템만 다룰 수 있다는 점이다. 즉, 리눅스용 dump 프로그램은 ext2 파일시스템에만 사용할 수 있다.

또한 dump는 우리가 곧 논의할 다단계 백업 레벨(backup level)을 직접 지원해 준다. 그 반면에 tar와 cpio는 다른 툴을 통해서만 이 기능을 구현할 수 있다.

그 밖의 다른 외부 업체들이 만든 백업 툴들은 여기서 다루지 않겠다. 기타 프리웨어들에 대한 정보는 Linux Software Map을 참고하기 바란다.

단순 백업

단순 백업 방식이라는 것은, 먼저 모든 것을 한꺼번에 백업하고 그 다음부터는 앞선 백업에서 변경된 부분만을 골라 백업하는 것을 말한다. 여기서 맨 처음 하는 백업을 ful backup(완전 백업)이라고 하며, 그 다음부터는 incremental backups(변경분 백업) 이라고 한다. 보통 풀 백업은 양이 많기 때문에, 여러장의 플로피와 테이프를 사용해야하는 고된 작업이 된다. 반면에, 풀 백업을 해두면 복원하기는 변경분 백업보다 훨씬 쉽다. 풀 백업 이후에도 언제나 모든 것을 백업해 두도록 하면 복원 작업은 좀 더 효율적일 수 있을 것이다. 다만 이렇게 하면 일이 좀 많아지는데, 물론 풀 백업과 변경분 백업을 사용해서 복원할 때의 작업량보다도 더 과중한 작업을 하면서까지 이렇게 할 필요는 없다.

만일 테이프 6개로 매일 백업을 하고 싶다면, 하루(금요일 같은 날에)는 1번 테이프로 풀 백업을 하고 2-5번 테이프로는 변경분 백업(월요일에서 목요일까지)을 하는 방법을 생각해 볼 수 있다(토요일과 일요일은 쉰다). 그리고 그 다음주 금요일에는 6번 테이프에 새로 풀 백업을 받고, 역시 2-5번 테이프로 변경분 백업을 받도록 한다. 6번 테이프에 새로 풀 백업을 받았다하더라도 1번 테이프의 풀 백업을 지워서는 안되며, 1번 테이프는 멀찌감치 다른 장소에 잘 보관해 두도록 한다. 이렇게 해두면, 불이 나서 다른 테이프가 다 타버린다고 해도 1번 테이프로 뭔가 복구를 시도해 볼 수 있을 것이다. 마찬가지 방법으로, 다시 한 주가 지나고 새 풀 백업을 받을 때에는 1번 테이프에 받도록 하고 6번 테이프를 보관하면 된다.

테이프가 6개 이상 있는 경우에는, 남는 테이프를 모두 풀 백업에 사용하도록 한다. 그리고 새로 풀 백업을 받을 때는 그 중에서 백업 받은 지 가장 오래된 테이프를 사용한다. 이렇게 하면 상당히 오래전의 풀 백업본을 가질 수 있게 되므로, 옛날에 지워진 파일들도 복구할 수가 있게 된다.

tar를 사용해 백업하기

tar를 사용하면 풀 백업을 쉽게 할 수 있다. # tar --create --file /dev/ftape /usr/src
tar: Removing leading / from absolute path names in the archive
#

위는 tar의 GNU 버전과 긴 이름 옵션을 사용한 예이다. 전통적인 tar는 원래 한 문자 옵션만을 인식한다. 또한 GNU tar는 한개 테이프나 플로피에 다 들어가지 않는 큰 용량의 백업도 다룰 수 있으며, 아주 긴 경로명도 사용할 수 있다. 이런 것들은 전통적인 tar에서는 할 수 없던 일들이다(리눅스는 GNU tar만을 사용한다).

만일 백업이 한 개 테이프에 다 들어가지 않는다면, multi-volume (-M) 옵션을 사용하면 된다: # tar -cMf /dev/fd0H1440 /usr/src
tar: Removing leading / from absolute path names in the archive
Prepare volume #2 for /dev/fd0H1440 and hit return:
#

플로피를 사용할 때에는 백업 받기 전에 꼭 포맷을 하여야 한다는 점을 주의하자. tar가 새 플로피를 요구할 때, 새 플로피를 넣고 다른 가상 터미널에서 포맷을 먼저 한 뒤 백업을 계속 해서 받을 수도 있다.

백업을 받고 나서는 그것이 제대로 되었는지 확인, 비교를 해야 한다. --compare (-d) 옵션을 사용하자. # tar --compare --verbose -f /dev/ftape
usr/src/
usr/src/linux
usr/src/linux-1.2.10-includes/
....
#

확인, 비교 과정에서 실패한 백업본을 그대로 방치한다면, 나중에 원본이 손상되고 난 후에야 그 백업본이 무용지물이었다는 사실을 깨닫게 될 것이다.

--newer (-N) 옵션을 사용하면, tar를 사용해 변경분 백업을 할 수 있다. # tar --create --newer '8 Sep 1995' --file /dev/ftape /usr/src --verbose
tar: Removing leading / from absolute path names in the archive
usr/src/
usr/src/linux-1.2.10-includes/
usr/src/linux-1.2.10-includes/include/
usr/src/linux-1.2.10-includes/include/linux/
usr/src/linux-1.2.10-includes/include/linux/modules/
usr/src/linux-1.2.10-includes/include/asm-generic/
usr/src/linux-1.2.10-includes/include/asm-i386/
usr/src/linux-1.2.10-includes/include/asm-mips/
usr/src/linux-1.2.10-includes/include/asm-alpha/
usr/src/linux-1.2.10-includes/include/asm-m68k/
usr/src/linux-1.2.10-includes/include/asm-sparc/
usr/src/patch-1.2.11.gz
#

아쉽게도, tar는 파일의 inode 정보(파일의 이름과 퍼미션의 변경같은 정보)가 변경된 것을 알아내지 못한다. 이 문제는 find를 사용해 지난번 백업의 파일리스트와 현재 파일시스템을 비교해 보는 방법으로 해결할 수 있는데, 이런 일을 해주는 스크립트와 프로그램들을 리눅스 ftp 사이트에서 구할 수 있다.

tar를 사용해 파일 복원하기
tar의 --extract (-x) 옵션을 사용하면 파일들을 추출해 낼 수 있다. # tar --extract --same-permissions --verbose --file /dev/fd0H1440
usr/src/
usr/src/linux
usr/src/linux-1.2.10-includes/
usr/src/linux-1.2.10-includes/include/
usr/src/linux-1.2.10-includes/include/linux/
usr/src/linux-1.2.10-includes/include/linux/hdreg.h
usr/src/linux-1.2.10-includes/include/linux/kernel.h
...
#

또한 커맨드 라인 상에서 이름을 명시해 주면, 특정 파일들과 디렉토리들을(그 안의 파일들과 하위 디렉토리를 포함해서) 빼낼 수가 있다. # tar xpvf /dev/fd0H1440 usr/src/linux-1.2.10-includes/include/linux/hdreg.h
usr/src/linux-1.2.10-includes/include/linux/hdreg.h
#

백업본에 어떤 파일이 들어있는지 보기만 하려면 --list (-t) 옵션을 쓰면 된다. # tar --list --file /dev/fd0H1440

```
usr/src/
usr/src/linux
usr/src/linux-1.2.10-includes/
usr/src/linux-1.2.10-includes/include/
usr/src/linux-1.2.10-includes/include/linux/
usr/src/linux-1.2.10-includes/include/linux/hdreg.h
usr/src/linux-1.2.10-includes/include/linux/kernel.h
...
#
```

tar는 백업본들을 순서대로만 읽기 때문에, 큰 백업본을 다루기엔 좀 느리다. 더구나 테이프 드라이브 같은 순차적 저장 장치들은, 원천적으로 랜덤 억세스 데이터베이스 테크닉을 사용할 수가 없다.

또한 tar는 지워버린 파일들을 제대로 다루지 못한다. 만약 풀 백업본 하나와 변경분 백업본 하나를 가지고 복원 작업을 한다고 했을 때, 두 백업본 사이에 지워버린 파일이 있다면 그 파일은 다시 복원되어 나타나게 된다. 이렇게 꼭 지워졌야만 하는 민감한 파일까지도 다시 복원된다는 사실은 큰 문제라고 할 수 있다.

다단계 백업

앞서 살펴본 단순 백업 방식은 개인적인 용도나 작은 규모의 사이트에서 사용하기에 좋다. 그러나 좀 더 중요한 업무를 다루는 곳이라면, 다단계 백업(Multilevel Backup)을 사용하는 것이 보다 알맞다.

단순 백업 방식은 풀 백업과 증가분 백업이라는 두 가지 레벨을 사용하고 있는 셈인데, 이것은 좀 더 많은 수의 레벨로 얼마든지 확장될 수 있다. 풀 백업을 레벨 0이라고 한다면, 각각 서로 다른 단계의 변경분 백업은 레벨 1,2,3,...이라고 할 수 있다. 각각의 백업 레벨에서는, 앞서 이루어진 백업 이후의 모든 변경 사항을 계속 백업하게 된다.

이런 다단계 백업을 하는 이유는, 좀 더 적은 비용을 들이면서도 백업 보장기간(backup history)을 길게 늘리기 위해서이다. 앞서 살펴본 단순 백업의 경우에, 백업 보장기간은 얼마나 오래전에 받아둔 풀 백업본이 남아 있느냐에 달려있다. 테이프가 많으면 그만큼 보장기간이 늘어날 수 있고, 이 경우에 있어서 기간을 한 주일 늘리려고 할 때마다 값비싼 테이프 하나를 더 사와야 한다. 백업에 있어서 그 보장 기간은 길수록 좋은데, 왜냐면 파일이 지워지거나 손상되었다는 사실은 아주 한참뒤에 깨닫게 되는 것이 보통이기 때문이다 이럴 때 복구할 수 있는 파일이 하나도 없는 것보다는 좀 옛날 파일이라도 남아 있는 것이 훨씬 좋을 것이다.

다단계 백업을 사용하면 훨씬 값싸게 백업 보장기간을 늘릴 수가 있다. 예를 들어, 테이프 10개를 샀다고 하자. 1번과 2번 테이프는 한달에 한번씩(매월 첫번째 금요일) 백업을 하는 데 쓰고, 3번에서 6번까지는 한주일에 한번씩(매주 금요일: 한달에 금요일이 5번 있다고 보면, 한번은 매월 백업을 하므로 4개의 테이프만 더 있으면 된다) 백업을 하는데 쓰도록 한다. 그리고 7번부터 10번까지는 하루에 한번씩(월요일부터 목요일까지 매일) 백업을 하는 데 쓰면 된다. 이런 방식을 통하면, 단지 4개의 테이프를 추가하는 것만으로도 백업 보장기간을 2주(10개의 테입을 모두 매일 백업하는데 쓴 경우)에서 2달로 크게 늘릴 수 있다. 이렇게 하면 2달 동안 매일매일의 파일 변경 사항을 모두 백업할 수는 없지만, 사실 이 정도면 파일을 복원하기에는 충분한 것이다.

Figure 10-1은 매일 어느 백업 레벨을 적용해야하는지, 그리고 매월 말일에는 어떤 백업본이 사용 가능한지를 보여주고 있다.

Figure 10-1. 다단계 백업 계획의 간단한 실례.

또한 백업 레벨을 사용하면 파일시스템을 복원하는데 드는 시간을 최소화할 수 있다. 만일 풀 백업 이후에 단순히 변경분 백업만을 계속한다면, 전체 파일시스템을 복원하기 위해서는 그 동안의 모든 백업본을 읽어들어야만 할 것이다. 그러나 백업 레벨을 사용한다면 파일을 복원하는데 필요한 백업본의 수를 훨씬 줄일 수 있다.

또한 파일을 복원하는데 드는 테이프의 수를 줄이기 위해서, 각각의 변경분 백업마다 좀더 낮은 수준의 백업 레벨을 적용할 수 있다. 반면에, 이렇게 하면 한번 백업을 받을 때마다 시간이 많이 걸리게 된다(각각의 백업본들이 앞선 풀 백업 이후의 모든 것을 다 백업해야 하므로). 좀더 나은 백업 기획안을 dump의 매뉴얼 페이지와 Table 10-1에서 볼 수 있는데, 보통 3,2,5,4,7,6,9,8,9,... 이런 식의 연속된 백업 레벨을 사용하게 되면 백업과 복원 모두에 걸리는 시간을 많이 줄일 수 있다. 또한 최근 이틀간의 작업 내용은 꼭 백업해 두어야 하며, 풀 백업의 간격을 길게 할 수록 복원에 드는 시간도 길어진다는 점을 주의하자.

Table 10-1. 많은 수의 백업 레벨을 사용한 효율적인 백업 기획안.

Tape Level Backup (days) Restore tapes
1 0 n/a 1
2 3 1 1, 2
3 2 2 1, 3
4 5 1 1, 2, 4
5 4 2 1, 2, 5
6 7 1 1, 2, 5, 6
7 6 2 1, 2, 5, 7
8 9 1 1, 2, 5, 7, 8
9 8 2 1, 2, 5, 7, 9
10 9 1 1, 2, 5, 7, 9, 10
11 9 1 1, 2, 5, 7, 9, 10, 11
... 9 1 1, 2, 5, 7, 9, 10, 11, ...

간단한 백업 기획안을 따른다면 품은 적게 들겠지만, 반면에 신경써야 할 부분이 많아지게 된다. 따라서 무엇을 버리고 무엇을 취할 것인지 결정하는 것이 중요하다.

dump는 이런 다단계 백업 지원을 내장하고 있다. tar와 cpio로 다단계 백업을 하려면 쉘 스크립트를 사용하여야 한다.

무엇을 백업해야 할 것인가
누구나 가능한 모든 것을 백업하고 싶어한다. 예외라면 재설치가 가능한 소프트웨어들은 보통 백업할 필요가 없는데, [1] 다만 그 설정 파일들은 나중에 다시 설정할 필요가 있다하더라도 꼭 백업해 두어야 한다. 또 하나의 예외는 /proc 파일시스템이다. 이 곳에는 커널이 언제나 자동으로 생성하는 데이터들이 위치하므로, 백업을 받아둘 필요는 절대 없다. 특히 /proc/kcore 파일이 쓸데없는데, 이것은 시스템의 물리적 메모리의 이미지이므로 크기도 무척 크다.

중요도가 어중간한 부분으로서는 뉴스 스풀 디렉토리와 각종 로그 파일들, /var 아래의 여러 파일들이 있다. 이들 중 무엇을 백업해야 할지는 여러분의 판단에 달려있다.

백업을 꼭 해야하는 가장 중요한 것은 각 사용자들의 개인 파일들(/home)과 시스템 설정 파일들(주로 /etc 아래에 있지만, 그밖에 많은 설정 파일들이 파일시스템 전역에 흩어져 있다)이다.

Notes
[1] 이런 것들을 백업하는 것이 더 편리한지 안하는 것이 더 편리한지는 의견이 제각각이다. 다만, 일일이 백업을 받아두는 것 보다는 플로피 수십장으로 재설치하는 것이 더 편리하다고 생각하는 사람들이 있다.

압축을 사용한 백업
백업은 큰 저장 용량을 필요로 하며, 따라서 돈이 많이 든다. 백업을 압축할 수 있다면 훨씬 비용이 싸게 먹힐 것이다. 이렇게 하는 데는 몇가지 방법이 있다. 어떤 프로그램은 압축 백업 지원을 내장하고 있기도 한데, 예를 들면 GNU tar의 --gzip (-z) 옵션은 백업 받는 내용을 gzip 압축 프로그램으로 파이프 연결시켜 준다. 그리고 gzip을 통해 압축된 내용이 백업 매체에 기록된다.

그러나 안타깝게도, 압축된 백업은 문제를 일으킬 소지가 있다. 압축이 이루어지는 근본 원리에 비춰보면, 전체 압축 데이터 중에서 단 하나의 비트만 손상되어도 다른 모든 데이터들이 쓸모 없게 되고 만다는 것을 알 수 있다. 어떤 백업 프로그램은 이런 문제에 대체하기 위한 자체 에러 수정 기능을 갖고 있기도 하지만, 그마저도 에러가 많이 발생하면 속수무책일 수 밖에 없다. 예를 들어, GNU tar를 써서 한덩어리의 압축된 백업본을 만들었다고 하자. 만일 여기서 딱 하나의 비트가 에러를 일으킨다면, 이 백업은 모두 쓸모없게 되고 만다. 백업은 신뢰성이 매우 중요한데, 이래서는 곤란하다.

한가지 대안은 각각의 파일을 따로 압축하는 것이다. 이렇게 하면, 파일 하나가 손상되었다고 해서 전체 백업을 모두 날려야하는 일은 없을 것이다. 결국 손상된 파일은 포기할 수 밖에 없지만, 그렇다고 해서 모든 파일을 압축하지 않는 것보다는 이 방법이 좀 낫다. afio 프로그램(cpio의 개정판)을 쓰면 이렇게 할 수 있다.

압축은 시간이 꽤 걸리는 작업이어서 테이프 드라이브에 즉시 데이터를 써넣기 힘들 수도 있는데, [1] 이 문제는 출력을 버퍼링함으로써 피할 수 있다(이런 문제를 자체적으로 해결할 수 있는 백업 프로그램도 있고, 다른 프로그램에 의존해서 처리하는 경우도 있다). 그러나 이런 일은 특별히 느린 컴퓨터에만 해당되는 문제일 것이다.

Notes
[1] 테이프 드라이브에 데이터를 바로바로 기록할 수 없다면, 드라이브는 자주 멈추어야 한다. 이렇게 되면 백업 작업도 더디게 될 뿐더러, 테이프와 드라이브에도 무리를 주게 된다.

Chapter 11. 시간 관리하기
Table of Contents
지역 시간대
하드웨어 시계와 소프트웨어 시계
시간 출력하기와 시계 맞추기
시계가 틀렸을 땐 어찌 하죠?
"Time is an illusion. Lunchtime double so." (Douglas Adams.)

"시간은 일종의 환영이다. 점심시간은 두배로 그렇고." (Douglas Adams: 영국의 SF 환타지 작가)

여기서는 리눅스 시스템이 시간을 어떻게 관리하는지, 그리고 시간 관련 문제의 발생을 막으려면 무엇을 해야 하는지를 알아본다. 보통, 시간에 대해선 특별히 신경 쓸 필요가 없겠지만, 그래도 이해를 해두는 것이 좋을 것이다.

지역 시간대
시간의 측정은, 행성의 자전으로 인한 밤낮의 바뀜과 같은 주기적인 자연 현상에 의해 이루어진다. 밤과 낮의 길이는 언제나 변하지만, 그 둘을 합친 시간은 일정하다. 특히, 그 중에서도 정오(noon) 시간은 일정한 기준이 된다.

정오라는 것은, 태양이 하늘에서 가장 높은 위치에 있을 때를 가리키는 용어이다. 그런데 지구는 둥글게 생겼으므로, [1] 정오 시간은 지역에 따라 다르게 된다. 여기서부터 나온 개념이 바로 지역 시간(local time)라는 것이다. 사람은 다양한 방법으로 시간을 측정하는데, 모두 정오와 같은 자연 현상에 의존한다. 만일 같은 장소에 계속 머무른다면, 지역마다 시간이 달라지는 것에 별로 신경쓸 필요가 없을 것이다.

그러나 멀리 떨어진 장소와 커뮤니케이션을 하기 위해서는, 어떤 표준적인 시간 개념이 필요하다는 점을 알 수 있다. 그래서 전세계적인 표준 시간을 정하게 되었는데, 이것을 세계 표준시간(universal time 또는 UT, UTC라고 부르며, 그리니치의 지역 시간을 기준으로 삼았기 때문에 예전에는 그리니치 표준시간(Greenwich Mean Time,GMT)이라고 불렀었다)이라고 한다. 따라서 다른 지역과 커뮤니케이션 할 때는, 이 세계 표준시간으로 시간을 표현하여야 혼란이 없게 된다.

반면에, 지역시간은 지역 시간대(time zone)에 따르게 된다. 실제로는 작은 범위 내에서도 시간이 조금씩 다 다를테지만, 그러면 너무 불편하므로 지역 시간대란 개념으로 묶어 놓은 시간을 쓰게 되는 것이다. 또한 이런 지역 시간대는 서머 타임(daylight saving, 일광 절약시간) 같은 정책적인 요인에 의해서 조정이 되기도 한다. 서머 타임이란 것은, 낮이 긴 여름 기간에 시간을 좀 앞당겨서 효율적인 시간 배분을 하는 것인데, 이런 규정은 나라마다 다른데다가 매년 바뀌기도 한

다. 이런 점은 지역 시간대를 환산하는 일을 까다롭게 만든다.

지역 시간대의 이름은, 보통 그 위치를 참고하여 짓거나 세계 표준시와의 시간 차이를 참고하여 짓게 된다. 미국을 포함한 여러나라들은 지역 시간대의 이름을 영단어 약자 세개로 나타내기도 하는데, 이런 약자들은 고유한 것이 아니므로 꼭 나라 이름과 함께 사용되어야 한다. 또한, 예를들어 헬싱키의 지역시간을 말한다고 할 때, 헬싱키가 동부 유럽 지역에 있다고 해서 이것을 `동부 유럽 시간(East European time)'이라고 하는 것은 별로 좋지 않으며 그냥 `헬싱키 시간'이라고 하는 것이 좋은데, 이것은 실제로 동부 유럽의 많은 국가들이 서로 다른 지역 시간을 사용하고 있기 때문이다.

리눅스는 모든 지역 시간대의 정보를 담고 있는 time zone 패키지를 갖고 있으며, 시간대 규정이 변경되었을 때 쉽게 갱신할 수도 있도록 되어 있다. 모든 시스템 관리자들은 적합한 지역 시간대를 선택해 두어야 하며, 또한 각각의 사용자들도 자신의 시간대를 지정해 두어야 한다. 보통 많은 사람들이 국가간 인터넷을 통해 협동 작업을 하기 때문에, 이 작업은 아주 중요하다. 만약 지역 시간대의 일광 절약시간 규정이 변경되었을 경우에는 리눅스 시스템의 time zone 부분을 업그레이드해야 한다는 점을 명심하자. 이렇게 시스템의 지역시간을 재설정하고 시간대 데이터 파일을 업그레이드하는 일만 주의한다면, 아마 시간에 대해선 크게 신경써야 할 일이 없을 것이다.

Notes
[1] 이것은 최근에 밝혀진 사실이다...-.-

하드웨어 시계와 소프트웨어 시계
개인용 컴퓨터는 보통 수은전지나 충전지로 작동되는 하드웨어 시계를 갖추고 있다. 이런 시계는 전지로 작동되기 때문에, 컴퓨터에 전기가 공급되지 않더라도 계속 움직이는 것이 가능하다. 하드웨어 시계는 BIOS 셋업 화면에서 조정할 수도 있고, 운영체제를 통해서도 조정할 수가 있다.

리눅스 커널도 자체적인 시계를 돌리고 있는데, 이것은 하드웨어 시계와 관계없는 독립적인 것이다. 다만, 부팅될 때만 커널 시계를 하드웨어 시계에 맞추는데, 그 이후에는 서로 독립적으로 작동한다. 리눅스가 자체 시계를 쓰는 이유는, 하드웨어 시계가 느리게 가거나 오작동할지 모르기 때문이다.

커널 시계는 언제나 세계 표준시(universal time)를 출력한다. 따라서 커널은 지역 시간대에 대해선 알 필요가 없다. 이런 단순성은 신뢰도를 높여주며 지역 시간대 정보를 업데이트하기도 쉽게 해준다. 따라서 각각의 프로세스들은 지역 시간대로의 변환을 스스로 해야한다(이런 작업은 time zone 패키지의 표준 설비들을 사용해 이루어진다).

하드웨어 시계는 세계 표준시간에 맞춰 둘 수도 있고 지역시간에 맞춰 둘 수도 있다. 보통 세계 표준시에 맞춰두는 것이 편리한데, 왜냐면 일광 절약시간 같은 것에 신경쓸 필요가 없기 때문이다(세계 표준시간에는 일광 절약시간 같은 것이 없다). 그렇지만, 불행하게도 어떤 PC 운영체제 -- MS-DOS, Windows, OS/2 같은 것 -- 들은 하드웨어 시계가 지역 시간에 맞춰져 있는 것으로 가정하기도 한다. 리눅스는 지역 시간에 맞춰진 하드웨어 시계도 다룰 수 있지만, 이렇게 했을 경우에는 일광 절약시간이 시작하고 끝날 때마다 시계를 다시 맞춰주어야만 한다(안그러면 시간이 틀리게 된다).

시간 출력하기와 시계 맞추기
데비안 시스템에서는, /etc/localtime이라는 심볼릭 링크가 어디에 걸려있느냐에 따라 시스템 time zone이 결정된다. 이 링크는 지역 시간대 정보가 있는 time zome 데이터 파일로 링크되어 있으며, time zone 데이터 파일은 /usr/lib/zoneinfo에 위치해 있다. 아마 다른 리눅스 배포본은 이와는 좀 다를 것이다.

사용자들의 개별적인 지역 시간대는 TZ 환경변수를 통해 설정할 수 있다. 이 환경변수가 설정되어 있지 않다면 해당 시스템의 지역시간대가 적용된다. TZ 변수의 설정 방법은 tzset 매뉴얼 페이지에 설명되어 있다.

현재 시간을 알려면 date 명령을 쓴다. 예를 들어, [1] $ date
Sun Jul 14 21:53:41 EET DST 1996
$

이렇게 나온다면, 현재 시간은 1996년 7월 14일이며 대략 밤 10시 10분전 쯤 된 것이다. 그리고 이 시간은 `EET DST'(이것은 동부 유럽 일광 절약시간일 것이다)라는 지역시간대에 따른 것이라는 점도 알 수 있다. 또한 date 프로그램은 세계

표준시로도 시간을 알려줄 수 있다. $ date -u
Sun Jul 14 18:53:42 UTC 1996
Sun Jul 14 18:53:42 UTC 1996
$

date는 커널의 소프트웨어 시계를 맞추는 데도 사용할 수 있다. # date 07142157
Sun Jul 14 21:57:00 EET DST 1996
date
Sun Jul 14 21:57:02 EET DST 1996
#

date 명령은 사용법이 좀 헷갈리기 쉬우므로, 좀더 상세한 내용은 해당 매뉴얼 페이지를 살펴보기 바란다. 시간은 오직 root만이 변경할 수 있고, 각 사용자들은 단지 자신의 시간대만 변경할 수 있을 뿐이다. 그러므로 결국 모두 같은 시계를 보고 있는 셈이다.

date는 단지 소프트웨어 시계만을 다룰 수 있다. clock 명령은 하드웨어 시계와 소프트웨어 시계를 동기화 시켜주는데, 이것은 부팅때 하드웨어 시간을 읽어서 소프트웨어 시계를 맞추는 데 쓰인다. 만일 두가지 시계를 모두 맞춰야 한다면, 우선 date로 소프트웨어 시계를 맞추고 clock -w 명령으로 하드웨어 시계를 소프트웨어 시간에 맞추면 된다.

clock에 -u 옵션을 쓰게 되면 하드웨어 시계가 세계 표준시에 맞춰져 있는 것으로 간주하게 된다. 따라서 -u 옵션은 주의해서 사용해야 한다. 안그러면 컴퓨터가 시간을 상당히 헷갈려할 것이다.

또한 유닉스 시스템의 많은 구성요소들은 시계를 보고 일을 처리하므로, 시간을 변경할 때는 주의를 기울여야 한다. 예를 들어 cron 같은 것은 명령을 주기적으로 실행시키는 데몬인데, 만일 시간을 바꾸게 되면 cron은 언제 명령을 실행시켜야 하는지 혼란스럽게 된다. 유닉스 시스템 초창기에는, 누군가가 시계를 12년 미래로 맞춰놓는 바람에 cron이 12년간 할일을 한번에 해내느라고 버벅여야만 했던 적이 있었다. 물론 지금의 cron은 이런 문제가 없지만, 그래도 역시 주의하여야만 한다. 특히 너무 먼 미래로 시간을 바꾸거나, 과거로 시간을 돌려놓는 일은 아주 위험하다.

Notes
[1] time 명령과 혼동하지 말자. 이것은 현재 시간을 보여주는 명령이 아니다.

시계가 틀렸을 땐 어찌 하죠?
리눅스 소프트웨어 시계는 사실 언제나 정확하다고는 할 수 없다. 이 시계는 단지 PC 하드웨어가 주기적으로 발생시켜 주는 타이머 인터럽트(timer interrupt)에 의존하고 있기 때문이다. 만일 시스템에 과부하가 걸려 있다면 타이머 인터럽트를 처리하는 시간도 지연될 수 밖에 없고, 결국 시계가 느리게 가게 된다. 그러나 하드웨어 시계는 독립적으로 작동하므로 비교적 정확하다. 따라서 부팅을 자주하는 컴퓨터(서버 역할을 하지 않는 대부분의 컴퓨터들)라면, 시계가 비교적 잘 가고 있다고도 볼 수 있다.

하드웨어 시계를 맞추고 싶다면, 리부팅한 후 BIOS 셋업 화면으로 들어가서 하는 방법이 보통 간단하긴 하다. 또한 이 방법을 쓰면, 시스템 시간 변경으로 인해 일어날 수 있는 문제들을 피할 수 있다. BIOS를 통해 시계를 맞출 수 없다면 일단 date와 clock을 사용해 하드웨어 시계를 맞춰보고 만일 시스템이 이상하게 동작한다면 즉시 리부팅할 수 있도록 한다.

네트워크에 연결된 컴퓨터라면(모뎀으로 연결된 경우라도), 다른 컴퓨터와 자신을 비교해서 시계를 맞출 수 있다. 상대편 컴퓨터가 아주 정확한 시간을 유지하고 있다면, 이쪽의 시계도 정확하게 맞출 수 있을 것이다. 이런 일은 rdate와 netdate 명령을 쓰면 할 수 있다. 이 명령들은 상대편의 시간을 체크해보고(netdate는 여러 컴퓨터들의 시간을 한꺼번에 비교해 볼 수 있다), 이쪽의 시계를 거기에 맞춰준다. 따라서 이런 명령들을 주기적으로 실행시킨다면 시간을 정확히 유지할 수 있을 것이다. (한국표준과학연구원 타임서버 주소가 time.kriss.re.kr이므로, rdate -s time.kriss.re.kr 이라고 하면 시간을 한국표준시에 정확히 맞출 수 있다.)

용어 해설 (초안)

"The Librarian of the Unseen University had unilaterally decided to aid comprehension by producing an Orang-utan/Human Dictionary. He'd been working on it for three months. It wasn't easy. He'd got as far as `Oook.'" (Terry Pratchett, ``Men At Arms'')

"아무개 대학의 도서관에서는 오랑우탄 말-사람 말 사전을 만들어 의사소통에 도움을 주기로 일방적인 결정을 내렸다. 그러나 이 일은 쉽지 않았다. 지금 3개월째 작업이 진행 중이다. 현재 `우우우욱'까지 했다고 한다. " (환타지 작가 Terry Pratchett, ``戰士'')

리눅스 시스템 관리에 관련된 용어들과 그 개념을 간략히 정리하였다.

ambition
야망 - 잘난척하는 글을 써내려 가면서, 그것이 리눅스 설명 파일에 끼워 넣어지기를 바라는 마음.

application program
응용 프로그램 - 어딘가에든 쓸모가 있는 프로그램. 아마 이런 프로그램을 쓰기 위해 컴퓨터를 샀을 것이다. system program과 operating system을 참고하기 바란다.

daemon
데몬(수호신) - 이 프로세스들은 뭔가 할일이 생길 때까지 백그라운드에 숨어 있으므로 눈에 잘 띄지 않는다. 예를 들면 update 데몬은 30초마다 버퍼 캐쉬를 디스크에 써넣는 역할을 한다. 또한 sendmail 데몬은 누군가가 전자 우편을 보냈을 때 비로소 활동을 개시한다.

file system
파일 시스템 - 디스크나 파티션 위의 파일을 추적하기 위한 데이터 구조이다. 즉, 운영체제가 디스크 위에 파일들을 편성해 넣는 방법을 말한다. 파일시스템은 각각의 디스크나 파티션마다 다르게 할 수 있다.

glossary
용어 해설 - 용어와 그 설명을 나열해 둔 것. 사전과는 다르므로 헷갈리지 말자.

kernel
커널 - 하드웨어를 통제하며 자원을 공유할 수 있게 해주는, 운영체제의 한 부분이다. system program을 참고하기 바란다.

operating system
운영체제 - 시스템의 자원(프로세서, 메모리, 디스크 공간, 네트워크 대역폭 등)을 사용자와 응용 프로그램들에게 배분하여 주는 소프트웨어이다. 또한 보안을 위해 시스템 접근을 통제하여 준다. kernel과 system program, application program을 참고하기 바란다.

system call
시스템 호출 - 응용 프로그램들에게 커널이 제공하여 주는 서비스들이다. 그리고 그런 서비스들을 불러내는 방법도 함께 제공한다. 매뉴얼 페이지의 두번째 섹션을 참고하기 바란다.

system program
시스템 프로그램 - 운영체제의 고수준 기능을 구현해 주는 프로그램들이다. 즉, 하드웨어에는 직접 의존적이지 않은 프로그램들이다. 보통, 이런 프로그램들에게는 특별한 권한이 필요하며(전자 우편을 배달하는 경우와 같이), 흔히 시스템의 일부분인 것으로 간주된다(예로 들면, 컴파일러는 보통 시스템의 일부분으로 간주된다). application program, kernel, operating system을 참고하기 바란다.